

**INTEGRATION OF VISUAL RESPONSE
IN TABLE TENNIS ROBOT**

Version 7

A

GROUP PROJECT

Submitted for fulfilment of

BACHELOR OF SCIENCE

IN

INFORMATION TECHNOLOGY (ROBOTICS)

Submitted by

J. Du Plooy	(CON-828169-S6R1)
B. B. Kirkpatrick	()
A. W. Mgwenya	()
T. Nhlapo	()

Under the guidance of

Gideon Ude Nnachi

Lecturer



DEPARTMENT OF INFORMATION TECHNOLOGY

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Background	1
1.3	Problem Statement	2
1.4	Research Questions	2
1.5	Research Objectives	3
1.6	Benefits of Study	4
1.7	Delimitation of Study	4
2	Literature Review	6
2.1	Introduction	6
2.2	Table Tennis	6
2.2.1	Physical Models:	6
2.2.2	3-D Reconstruction Framework:	7
2.2.3	Ping-Pong Ball Trajectory Prediction:	7
2.2.4	User experience:	8
2.3	Human-Machine Feedback loop	8
2.3.1	Overview	8
2.3.2	Foundation of human machine interaction	8
2.3.3	Machine learning architectures	9
2.3.4	Human machine feedback in learning environments	9
2.3.5	Limitations	9
2.4	Impact Dynamics Between Table Tennis Ball and Racket	10
2.4.1	Fundamental Mechanics of Frictional Impact	10
2.4.2	Viscoelastic and Frictional Behaviour	11
2.4.3	Empirical Influence of Rubber Type on Ball Velocity	11
2.4.4	Modelling Considerations and Parameter Identification	12
2.4.5	Experimental Measurement and Kinematic Reconstruction	12
2.4.6	Synthesis and Research Gaps	12
2.5	Trajectory Modelling	13
2.5.1	The basics	13
2.5.2	Physic Based Trajectory Modelling	13
2.5.3	Spin in Trajectory	14
2.5.4	AI Approach in Trajectory Prediction	14
2.6	conclusion	15

3	Modelling	23
3.1	Introduction	23
3.2	Camera capture	24
3.3	Ball detection 2D	25
3.3.1	Algorithm	25
3.4	3D reconstruction	26
3.4.1	Mathematical modelling	27
3.4.2	Numerical modelling	27
3.5	3D velocity, Acceleration and spin calculation	28
3.5.1	Mathematical modelling	28
3.5.2	Numerical modelling	30
3.5.3	Algorithm	31
3.6	Trajectory and Spin Prediction	33
3.6.1	Mathematical modelling	33
3.6.2	Numerical modelling	36
3.6.3	Algorithm	38
3.7	Simulation of Ball–Racket Impact	41
3.7.1	Mathematical modelling	41
3.7.2	Numerical modelling	44
3.7.3	Algorithm	46
3.8	Determination of racket orientation for target-constrained return	49
3.8.1	Mathematical modelling	49
3.8.2	Numerical modelling	51
3.9	Algorithm to convert target ball speed, spin and direction to motors	54
3.9.1	Mathematical modelling	54
3.9.1.1	Launcher Orientation	55
3.9.1.2	Wheel launcher model	55
3.9.1.3	Convert wheel speed to motor RPM	56
3.9.2	Algorithm	56
4	Design	58
4.1	Introduction	58
4.2	Design requirements	58
4.2.1	Requirements catalogue	58
4.3	Physical Design	61
4.4	Electrical Design	68
4.4.1	Electrical Overview	68
4.4.2	Power Distribution	70
4.4.3	Voltage Regulation	71

4.4.3.1	24V	71
4.4.3.2	5V	71
4.4.3.3	3.3V	72
4.4.4	Control Systems	72
4.4.4.1	Jetson Nano (Vision Processing)	72
4.4.4.2	STM32 (Real-Time Control)	72
4.4.4.3	UART Communication	73
4.4.5	Motor Drive System	73
4.4.6	Servo Actuation	73
4.4.7	Grounding	73
4.4.8	Safety	74
4.5	Vision Control Design	74
4.5.1	Software Environment Setup	74
4.5.2	Hardware	75
4.5.3	Headless Configuration	75
4.5.4	Remote Communication	75
4.5.5	Design Significance	76
4.6	Computational Software Design	76
4.6.1	Overview and Relationship to the Modelling Chapter	76
4.6.2	Computational Requirements and Design Constraints	77
4.6.3	Repository Architecture and Configuration	78
4.6.4	Pipeline Orchestration and Concurrency Model	79
4.6.5	Perception Subsystem	80
4.6.5.1	Ball Detection: YOLO + HSV Hybrid	80
4.6.5.2	Monocular 3-D Reconstruction	82
4.6.5.3	Kalman Filter State Estimator	83
4.6.5.4	Spin-Axis Estimation	84
4.6.6	Physics and Prediction Subsystem	84
4.6.6.1	Aerodynamic Trajectory Simulation	84
4.6.6.2	Rigid-Body Impact Simulation	85
4.6.7	Control Subsystem	86
4.6.7.1	Racket Orientation Solver	87
4.6.7.2	Launcher Command Generation	87
4.6.7.3	Serial Interface to the Motor MCU	89
4.6.8	Visualisation and Headless Operation	91
4.6.9	Error Handling and Failure Modes	91
4.6.10	Verification Strategy	93
4.6.11	Summary and Known Limitations	93
4.7	Microcontroller Software Design	94

4.7.1	STM32 Implementation	94
4.7.2	STM32 Cube MX	94
4.7.3	System schematic	94
4.7.4	Hardware setup	95
4.7.5	STM32CubeIDE Development environment	96
4.7.6	Motor control firmware implementation	96
4.7.7	Uart communication with STM32	97
4.8	Conclusion	97
5	Simulation	99
5.1	Introduction	99
5.2	Definition of Testing Objectives and Scope	100
5.2.1	Testing Objectives	100
5.2.2	Scope and Delimitations	101
5.3	Test Cases and Scenarios	102
5.3.1	Component-Level Test Cases	102
5.3.2	Integration Scenarios	103
5.4	Simulation-Based Testing	104
5.4.1	Simulation Environment	104
5.4.2	Physics Ground-Truth Validation	105
5.4.3	Magnus Effect Verification	106
5.4.4	Launcher Characterisation	109
5.4.4.1	Speed Control	109
5.4.4.2	Placement Repeatability	109
5.4.4.3	Head Positioning Accuracy	110
5.4.5	Vision Gazebo Integration	111
5.4.5.1	Ball Detection Rate (TC-03)	111
5.4.5.2	Kalman Filter Accuracy (TC-04)	111
5.4.5.3	Trajectory Prediction Accuracy (TC-05)	112
5.4.5.4	Spin Estimation (TC-06)	112
5.4.6	Automated Ball Placement Evaluation	112
5.4.6.1	Landing Accuracy	113
5.4.6.2	Spin-Induced Landing Shift	115
5.4.6.3	Repeatability	116
5.4.6.4	Intelligent-Mode Return Performance	117
5.5	Hardware-Based Testing	119
5.5.1	Renode Emulation Environment	119
5.5.2	UART Communication Testing (TC-09)	120
5.5.3	Motor Speed Tracking (TC-10)	121

5.5.4	Safe-Stop Response (TC-11)	122
5.5.5	Renode Gazebo Integration	123
5.6	Execute Tests and Analyse Results	123
5.6.1	Evaluation Method Overview	123
5.6.1.1	Vision System Evaluation	124
5.6.1.2	Tracking and Trajectory Evaluation	124
5.6.1.3	Communication System Evaluation	124
5.6.1.4	Complete System Evaluation	126
5.6.2	Test Execution Procedure	126
5.6.3	Integration Scenario Execution	127
5.6.3.1	Scenario A: Topspin Return	127
5.6.3.2	Scenario B: Spin Adaptation	127
5.6.3.3	Scenario C: Placement Targeting	128
5.7	Results	128
5.7.1	Summary of Findings	128
5.7.2	Performance Metrics	129
5.7.3	Automated Evaluation: Key Findings	129
5.7.4	Issues and Anomalies	130
5.8	Conclusion	131
6	Conclusion	132
6.1	Project summary	132
6.2	Evaluation of the Robotic System	132
6.3	Lessons Learned	133
6.4	Recommendations	133
6.5	Closing Remarks	133
A	Modelling Scripts	134
A.1	data gathering	134
A.1.1	Camera Capture	134
A.1.2	Ball Detection	135
A.1.3	3D Reconstruction	136
A.1.4	Spin estimation	137
A	Work Breakdown	140
	Bibliography	145

List of Figures

2.1	Schneider et al. (2022)	15
3.1	Pipeline Flowchart	24
3.2	Force Diagram	28
3.3	Kinematics Diagram of ball-racket impact	41
3.4	Ball launcher vector diagram, Dittrich et al. (2022)	54
4.1	Physical Design	63
4.2	Camera mount	64
4.3	Drawing 1/2 (Camera Mount)	64
4.4	Drawing 2/2 (Camera Mount)	65
4.5	Drawing 1/2 (angle adjustment)	66
4.6	Drawing 2/2 (angle adjustment)	66
4.7	Main power distribution	69
4.8	Control system, motor drive system, and logic system	70
4.9	Power Distribution	71
4.10	Physical hardware Setup	75
4.11	System schematic	94
4.12	MC workbench	95
4.13	Power board	95
4.14	T-motor Antigravity brushless motor	96
4.15	STM32CubeIDE	96
4.16	Uart configuration	97
5.1	High-level simulation architecture. Commands flow from the Control Panel (left) through the Renode STM32 emulator (centre-left) to the Gazebo physics world (centre-right). In intelligent mode the Jetson pipeline (right) processes camera frames, predicts the ball trajectory, and feeds RPM commands back to the Control Panel, closing the perception-to-actuation loop.	100
5.2	Simulation environment during a test session. <i>Left:</i> Gazebo Sim 10 showing the ITTF-regulation table tennis table, net, and the three-wheel launcher robot at the far end. <i>Right:</i> The Robot Control Panel (<code>scripts/control_panel.py</code>) showing live joint-state readouts, wheel-speed and head-angle sliders, mode controls (manual / intelligent / Renode mirror), ball-launch buttons, and the camera feed from the robot-mounted sensor.	105

5.3	TC-01: Gazebo physics vs. analytical aerodynamic model for a 10% top-spin shot ($C_d = 0.40$, $C_l = 0.60$, $\rho = 1.225 \text{ kg/m}^3$). Both curves are clipped at $z = 0.87 \text{ m}$ to avoid bounce-region artefacts. Peak height agreement is 0.04%, confirming the Gazebo <code>BallAerodynamics</code> plugin faithfully implements the published aerodynamic model.	106
5.4	Magnus effect verification: pre-bounce trajectory depth profiles for three topspin levels (same forehand start $\rightarrow$ robot-forehand target, repeat 0). Full topspin ($\omega_y = -393 \text{ rad/s}$) produces the shortest, steepest arc; the near-zero-spin 10% variant follows the longest, flattest arc. The depth shift between full and 10% topspin is 81 cm at the $z = 0.87 \text{ m}$ clip plane, consistent with the analytical prediction of additional Magnus downforce at full spin.	107
5.5	Ball altitude versus world depth for six spin variants (forehand start, robot-forehand target, repeat 0). Net height (0.9375 m) and table surface (0.785 m) are marked. All trajectories clear the net; full topspin produces the lowest, shortest arc while the 10% baseline follows a flatter, longer arc.	108
5.6	Violin plot of peak ball height by spin type across all 602 shots with detected bounces. The dashed red line marks net height (0.9375 m). All spin variants clear the net; the full-topspin distribution peaks lowest due to Magnus downforce suppressing the apex.	108
5.7	TC-02: Landing scatter for 8 repeated shots (mid-backhand start $\rightarrow$ robot-backhand target, quarter topspin). Bounce positions (blue dots) are tightly clustered with $\sigma_x = 0.8 \text{ cm}$ and $\sigma_y = 0.3 \text{ cm}$, demonstrating near-deterministic physics. The 63 cm systematic offset from the commanded target (red cross) reflects the known depth-calibration error at this start target pair and is consistent with the landing accuracy analysis of Section 5.4.6.	110
5.8	Box plot of Euclidean landing error by spin type. Boxes span the interquartile range; whiskers extend to $1.5 \times \text{IQR}$. Quarter and half topspin achieve the tightest distributions. The 10% baseline is compact but systematically offset from the target.	114
5.9	Ball landing positions (coloured dots) versus commanded targets (red crosses) for each spin variant. The robot side of the table ($x < 0$, shaded orange) is where all targets are located. Hook shots drift toward $+y$ (forehand) and fade shots drift toward $-y$ (backhand) due to sidespin Magnus interaction.	114
5.10	2-D density heatmap of detected bounce positions on the robot side of the table for each spin variant. Warmer colours indicate higher landing frequency; blue crosses mark commanded targets.	115

5.11	Mean landing shift (Δx depth, left panel; Δy lateral, right panel) for each spin variant relative to the 10% topspin baseline, plotted per target position index. Hook curves forehand (+y); fade curves backhand (−y). Pure-topspin variants shift only in depth.	116
5.12	Mean landing standard deviation with standard error bars, by spin variant. Five of six spin types meet the 5 cm repeatability target. Hook topspin exceeds the target due to sidespin drag coupling amplifying small inter-step timing differences over the ball’s curved path.	117
5.13	Euclidean landing error by player start position. Forehand and backhand extremes show slightly larger errors because the lateral launch velocity component is greatest at these positions, amplifying any calibration offset in the horizontal model.	117
5.14	<i>Left</i> : Spatial distribution of predicted return landing positions, coloured by spin type. <i>Right</i> : Net clearance versus return pitch angle; blue = in-target, red cross = out-of-target. The minimum clearance recorded was −0.6 cm (one ball clipped the net), corresponding to a near-flat 10% topspin return.	118
5.15	Distribution of net clearance (cm) for all 334 robot return shots, separated by spin type. Higher-topspin incoming balls require steeper return pitch angles, which increase mean net clearance.	119
5.16	TC-10: Motor step-response model for the pro-am topspin configuration (W1: 2 400 RPM / 500 ms ramp; W2: 4 800 RPM / 1 000 ms ramp; W3 feeder: 3 000 RPM / 800 ms ramp). Curves are derived from the MCSDK linear-ramp function with a 100 Hz FOC speed-loop model. T_{90} markers indicate 90% rise times. The Renode UART echo latency of 0.4 ms (shown in annotation) confirms the step command was accepted and registered in the firmware before the ramp would begin on physical hardware. Full FOC emulation requires TIM1/TIM8 support in the Renode platform file and is identified as future work.	122
5.17	Renode	125
5.18	Renode (stm32 firmware)	125
5.19	Python flowchart	126
A.1	Enter Caption	134

List of Tables

2.1	State Variables and Kinematics	19
2.2	Time Discretisation	19
2.3	Image and Vision Coordinates	19
2.4	3D Reconstruction (Camera Model)	19
2.5	Fundamental Physics	20
2.6	Aerodynamic Parameters	20
2.7	Composite Aerodynamic Constants	20
2.8	Discrete Numerical Variables	20
2.9	Spin Estimation	20
2.10	Impact Modelling Variables	21
2.11	Contact and Impact Physics	21
2.12	Inertia and Geometry	21
2.13	Post-Impact Variables	21
2.14	Trajectory Prediction	22
2.15	Control Variables	22
4.1	Requirements catalogue (functional, electrical, mechanical, software, and constraints). Implementing sections are noted in the category headers.	58
4.2	Performance requirements with target values and measurement methods.	60
4.3	Forward traceability from requirements to implementing design subsections.	61
4.4	Traceability map from Modelling chapter equations, through design subsections, to implementation modules.	77
4.5	Key parameters declared in <code>config.py</code> , grouped by semantic category.	79
4.6	Failure modes, detection, and response.	92
5.1	Scope matrix: performance requirement, test method, and simulation layer.	102
5.2	Component-level test cases.	103
5.3	Commanded wheel speeds versus measured exit speed. n = valid shots after outlier exclusion; error is relative to the analytically computed target.	109
5.4	Head tilt positioning accuracy. Commanded angle sent via <code>/robot_launcher/head_tilt/cmd</code> achieved angle read from joint state topic after 1 s settling window.	110
5.5	Landing accuracy by spin type. n = shots with detected bounce; error is Euclidean distance from commanded target position.	113
5.6	Mean landing shift relative to the 10% topspin baseline. $\Delta x > 0$ = shallower (closer to player); $\Delta y > 0$ = shift toward robot forehand.	115
5.7	Shot-to-shot landing standard deviation (σ) across 20 repeats. Lower σ indicates higher repeatability.	116

5.8	Intelligent-mode return performance from the 3 000-shot evaluation, broken down by spin type.	118
5.9	TC-09 UART communication test results. 100 valid and 20 edge-case packets (feeder pulse below nominal range) sent to Renode USART2 via TCP port 4444.	120
5.10	Verification matrix: performance requirements against measured results.	128
5.11	Simulation reality gap: modelling omissions and their expected impact on test results.	131

Chapter 1: Introduction

1.1 Introduction

Table tennis is one of the fastest and technically intense sports, where players have to process and react to balls travelling at high speeds, in addition to complex spins and placement variations. Traditional training methods and automated ball launchers have provided valuable forms of practice however, they lack the adaptability and quality of real rallies against professional human opponents. The robotics field has developed a number of sophisticated autonomous systems but there is a gap between these systems and athlete focused training systems. Although there have been several advances in autonomous systems, several sports training devices are open-loop and need contextual awareness. This constraint lessens their capability to duplicate the decision making dynamics implicit in actual competitive play. It is therefore necessary to have semi-autonomous systems that have the ability to perceive, interpret, and acknowledge human performance in an interrupted feedback cycle. This study aims to address this gap by developing a semi autonomous table tennis ball launcher. The system will use real time vision to track and analyse a players shot then make use of a simulation to model a possible blockers response. The system will then dynamically adjust the next launched ball's characteristics. By doing this, we aim to preform training that is highly responsive and restores the feeling of natural feedback as if it were a real rally.

1.2 Background

Table tennis has long been a proving ground for robotics research because of its demanding combination of speed, precision, and dynamic unpredictability. Human players read spin, velocity, and placement in fractions of a second; replicating even a portion of this responsiveness in a machine calls for tightly integrated sensing and control. Early work, such as the electromechanical launcher developed by Nemire et al. [Nemire et al. \(1991\)](#), illustrated the importance of controlled, repeatable trajectories for studying visual-motor coordination. Later designs, including low-cost mechanical launchers [Ponnusamy et al. \(2015\)](#), offered affordable means of projecting balls but lacked fine control over spin and flight.

Modern systems such as AIMY [Dittrich et al. \(2022\)](#) demonstrate that high-fidelity ball launching is achievable with careful mechanical design and differential wheel control. Meanwhile, research platforms addressing full robotic play—such as those exploring trajectory prediction [Koç et al. \(2018\)](#), spin-aware control [Yang et al. \(2021\)](#), and learning-based motion generation [Mülling et al. \(2013\)](#)—highlight how complex and nonlinear table tennis dynamics truly are. Commercial machines like AMICUS Prime

and Power Pong have become essential coaching tools, yet they remain preset-driven devices that reproduce stored drills rather than responding to a live player's behaviour. This project positions itself between these two worlds: not a fully autonomous returning robot, but not a static preprogrammed launcher either. Instead, it aims to capture some of the adaptability found in advanced research systems while keeping the mechanical and computational architecture approachable for real-world training use.

1.3 Problem Statement

Current training launchers, even those with rich programmability, operate independently of the player. They assume a one-directional flow of information: the robot fires, the player responds, and the robot continues its pre programmed script. Nothing about the shot of the users ball affects the next ball the machine delivers. This creates a fundamental disconnect between real rally conditions and machine-based practice. Human players naturally adjust the quality of their next shot based on what their opponent has done; spin, speed, and timing evolve dynamically throughout the rally. A traditional launcher cannot replicate that evolving conditions.

Research on ball-flight behaviour and spin effects [Lee et al. \(2019\)](#), [Yang et al. \(2021\)](#) shows that the characteristics of an incoming ball influences not only the shot required to return it, but also the response a hypothetical blocker would send back. By ignoring the player's actual strokes, commercial machines break the feedback loop that defines real table tennis exchanges. Even the most advanced devices remain rigid in timing and fixed in ball behaviour, failing to reflect what would happen if an opponent were present. The gap is not one of mechanical capability but of autonomy: high-quality hardware exists, yet the systems do not react.

The second issue concerns the realism of the shot progression. In drills such as "back-hand, pivot forehand, forehand wide", human play inherently carries lingering effects from each exchange. Spin decays, contact quality varies, and rhythm shifts as the rally evolves. Without incorporating these effects, machine training risks teaching patterns that do not translate cleanly to live play.

It follows that the central problem addressed in this study is:

How to introduce real-time adaptive feedback into table tennis ball launchers in order to emulate the dynamic continuity of human rally play?

1.4 Research Questions

Addressing the identified research problem, it is necessary to define a gap between static launch systems and adaptive reaction systems such that this study can focus

on specific research questions. These questions are designed to narrow potential research into manageable analytical components covering: tracking, modelling, system integration and evaluation. This structure allows for investigation in clearly defined research questions, ensures a uniform methodology and technical implementation.

1. How have past studies solved the human feedback problem?
 - How have past studies conducted trajectory analysis?
 - How have past studies conducted an analysis of object impact?
2. How can this study model the human feedback problem?
 - How can this study conduct a trajectory analysis?
 - How can this study model the impact of striking a ball?
3. How can this study implement a human feedback model in the context of table tennis ball launchers?
4. How can this study conduct an evaluation of the accuracy of the system?
5. What future work can be done?

1.5 Research Objectives

1. At the time of this study, there are no comprehensive studies that can be found that directly answer this question. However, a literature review can be conducted on the performance of table tennis ball launchers in general.
 - This study aims to conduct a literature review on trajectory analysis.
 - This study aims to conduct a literature review on object impact analysis.
2. By implementing an algorithm where the incoming ball state is the input representing a "human blocker" and the output is ball launching.
 - This study aims to model the trajectory of a table tennis ball accounting for the Magnus effect.
 - This study aims to model the impact that a table tennis ball has on a table tennis racket during a stroke.
3. The study uses appropriate software to not only simulate the robotic system, but also the surrounding environment, by doing this we can simulate gravity and other factors impacting the balls trajectory.

4. The study collects real world ball trajectories data and inputs it into the above mentioned simulation. Additionally, it also conducts a statistical analysis on the output data of the simulation.
5. Future work can be built on this system by increasing the overall accuracy of the system.

1.6 Benefits of Study

A semi-autonomous, simulation-informed launcher offers a training experience that bridges a long-standing divide between machine practice and real match conditions. By responding to the user's incoming shots, the robot restores the feedback loop that defines genuine rally play, something commercial launchers, despite their sophistication, do not attempt. The system does not aim to replace fully autonomous striking robots found in research, but instead to translate their insights on ball-flight modelling [Koç et al. \(2018\)](#), spin estimation challenges [Yang et al. \(2021\)](#), and launch accuracy [Dittrich et al. \(2022\)](#) into a practical, accessible form.

The approach benefits athletes by producing patterns that feel more organic and by adjusting rhythm and ball behaviour according to the quality of their strokes. It benefits engineering education and research by demonstrating how perception, simulation, and embedded control can be integrated into a cohesive, real-time system. Most importantly, it introduces autonomy not for the sake of complexity but for the sake of realism, preserving the simplicity of a launcher while granting it the responsiveness of a training partner.

1.7 Delimitation of Study

This study is intentionally confined to the design and simulation of a semi-autonomous table tennis ball launcher that adapts its behaviour based on real-time visual data of the player's preceding shot. The system is not designed to function as a fully autonomous table tennis opponent and does not include a robotic arm striking mechanism capable of returning balls physically.

The perception framework is limited camera based ball tracking and trajectory estimation. The system is not designed for high precision three-dimensional reconstruction, spin measurement through textured tracking, and multi-camera calibration techniques are explicitly excluded due to computational and hardware constraints. Trajectory is instead estimated through single camera tracking and machine learning techniques. Additionally, from the balls estimated trajectory spin is calculated through mathematical modelling.

The system's simulation component is constrained to modelling a simplified and consistent "blocker" response within predefined drill structures. The system does not attempt tactical reasoning, opponent modelling, reinforcement learning, or strategic decision-making. Adaptation is restricted to modulation of launch parameters—such as speed, spin, timing, and placement, within the boundaries of structured training patterns, colloquially known as "regular drills".

The control architecture prioritises deterministic, low-latency operation, with the assumption that outside of simulation it makes use of a Raspberry Pi for high-level logic and an STM32 microcontroller for motor control. Advanced optimisation techniques, adaptive control theory, and learning-based control algorithms are outside the scope of this study.

Furthermore, the project focuses on functional simulation and simulation performance evaluation rather than large-scale athlete trials or long-term training impact studies. User performance analytics and longitudinal skill assessment are therefore excluded from the present investigation.

These delimitations make sure that the study remains technically feasible while preserving the introduction of meaningful semi-autonomous adaptation within a traditionally static launcher framework.

Chapter 2: Literature Review

2.1 Introduction

Table tennis is one of the fastest racket sports in the world, known for its precision, unpredictability, and strategy. In this sport, points are won and mistakes are made within milliseconds. Despite the perceived simplicity of the game, the mechanics involved in physical body movements, trajectory manipulation, spin generation, and rebound control are extremely complex. These processes come from non-linear frictional impact dynamics, viscoelastic material behaviour ([Rinaldi et al. \(2019\)](#)), and human motor control operating under tight tolerances.

Advances in robotics and artificial intelligence enable the development of autonomous and semi-autonomous systems that interact with dynamic environments. Under the physical requirements for table tennis, any robotic table tennis or intelligent training platform needs accurate modelling of ball-racket impact for prediction, control, and adaptation. Purely automated systems lack robustness when encountering variability in material properties, environmental disturbances, or human interaction.

Conceptually, the human-machine feedback loop provides a framework for integrating human insight into automated decision-making. By combining human-in-the-loop strategies, intelligent systems can improve model accuracy, safety, and adaptability to ensure regulatory compliance and accountability [Romero-Obon et al. \(2025\)](#). In table tennis, this feedback loop may operate at multiple levels, namely equipment, robotic control systems, data-driven trajectory prediction, and performance optimisation.

This chapter first outlines the essential characteristics of table tennis as a dynamic physical system in section [2.2](#), followed by examining the principles of human-machine feedback and closed-loop control section [2.3](#). Section [2.4](#) presents a detailed literature review of the impact dynamics between table tennis balls and rackets, and section [2.5](#) covers the mechanical foundations of impact necessary to support both equipment optimisation and development of intelligent human-machine systems.

2.2 Table Tennis

2.2.1 Physical Models:

Experiences brought to you by a tennis table robot is versatility, while training using tools that offer a realistic and challenging training experience. Training partners are out the door because of the particular use of a player to improve technique, reaction time, and footwork without the need for a training partner [Büchler et al. \(2022\)](#). Capabilities

are different because these robots come in various models [Toussaint & Raison \(2026\)](#). A ball frequency table tennis robot is an example of the capabilities; the following is evidence. You can change the frequency at which the robot shoots the ball at you [Mat Sanusi et al. \(2021\)](#).

2.2.2 3-D Reconstruction Framework:

The 3D construction framework has a hardware setup consisting of camera synchronization. Various threads may not be executed depending on the resource allocation decision of the CPU at the time. Therefore, captured images by a camera at the time will not be simultaneous. The use of two industrial cameras are used to establish a 3D space. The robot arm coordinate system is converted into the coordinate system of the camera. Deriving the location of the 3D space from the camera system is the location of the object in 3D imagery. To track a ball, it must be encompassed by a full view of the table. The camera registering pixel scale, intrinsic matrix, and extrinsic matrix is the image projection. Intersecting the location of the target object are two vectors in the 3D space; this is the calculation of the 3D location. Error analysis is present when calculating the 3D location of the target object because the distance between the target object and the camera origins increases the coordinate systems [Lin et al. \(2020\)](#).

2.2.3 Ping-Pong Ball Trajectory Prediction:

The table tennis robot predicts where the ball is flying when it is hit towards it. Time stamps and location data are used for a parabolic trajectory. Expression of trajectory data through a regression model has several types. The types are found through independent variables and dependent variables with regard to the relationship between them. Establishing the appropriate mathematical equation for trajectory data is determined. To use ANN's to predict the polynomial coefficients of the ball trajectories, two parts of the ping pong ball trajectory can fit a polynomial. Coefficients of the polynomial express the trajectory effectively. Curve fitting was demonstrated through the inadequacy of curve fitting, the prediction performance of the ANN for the first trajectory, and predicting landing points of the first trajectory using the proposed methods, a quadratic polynomial resulting in curve fitting. To predict the first trajectory prediction procedure of this study, once the vision system receives the data from a few anterior positions of the ANN, the first trajectory can then be predicted. The trajectory before landing, the trajectory after landing, and ten anterior positions: The three things that are obtained from a complete flight trajectory [Lin et al. \(2020\)](#).

2.2.4 User experience:

Experiences come along with selecting a table tennis robot model; it is imperative to evaluate your training aspirations and the specific features that will undeniably suit your playing style [Emami et al. \(2024\)](#). The right robot can achieve a more effective and enjoyable training experience. Striking robot complexity is the central inquiry: designing a launcher capable of exhibiting behaviour as if it were participating in the rally.

2.3 Human-Machine Feedback loop

2.3.1 Overview

The rise of intelligent systems has opened up new opportunities for human-machine collaborations [Gómez-Carmona et al. \(2024\)](#). When talking about human machine feedback loops, we refer to the human-in-the-loop concept which we can describe as the process or system whereby humans actively take part in the operation, supervision or decision-making of an automated system, including the involvement of humans in the artificial intelligence workflow to guarantee accuracy, safety and ethical decision-making. Human-in-the-loop systems place human judgement and insight at the core of AI workflows, acting against unintended errors and biases [Turgunov et al. \(2025\)](#). The concept of human-in-the-loop essentially involves human insight in the ongoing feedback and interaction between machines and humans.

2.3.2 Foundation of human machine interaction

Human machine interaction can be described as the collaboration between machines and humans through interfaces. Customary user interfaces are the physical parts of machines, and in order for a user to be able to interact with them, the user has to undergo training in order to be able to operate the machines [Ke et al. \(2018\)](#). The normality of human machine interactions has outstandingly transformed our modern society. Modern AI systems rely on machine learning algorithms such as Convolutional neural networks(CNN), which uses three dimensional data for image classification and object recognition tasks and Transformers, which is a type of neural network architecture that processes sequential data [Glickman & Sharot \(2025\)](#) to identify complex patterns in huge datasets without requiring ample programming. Human feedback essentially corrects errors and reduces bias in machines and AI systems.

2.3.3 Machine learning architectures

We will now take a close look at the closed loop feedback mechanism, which is a system whereby the output is measured continuously to a reference value, and that error is used to modify the input. In recent studies, it was established that human machine feedback can also influence learner's learning styles and processes in a smart learning environment, this was done by implementing the random effects model in meta-analysis. The heterogeneity and main effect tests were used to assess the effect of human-machine feedback mechanism on the learning performance [Kumar et al. \(2024\)](#).

Recent approaches in artificial intelligence has switched focus on systems where machines adapt dependency on human feedback, but it's also important to acknowledge that although this is the case, simulated conversations with algorithm driven grammar and phrases do not grant chat bots the range for human insight and empathy. The cornerstone of human intelligence permits complex cognitive tasks spanning from logical deduction and programming to mathematical problem solving. The recent AI approaches have displayed the ability of large language models (LLMs) to exhibit emergent behaviours and reasoning abilities when scaled to adequate sizes [Wei et al. \(2022\)](#) which lead to the general use of pre-trained and automated large language models namely: Generative pre-trained transformer(GPT) and the Bidirectional Encoder Representations From Transformers(BERT). These models make use of translator architecture to generate human-like responses in response to prompts.

citeKaliisa02012026.

2.3.4 Human machine feedback in learning environments

Human machine feedback can be applied in systems like drone control, whereby the system captures hand gestures for intuitive and accurate control. An integrated tactile actuator array will then translate the posture of the drone into two-dimensional tactile information, hence enhancing the users perception of the flight situation. This closed loop system multimodal feedback and intuitive controls and reduces training time while improving the drone's environmental awareness, it's flight stability as well as it's posture [Yiu et al. \(2025\)](#).

2.3.5 Limitations

Although we have acknowledged that fully automated systems offer speed, efficiency and scalability, they also have significant limitations, mainly when dealing with com-

plex and ethically sensitive situations. These limitations can include slowed down performance and escalated costs as reviewers and decision-makers sometimes cannot match the speed of artificial intelligence especially when dealing with large scale applications. Another challenge would be the failure to minimize human bias as different people or reviewers interpret the same data differently, leading to inconsistencies in training data and decision making. To bypass this, organizations can use multiple human annotations to label data and accumulate the results to improve the quality of annotations [Kumar et al. \(2024\)](#).

The human feedback loop awards a foundational structure in modern robotics and intelligent systems. The concept has advanced to embody machine learning based adaptation and cognitive modelling. As robotic systems rapidly operate in communal environments, enhancing the human machine feedback loop remains essential in making sure that there is high performance interaction and safety.

2.4 Impact Dynamics Between Table Tennis Ball and Racket

2.4.1 Fundamental Mechanics of Frictional Impact

The interaction between a table tennis ball and racket during a stroke (from here on defined as ball-racket impact) is an extremely short and non-linear event that is dictated by coupled normal and tangential impulse exchanges. The post ball-racket impact ball state (linear velocity, angular velocity, and rebound direction) is dependent on the interaction between normal restitution, frictional traction, and rotational coupling.

Theoretically, this problem belongs to a broader class of rigid-body dynamics with unilateral contact and Coulomb friction. Classical rigid-body impact theory shows that frictional contact introduces discontinuities and non-smooth dynamics, resulting in a mathematically delicate system [Stewart \(2000\)](#). Large friction coefficients produce non-intuitive or paradoxical behaviour; additionally, consistent formulations require complementary or variational approaches as opposed to impulse approximations.

Similarly, in multi-body systems, different modelling strategies (rigid-body impulse models, compliant contact formations, hybrid approaches) produce materially different predictions of post impact states [Khulief \(2012\)](#). This implies that in the context of table tennis, the choice of impact model (purely elastic, partially elastic, compliant, viscoelastic, etc) determines predicted spin transfer, rebound speed, and energy dissipation.

Therefore, any usable model of ball-racket impact must include:

- Normal compliance and restitution behaviour,
- Tangential frictional interaction,
- Translational–rotational coupling,
- Potential rate dependence of material response.

2.4.2 Viscoelastic and Frictional Behaviour

Contrary to canonical rigid-body impact problems, the effective contact interface in table tennis consists of a racket. Colloquially, a table tennis racket is made up of a "blade" (the "wooden" part) between two "rubbers". A pip-in "rubber" (the type of rubber most commonly used) is made up of a top-sheet and sponge. The mechanical response of these materials is viscoelastic and dissipative. This means that restitution cannot be fully characterised by a single constant coefficient.

[Rinaldi et al. \(2019\)](#), has one of the most detailed investigations of this problem. Their work goes well beyond previous normal, no-spin impact studies by introducing oblique impacts with varying incident angles and spin. They show that a linear restitution coefficient is insufficient to characterise post racket-ball impact behaviour under realistic playing conditions. They even introduce additional physical metrics that describe the full reflected trajectory.

With particular importance, [Rinaldi et al. \(2019\)](#), developed a three-dimensional finite element model that includes time-dependent dissipative compliance measured through dynamic mechanical analysis and compression testing. Additionally, the results indicate that post ball-racket impact dynamics is dependent on:

1. Intrinsic viscoelastic properties of the polymer,
2. The friction coefficient at the ball–polymer interface.

2.4.3 Empirical Influence of Rubber Type on Ball Velocity

Experimental studies examining equipment performance provide complementary evidence. [Rusdiana & Ichsan \(2017\)](#) investigated the effect of different rubber types (inverted, pimped-out, and hard rubber) on maximal ball velocity during smash strokes. Using motion analysis, they report statistically significant differences in ball speed across certain rubber configurations, attributing variations to structural properties such as pips' orientation and thickness.

While this study primarily measures outgoing speed rather than full kinematic reconstruction (including spin), it supports the hypothesis that material configuration affects impact outcomes. However, because measurements were performed under athlete-driven stroke conditions, the results inherently combine biomechanical variability with contact physics. From a mechanics perspective, controlled laboratory impacts with fixed incident velocities and angles would provide clearer isolation of material effects.

2.4.4 Modelling Considerations and Parameter Identification

Theoretical impact frameworks [Stewart \(2000\)](#), [Khulief \(2012\)](#) emphasise that restitution and friction are not merely scalar constants but emerge from contact mechanics and material behaviour. In contrast, many reduced-order models used in practice assume constant effective parameters. The work of [Rinaldi et al. \(2019\)](#) suggests that such parameters may depend on impact speed, angle, and spin due to viscoelastic dissipation and frictional effects.

This raises an important modelling challenge: how to reconcile detailed finite element representations of polymer behaviour with computationally efficient rigid-body collision models suitable for simulation or robotic control. A systematic parameter identification methodology linking measurable material properties (e.g., DMA-derived moduli and friction coefficients) to effective restitution and tangential recovery parameters remains underdeveloped in the literature.

2.4.5 Experimental Measurement and Kinematic Reconstruction

Accurate experimental quantification of impact dynamics requires precise estimation of ball position and orientation before and after contact. The problem of estimating rigid-body transformations from corresponding point sets has been extensively studied, with several closed-form solutions compared by [Eggert et al. \(1997\)](#). Their analysis of singular value decomposition and eigenvalue system-based methods highlights considerations of accuracy, robustness to noise, and computational efficiency.

Although not specific to table tennis, such methodologies are essential for multi-camera trajectory reconstruction and motion capture systems used in impact studies. Errors in spatial registration propagate directly into estimates of restitution coefficients, spin transfer, and energy dissipation. Thus, rigorous calibration and transformation estimation are foundational to reliable experimental characterisation of ball–racket impact.

2.4.6 Synthesis and Research Gaps

The literature establishes several critical principles:

- Frictional rigid-body impact theory provides the mechanical foundation for understanding translational–rotational coupling [Stewart \(2000\)](#), [Khulief \(2012\)](#).
- Polymeric racket coatings exhibit viscoelastic and friction-dependent behaviour that strongly influences rebound dynamics [Rinaldi et al. \(2019\)](#).
- Rubber configuration measurably affects outgoing ball velocity [Rusdiana & Ichsan \(2017\)](#).
- Accurate experimental validation requires robust kinematic reconstruction methods [Eggert et al. \(1997\)](#).

However, significant gaps remain. There is limited availability of controlled experiments simultaneously measuring outgoing velocity and spin under systematically varied incident conditions. The dependence of effective restitution and friction coefficients on impact rate and angle requires further quantification. Finally, integration between detailed viscoelastic finite element models and reduced-order rigid-body impact formulations remains insufficiently developed.

Addressing these issues would enable a predictive, physics-grounded framework for optimising racket materials and understanding performance differences in table tennis impact dynamics.

2.5 Trajectory Modelling

2.5.1 The basics

In table tennis, a sport that is heavily influenced by speed and spin, it is challenging to analyse the trajectory of the ball. A major contributor to this is the lightweight state of the ball being used, which is approximately $2.7g$ and $40mm$ in size; [Zhang et al. \(2015\)](#), this makes it more prone to aerodynamic forces, specifically drag, and Magnus effect, as compared to heavier balls. Accurate trajectory prediction necessitates a multi-fidelity approach, integrating classical aerodynamics with modern state-estimation techniques. We focus on the ball's flight, estimating the ball's spin and AI prediction modelling to calculate the trajectory.

2.5.2 Physic Based Trajectory Modelling

Earlier research on ball trajectory discusses the fundamental forces that dictate flight path this is the founding model advanced predictive systems are based off of.

[Schneider et al. \(2022\)](#) shows a physics based analysis in the chapter "Table tennis and Physics" they use a hollow sphere and use molecular dynamics simulation to explain the balls high CoR. Coefficient of restitution (CoR) is defined as the ratio of velocity

after impact to velocity before impact. The CoR measures how much kinetic energy is conserved in a collision. A higher CoR the bouncier the ball with values reducing as impact speed increases due to energy loss. A high CoR is critical in table tennis as it insures consistent rebound speeds and heights this allows players to accurately predict the ball's trajectory and maintain rally control. The lower the score the less bounce the ball has The complete trajectory of the ball is determined by the total forces. These forces are as followed:

- Gravitational Force: A constant downward force acting on the ball that forces it toward the table.
- Drag Force: Air resistance that opposes the ball's motion. This force is influenced by the air density, speed and area.
- Lift Force (Magnus Effect): A force acting perpendicular to the direction of travel, created by the ball's spin. This changes the ball's parabolic path into a hard to predict path.

2.5.3 Spin in Trajectory

Spin is an important component that heavily impacts the trajectory of a ball. Spin impacts flight path enough to make it incredibly difficult to predict trajectory using position only vision systems. [Kusubori et al. \(2024\)](#). stated that for topspin the Theoretical Spin-Free (TSF) length was longer than observed shots, meaning the ball lands shorter. Backspin the TSF balls travelled a shorter distance this proves that backspin generates lift to extend flight. To account for these deviations [Zhang et al. \(2015\)](#) proposed a real time spin estimation system using the ball's logo. By capturing the 3D pose of the ball they were able to estimate the spin which are essential for accurate trajectory prediction using an Extended Kalman Filter (EKF).

Figure 2.1 shows the impact of spin on ball

2.5.4 AI Approach in Trajectory Prediction

While traditional models use Newtonian physics, newer systems have started to integrate artificial intelligence (AI) to work with real time complexities of spin. [Gomez-Gonzalez et al. \(2020\)](#) suggested a Deep Conditional Generative Model that maps partial trajectories to a Gaussian latent space. This model provides more accurate predictions with lower latency than traditional Recurrent Neural Networks (RNNs).

[Song & Lu \(2023\)](#) suggests the YOLO (You Only Look Once) and SORT (Simple Online and Real-time Tracking) algorithms to track the ball. this helps predict the path and landing point. By training on large datasets these models can have a higher rate of accurate predictions.

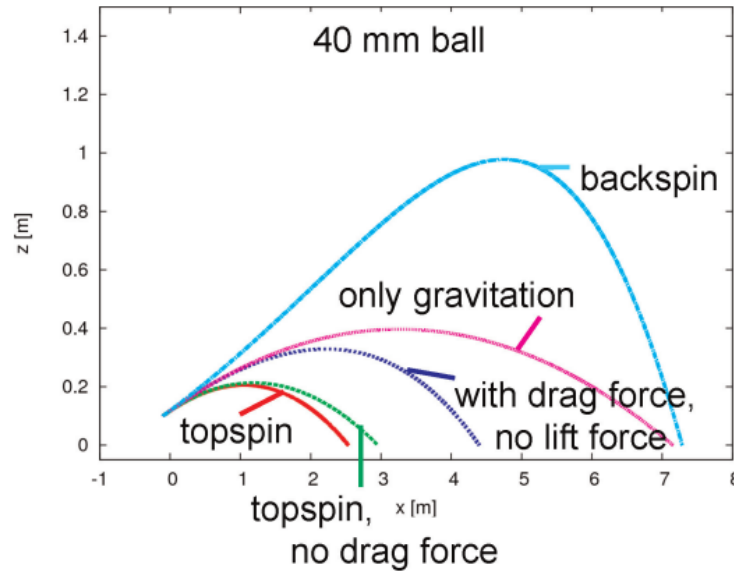


Figure 2.1: [Schneider et al. \(2022\)](#)

2.6 conclusion

In this literature review we establish that to develop an autonomous table tennis launcher we have to have an in depth understanding of aerodynamic modelling and high speed control theory. The table tennis ball launcher has evolved from simple double wheel systems to high quality 3 wheel systems like AIMY it is shown that fine tuned control over spin and speed is the prerequisite for realistically training the player.

An important point to understand from the research into impact dynamics and trajectory analysis is that a table tennis ball's flight path is not just a parabolic arc but a complicated arc manipulated by the deformation of the ball and fluid dynamics. We establish in the review that due to material nature and the Magnus effect that spin is the primary variable that decides the difficulty of the shot as well as the complexity of its prediction. The move towards more AI usage in real time trajectory estimation is a significant one as we can move away from static pre programmed gameplay into "real partner" simulation that can perceive and respond to each stroke a player makes.

However, a gap remains in this field between high end and expensive systems compared to consumer grade launchers. This project puts itself in the middle ground. The proposed system aims to emulate the human machine feedback loop. The integration of real time visual feedback with mechanical execution allows the launcher to act as more than just a ball thrower but instead becoming a reactive training system that simulates the unpredictability of a live rally. Moreover, the proposed system aims to standardize a perception control actuation pipeline in a closed loop framework by merging fast moving vision for estimating trajectory, aerodynamic modelling for predictive state cal-

culations and adaptive control for live motor regulation. Our launcher will operate as an interactive robotic system instead of a repeatable system.

Definitions

This section explains the main terms and variables used in the modelling chapter.

Trajectory: The trajectory is the path the ball follows in three-dimensional space over time. It depends on the ball's starting conditions and the forces acting on it.

Kinematics: Kinematics is the part of mechanics that describes motion such as position, velocity, and acceleration without looking at the forces that cause it.

Dynamics: Dynamics is the study of forces and how they affect motion. This includes gravity, drag, and forces caused by the ball's spin.

Acceleration: Acceleration is how quickly velocity changes over time. It is defined as

$$\mathbf{a} = \frac{d\mathbf{v}}{dt} \quad (2.1)$$

Gravity: Gravity is a constant acceleration that pulls the ball downward. It is usually given as $g = 9.81 \text{ m/s}^2$.

Drag Force: Drag force is a resistance that slows the ball as it moves through the air. It is usually related to the ball's speed and acts in the opposite direction of motion.

Magnus Effect: The Magnus effect is a force created by the ball's spin. It makes the ball curve because of pressure differences in the air around it.

Angular Velocity (ω): Angular velocity is a vector that shows how fast and around which axis the ball is spinning.

2D Image Coordinates: The 2D image coordinates are the pixel positions where the ball is found in the image, written as (u, v) .

3D Reconstruction: 3D reconstruction means turning 2D image coordinates into real-world 3D positions by using camera calibration and geometric methods.

Frame Rate: Frame rate is how many images the camera takes each second. It sets how detailed the timing is in system.

Computer Vision: Computer vision is the field that teaches machines to understand and get useful information from images and video.

Closed-Loop System: A closed-loop system uses feedback from its output to keep adjusting how it works in real time.

Prediction Model: A prediction model is a mathematical tool that estimates where the ball will go next, based on its current state.

Actuation: Actuation is turning control signals into actual movement in the robot.

Time Step (Δt): A time step is a small chunk of time used in simulations to update the system's state.

Assumptions

The modelling process is based on these assumptions:

- The table tennis ball is treated as a rigid body.
- Air density is constant throughout the motion.
- External disturbances such as wind are neglected.
- The camera system is properly calibrated and distortion is negligible.

Nomenclature

State Variables and Kinematics

Table 2.1: State Variables and Kinematics

Symbol	Description	SI Unit
x, y, z	Ball position in 3D space	m
v_x, v_y, v_z	Velocity components	m s^{-1}
$\mathbf{v}$	Velocity vector	m s^{-1}
$v = \ \mathbf{v}\ $	Speed (magnitude of velocity)	m s^{-1}
$\mathbf{a}$	Acceleration vector	m s^{-2}
t	Time	s
$\boldsymbol{\omega}$	Angular velocity vector	rad s^{-1}
$\mathbf{e}_\omega$	Unit spin direction vector	—

Time Discretisation

Table 2.2: Time Discretisation

Symbol	Description	SI Unit
Δt	Time step	s
f	Frame rate	Hz
t_i	Discrete time step	s
i, k	Index variables	—

Image and Vision Coordinates

Table 2.3: Image and Vision Coordinates

Symbol	Description	Unit
u, v	Pixel coordinates	pixels
(u_k, v_k, t_k)	Detected ball coordinates per frame	pixels, s

3D Reconstruction (Camera Model)

Table 2.4: 3D Reconstruction (Camera Model)

Symbol	Description	SI Unit
X, Y, Z	World coordinates	m
f_x, f_y	Focal lengths	pixels
c_x, c_y	Principal point	pixels
z	Depth from camera	m

Fundamental Physics

Table 2.5: Fundamental Physics

Symbol	Description	SI Unit
g	Gravitational acceleration	m s^{-2}
m	Mass of ball	kg
$\sum F$	Sum of forces	N

Aerodynamic Parameters

Table 2.6: Aerodynamic Parameters

Symbol	Description	SI Unit
ρ	Air density	kg m^{-3}
A	Cross-sectional area	m^2
C_D	Drag coefficient	–
C_L	Lift coefficient	–

Composite Constants

Table 2.7: Composite Aerodynamic Constants

Symbol	Description	SI Unit
$C_1 = \frac{\rho A C_D}{2m}$	Drag constant	m^{-1}
$C_2 = \frac{\rho A C_L}{2m}$	Magnus constant	m^{-1}

Discrete Numerical Variables

Table 2.8: Discrete Numerical Variables

Symbol	Description	SI Unit
$\mathbf{x}_i$	Position vector at step i	m
$\mathbf{v}_i$	Velocity at step i	m s^{-1}
$\mathbf{a}_i$	Acceleration at step i	m s^{-2}

Spin Estimation

Table 2.9: Spin Estimation

Symbol	Description	SI Unit
$\mathbf{e}_{\omega,i}$	Spin direction at step i	–
$\bar{\mathbf{e}}_{\omega}$	Averaged spin direction	–

Impact Modelling

Table 2.10: Impact Modelling Variables

Symbol	Description	SI Unit
$\mathbf{n}(\theta)$	Racket normal vector	–
r	Ball radius	m
$\mathbf{r}_{P/B}$	Contact vector	m
$\mathbf{v}_B$	Ball velocity	m s^{-1}
$\mathbf{v}_R$	Racket velocity	m s^{-1}
ω_B	Ball angular velocity	rad s^{-1}
ω_R	Racket angular velocity	rad s^{-1}

Contact and Impact Physics

Table 2.11: Contact and Impact Physics

Symbol	Description	SI Unit
J	Impulse	N s
J_n	Normal impulse	N s
J_t	Tangential impulse	N s
C_{res}	Coefficient of restitution	–
μ	Friction coefficient	–

Inertia and Geometry

Table 2.12: Inertia and Geometry

Symbol	Description	SI Unit
I_B	Moment of inertia	kg m^2
I	Identity matrix	–
I_t	Tangential projection matrix	–
K_t	Tangential mass operator	kg^{-1}

Post-Impact Variables

Table 2.13: Post-Impact Variables

Symbol	Description	SI Unit
$\mathbf{v}_B^+$	Post-impact velocity	m s^{-1}
$\mathbf{v}_B^-$	Pre-impact velocity	m s^{-1}
ω_B^+	Post-impact angular velocity	rad s^{-1}

Trajectory Prediction

Table 2.14: Trajectory Prediction

Symbol	Description	SI Unit
$\mathbf{x}_{bounce}$	Bounce position	m
$\mathbf{x}_T$	Target position	m
$J(\theta, \phi)$	Cost function	m ²

Control Variables

Table 2.15: Control Variables

Symbol	Description	SI Unit
θ, ϕ	Racket orientation angles	rad
V_R	Racket speed	m s ⁻¹

Chapter 3: Modelling

3.1 Introduction

The main objective in this section is to use real time vision detection, physical simulations and algorithms to predict a table tennis ball's trajectory and create a realistic returning shot based on the players previous shot.

We attempt to estimate the ball state vector $(x, y, z, v_x, v_y, v_z, \omega)$ and then predict the future trajectory using a physics model.

By creating a simulated "blocker"'s response the system turns static drills into responsive dynamic drills through the use of kinematic and motion planning.

The proposed model is of a closed loop system: input from camera data calculates trajectory and spin which then is put into an impact simulations and then outputs motor commands to a controller using a Raspberry Pi for vision systems and STM32 for motor control systems.

The main modules are vision tracking for positional data (t, x, y, z, v, ω) , aerodynamic modelling accounting for drag and Magnus effects, impact representation for racket-ball simulation and algorithms to convert the output values into motor speed.

The primary method we implement is use of python libraries such as OpenCV for data collection NumPy for mathematical calculations and Matplotlib for visualization. We define the main constants (e.g., ball mass, air density) and create the model in steps starting with data collection and ending with control outputs. Mathematical modelling focuses on creating equations for the robot's kinematics (e.g., position and velocity) and dynamics (e.g., force balances including gravity, drag, and lift).

To illustrate, Figure 3.1 shows a flowchart of the process. This approach ensures the model's predictive accuracy for tasks, such as adjusting spin and speed in response to players inputs.

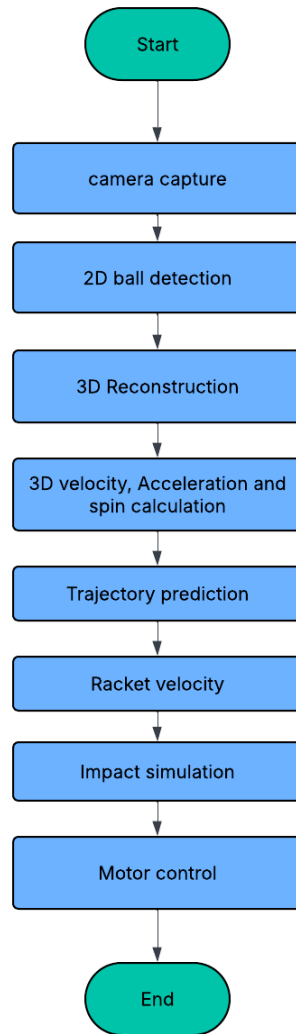


Figure 3.1: Pipeline Flowchart

3.2 Camera capture

The first step we have to make in the robotic system modelling is gathering the video data, for us to predict the trajectory and spin of the ball we must collect time based positional data that shows the movement of the ball through a 2D plane. The video capture step is of the utmost importance as the accuracy of the rest of the modelling processes depends on the quality and resolution of the captured footage. High frame rates are better because they give more data to reconstructing the trajectory of the ball.

3.3 Ball detection 2D

To gather 2D ball data from the now captured footage we use computer vision techniques frame by frame to extract the coordinates of the ball to output:

$$(t, u, v)$$

where: t is the time in seconds u is pixel coordinates in x v is pixel coordinates in y

We track a table tennis ball by combining spatial, temporal, and color based filters. We start by narrowing the search space through a masking three phased pre processing. We start with a Region of Interest (ROI) mask that restricts the search to a specific window. We then use a background subtractor that identifies moving pixels while removing ghosting by use of thresholding. Finally we use an HSV color filter that targets white pixels within the range $[0, 0, 180]$ to $[180, 75, 255]$. These masks are combined using a bitwise AND operation to ensure only pixels that pass all 3 are analysed further [TAMAKI & SAITO \(2013\)](#).

We then perform shape and geometry analysis on the results filtering by area to ignore noise and then using the formula for a circle circularity $= \frac{4\pi \cdot \text{Area}}{\text{Perimeter}^2}$ to check if the object is a ball.

To avoid false positives we use a scoring system that evaluates possible balls by multiplying their circularity and area and at the same time applying a penalty on the distance from a predicted pixel position. This drops the score of possible balls far from the expected trajectory. The final detection is mapped into a (t, u, v) coordinate system where (u, v) represents the horizontal and vertical pixel coordinates (x, y) extracted from the detection's centre, and t represents the time component,

3.3.1 Algorithm

This algorithm is used to locate the table tennis ball in each frame and output its pixel coordinates (u, v) . The algorithm uses motion based detection with filtering to isolate the ball from the background. Similar to [Ning et al. \(2023\)](#). The algorithm works like this:

1. Start the camera or and load the first frame.
2. Convert the frame to greyscale.
3. Do a background subtraction.
4. Apply a binary threshold to the motion mask.
5. Perform erosion and dilation to remove small noise.

6. Detect contours in the binary image.
7. For each detected contour:
 - Contour area
 - Bounding circle
 - Circularity
8. Filter the detected contours based on realistic ball properties:
 - Check radius is in the acceptable range.
 - Circular shape
 - Enough motion between frames
9. Select the contour that best matches and calculate its centre coordinates (u, v) .
10. Store the detected ball position together with the timestamp for later.
11. Repeat until the video ends.

The output is a sequence of detected ball positions:

$$(u_k, v_k, t_k) \tag{3.1}$$

where k represents the frame and t_k is the timestamp of the frame.

3.4 3D reconstruction

These measurements (t, u, v, w) are then converted into 3D coordinates and used as input for the physical modelling algorithms. The data collected are:

$$(t, x, y, z)$$

where:

t =time

x =Horizontal table axis

y =Vertical table axis

z =Vertical height above table

3.4.1 Mathematical modelling

To map a pixel (u, v) to a 3D coordinate we have to use the Pinhole Camera Model. The relationship between the 3D coordinates (X, Y, Z) and its pixel location (u, v) is defined by the intrinsic matrix:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{z} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \quad (3.2)$$

Where:

- f_x, f_y : Focal lengths (in pixels).
- c_x, c_y : Optical center (the Principal Point).
- Z : The depth (distance from the camera).

3.4.2 Numerical modelling

The camera captures frames at a fixed frame rate f that depends on the camera used to capture the video. The time interval between frames is then stated as:

$$\Delta t = \frac{1}{f} \quad (3.3)$$

e.g.: If the camera records at 60 frames per second it is then shown as:

$$\Delta t = 0.0167s \quad (3.4)$$

3.5 3D velocity, Acceleration and spin calculation

The motion of a spinning ball in flight cannot be directly describes with just kinematics, however, Newtonian mechanics can, to an extent, describe the ball's motion [Schneider et al. \(2022\)](#), [Lin et al. \(2020\)](#). The trajectory of a table tennis ball is influenced by gravity, aerodynamic drag, and the Magnus force generated by ball spin [Schneider et al. \(2022\)](#), [Kusubori et al. \(2024\)](#), [Lin et al. \(2020\)](#). The purpose of this section is to develop an analytical expression that can use the measurable data from the previous section to describe the spin vector. This derivation forms the foundation for a numerical algorithm that can be used plot the ball's trajectory [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#).

3.5.1 Mathematical modelling

Newton's second law states that the total force acting on a body equals the product of its mass and acceleration [Stewart \(2000\)](#), [Khulief \(2012\)](#).

$$\sum \mathbf{F} = m\mathbf{a} \quad (3.5)$$

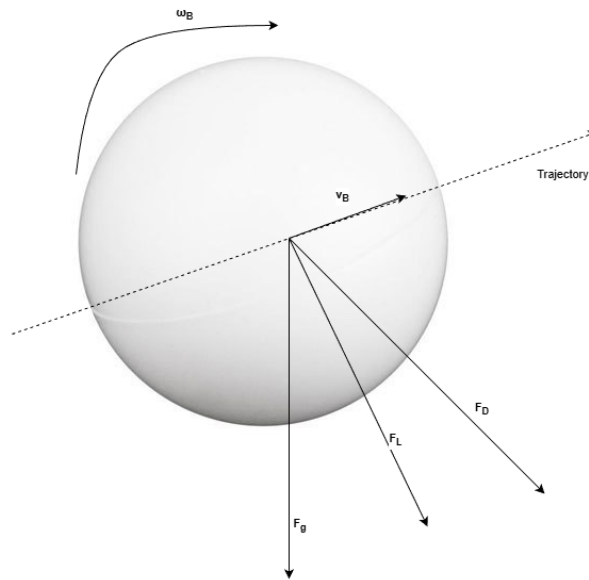


Figure 3.2: Force Diagram

As seen in figure 3.2 a spinning ball moving through air has three primary forces acting on the ball: gravity, aerodynamic drag, and the Magnus lift force caused by spin [Schneider et al. \(2022\)](#), [Kusubori et al. \(2024\)](#), [Lin et al. \(2020\)](#). By combining these

forces, the following equation of motion is revealed [Schneider et al. \(2022\)](#), [Lin et al. \(2020\)](#).

$$m\mathbf{a} = m\mathbf{g} - \frac{1}{2}\rho AC_D v\mathbf{v} + \frac{1}{2}\rho AC_L v (\mathbf{e}_\omega \times \mathbf{v}) \quad (3.6)$$

To simplify the expression and reduce the number of repeated constants, we define two composite aerodynamic coefficients that group the physical constants and ball properties.

$$C_1 = \frac{\rho AC_D}{2m}, \quad C_2 = \frac{\rho AC_L}{2m} \quad (3.7)$$

Substituting these constants into the equation of motion produces a simplified acceleration equation expressed entirely in vector form.

$$\mathbf{a} = \mathbf{g} - C_1 v\mathbf{v} + C_2 v(\mathbf{e}_\omega \times \mathbf{v}) \quad (3.8)$$

Next, the Magnus force term is isolated so that the spin direction vector can be solved explicitly in terms of measurable motion variables [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#).

$$C_2 v(\mathbf{e}_\omega \times \mathbf{v}) = \mathbf{a} - \mathbf{g} + C_1 v\mathbf{v} \quad (3.9)$$

Dividing both sides by the aerodynamic constant allows the cross-product term containing the spin direction to be isolated.

$$(\mathbf{e}_\omega \times \mathbf{v}) = \frac{1}{C_2 v}(\mathbf{a} - \mathbf{g} + C_1 v\mathbf{v}) \quad (3.10)$$

To eliminate the cross product containing the unknown vector, the equation is multiplied by the velocity vector using the vector cross-product identity.

$$\mathbf{v} \times (\mathbf{e}_\omega \times \mathbf{v}) = \frac{1}{C_2 v}\mathbf{v} \times (\mathbf{a} - \mathbf{g} + C_1 v\mathbf{v}) \quad (3.11)$$

Applying the vector triple product identity expands the left-hand side into terms containing scalar products.

$$\mathbf{e}_\omega(\mathbf{v} \cdot \mathbf{v}) + \mathbf{v}(\mathbf{v} \cdot \mathbf{e}_\omega) = \frac{1}{C_2 v}\mathbf{v} \times (\mathbf{a} - \mathbf{g} + C_1 v\mathbf{v}) \quad (3.12)$$

For most practical table tennis ball trajectories, the spin axis can be approximated to be perpendicular to the velocity vector, [Kusubori et al. \(2024\)](#), [Schneider et al. \(2022\)](#). Therefore, the scalar projection term is negligible and the velocity magnitude squared replaces the dot product term.

$$\mathbf{v}(\mathbf{v} \cdot \mathbf{e}_\omega) \approx 0, \quad \mathbf{v} \cdot \mathbf{v} = v^2 \quad (3.13)$$

Substituting this approximation into the previous equation yields a direct expression for the spin direction vector.

$$\mathbf{e}_\omega = \frac{1}{C_2 v^3}\mathbf{v} \times (\mathbf{a} - \mathbf{g} + C_1 v\mathbf{v}) \quad (3.14)$$

This final expression provides a physically meaningful relationship between the spin axis and measurable motion variables. It forms the analytical basis for estimating spin orientation from trajectory data.

3.5.2 Numerical modelling

The equations derived in the previous subsection are written in continuous form, where the position, velocity, and acceleration of the ball are functions of time. In practice, however, experimental measurements from video tracking or motion capture systems are available only at discrete time instants, [Aakanksha et al. \(2025\)](#), [Zhang et al. \(2015\)](#). It is therefore necessary to reformulate the governing equations in a discrete form so that the trajectory and speed of the ball can be computed numerically.

Assume that the trajectory is sampled at times

$$t_0, t_1, t_2, \dots, t_N \quad (3.15)$$

with constant time step

$$\Delta t = t_{i+1} - t_i. \quad (3.16)$$

At each time step t_i , the ball position is represented by the vector

$$\mathbf{x}_i = \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix}. \quad (3.17)$$

The first step in the numerical model is to approximate the ball velocity from successive position measurements. Using a forward finite-difference approximation, [Chapra & Canale \(2015\)](#), the velocity at time t_i is given by

$$\mathbf{v}_i \approx \frac{\mathbf{x}_{i+1} - \mathbf{x}_i}{\Delta t}. \quad (3.18)$$

The speed of the ball is then obtained as the magnitude of the velocity vector:

$$v_i = \|\mathbf{v}_i\|. \quad (3.19)$$

Next, the ball acceleration is approximated using the change in velocity between successive time steps. Again using a forward finite-difference approximation, the acceleration is written as

$$\mathbf{a}_i \approx \frac{\mathbf{v}_{i+1} - \mathbf{v}_i}{\Delta t}. \quad (3.20)$$

Substituting these discrete approximations into the continuous equation of motion,

$$\mathbf{a} = \mathbf{g} - C_1 v \mathbf{v} + C_2 v (\mathbf{e}_\omega \times \mathbf{v}), \quad (3.21)$$

gives the discrete form

$$\mathbf{a}_i = \mathbf{g} - C_1 v_i \mathbf{v}_i + C_2 v_i (\mathbf{e}_{\omega,i} \times \mathbf{v}_i). \quad (3.22)$$

Rearranging this expression to isolate the Magnus-force contribution gives

$$C_2 v_i (\mathbf{e}_{\omega,i} \times \mathbf{v}_i) = \mathbf{a}_i - \mathbf{g} + C_1 v_i \mathbf{v}_i. \quad (3.23)$$

Using the same vector manipulation developed in the continuous model, the discrete estimate of the spin-axis unit vector can be written as

$$\mathbf{e}_{\omega,i} = \frac{1}{C_2 v_i^3} \mathbf{v}_i \times (\mathbf{a}_i - \mathbf{g} + C_1 v_i \mathbf{v}_i). \quad (3.24)$$

This equation allows the spin direction to be estimated at each sampled time step from the measured trajectory data [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#).

To update the ball position numerically, the discrete kinematic equations are then applied. Once the acceleration at time step i is known, the velocity at the next time step is updated as

$$\mathbf{v}_{i+1} = \mathbf{v}_i + \mathbf{a}_i \Delta t, \quad (3.25)$$

and the position is updated as

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{v}_i \Delta t. \quad (3.26)$$

If greater numerical accuracy is required, the position update could include the acceleration term, [Chapra & Canale \(2015\)](#):

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{v}_i \Delta t + \frac{1}{2} \mathbf{a}_i (\Delta t)^2. \quad (3.27)$$

Hence, the continuous aerodynamic model is converted into a step-by-step numerical scheme in which the measured or computed position data are used to estimate velocity, speed, acceleration, and spin direction throughout the ball trajectory.

3.5.3 Algorithm

The discrete equations above can be translated directly into a computational algorithm. The algorithm takes as input a sequence of measured ball positions and returns the velocity, acceleration, and estimated spin direction, [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#).

Algorithm Overview:

1. **Input trajectory data**

Load the sequence of measured ball positions x_i

2. **Compute velocities**

Use finite differences to estimate velocity between successive frames.

3. **Compute accelerations**

Estimate acceleration using differences of velocity.

4. Compute aerodynamic corrections

Apply the drag and Magnus force constants C_L and C_D .

5. Estimate spin axis

Use the discrete Magnus equation to compute e_ω , [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#)

6. Output results

Return the estimated velocity, acceleration, and spin direction for each time step.

3.6 Trajectory and Spin Prediction

The purpose of this section is to develop a modelling framework that uses measured ball-position data to estimate the spin direction of the ball and then uses this estimated spin to predict the subsequent trajectory, velocity, and spin state as functions of time, [Lin et al. \(2020\)](#), [Song & Lu \(2023\)](#), [Shi et al. \(2025\)](#). The approach is based on the aerodynamic flight model derived previously, in which the ball is subjected to gravity, drag, and the Magnus force, [Schneider et al. \(2022\)](#), [Kusubori et al. \(2024\)](#). The key idea is to use an initial portion of the measured trajectory to infer the spin axis from the observed motion, and then to treat this estimated spin as an input to a forward simulation of the remaining flight, [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#).

3.6.1 Mathematical modelling

The translational motion of the ball is governed by the sum of forces acting on it, [Schneider et al. \(2022\)](#), [Lin et al. \(2020\)](#). Using the previously defined constants C_1 and C_2 , the acceleration of the ball may be written as

$$\mathbf{a} = \mathbf{g} - C_1 v \mathbf{v} + C_2 v (\mathbf{e}_\omega \times \mathbf{v}) \quad (3.28)$$

Where $\mathbf{g}$ is the gravitational acceleration, $v = \|\mathbf{v}\|$ is the ball speed, and $\mathbf{e}_\omega$ is the unit vector describing the spin direction. This equation forms the basis of both the spin-estimation stage and the forward trajectory-prediction stage.

To estimate the spin direction from measured trajectory data, the equation of motion is rearranged so that the Magnus contribution is isolated, [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#). This gives

$$C_2 v (\mathbf{e}_\omega \times \mathbf{v}) = \mathbf{a} - \mathbf{g} + C_1 v \mathbf{v} \quad (3.29)$$

Dividing through by $C_2 v$ gives an expression for the vector product involving the unknown spin direction:

$$\mathbf{e}_\omega \times \mathbf{v} = \frac{1}{C_2 v} (\mathbf{a} - \mathbf{g} + C_1 v \mathbf{v}) \quad (3.30)$$

To eliminate the cross product and solve for the spin direction, the vector $\mathbf{v}$ is crossed with both sides of the equation:

$$\mathbf{v} \times (\mathbf{e}_\omega \times \mathbf{v}) = \frac{1}{C_2 v} \mathbf{v} \times (\mathbf{a} - \mathbf{g} + C_1 v \mathbf{v}) \quad (3.31)$$

Applying the vector triple-product identity gives:

$$\mathbf{e}_\omega (\mathbf{v} \cdot \mathbf{v}) - \mathbf{v} (\mathbf{v} \cdot \mathbf{e}_\omega) = \frac{1}{C_2 v} \mathbf{v} \times (\mathbf{a} - \mathbf{g} + C_1 v \mathbf{v}) \quad (3.32)$$

For a spinning ball in flight, the spin axis is commonly assumed to be approximately perpendicular to the velocity vector, so that the projection of the spin direction onto the velocity is negligible, [Kusubori et al. \(2024\)](#), [Schneider et al. \(2022\)](#). Hence,

$$\mathbf{v} \cdot \mathbf{e}_\omega \approx 0, \quad \mathbf{v} \cdot \mathbf{v} = v^2 \quad (3.33)$$

Substituting this approximation into the previous equation gives the following expression for the spin-direction vector:

$$\mathbf{e}_\omega = \frac{1}{C_2 v^3} \mathbf{v} \times (\mathbf{a} - \mathbf{g} + C_1 v \mathbf{v}) \quad (3.34)$$

Therefore, spin direction can be estimated directly from measured velocity and acceleration data, [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#).

While running, ball trajectory is recorded at discrete times, depending on the frame rate of camera used, and velocity and acceleration must also be estimated numerically from measured positional data $t_0, t_1, \dots, t_N$, with corresponding measured positions $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_N$, then the velocity at time step k may be approximated using a central-difference expression, [Chapra & Canale \(2015\)](#):

$$\mathbf{v}_k = \frac{\mathbf{x}_{k+1} - \mathbf{x}_{k-1}}{2\Delta t} \quad (3.35)$$

where $\Delta t = t_{k+1} - t_k$ is the sampling interval. The acceleration is similarly approximated by

$$\mathbf{a}_k = \frac{\mathbf{x}_{k+1} - 2\mathbf{x}_k + \mathbf{x}_{k-1}}{\Delta t^2} \quad (3.36)$$

The corresponding speed is then

$$v_k = \|\mathbf{v}_k\| \quad (3.37)$$

Substituting these discrete velocity and acceleration estimates into the spin direction expression gives a time dependent estimate of the spin axis at each measured sample, [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#):

$$\mathbf{e}_{\omega,k} = \frac{1}{C_2 v_k^3} \mathbf{v}_k \times (\mathbf{a}_k - \mathbf{g} + C_1 v_k \mathbf{v}_k) \quad (3.38)$$

Since experimental trajectory data may contain measurement noise, [Aakanksha et al. \(2025\)](#), the spin estimates obtained at individual time steps may vary slightly. A more robust estimate of the overall spin direction is therefore obtained by averaging the spin-axis estimates over the available data window, [Aakanksha et al. \(2025\)](#):

$$\bar{\mathbf{e}}_\omega = \frac{1}{M} \sum_{k=k_1}^{k_2} \mathbf{e}_{\omega,k} \quad (3.39)$$

where $M = k_2 - k_1 + 1$ is the number of samples used in the estimation interval. The final estimated spin direction is then normalised to give a unit vector:

$$\mathbf{e}_\omega = \frac{\bar{\mathbf{e}}_\omega}{\|\bar{\mathbf{e}}_\omega\|} \quad (3.40)$$

Once the spin direction has been estimated, it may be used to predict the future motion of the ball by substituting it back into the governing flight equation. The acceleration during the prediction stage is therefore written as

$$\mathbf{a} = \mathbf{g} - C_1 v \mathbf{v} + C_2 v (\mathbf{e}_\omega \times \mathbf{v}) \quad (3.41)$$

This produces a closed system for the translational motion of the ball, since $\mathbf{e}_\omega$ is now treated as known.

The angular velocity of the ball may also be written in terms of its magnitude and direction as

$$\boldsymbol{\omega} = \omega \mathbf{e}_\omega \quad (3.42)$$

where $\omega = \|\boldsymbol{\omega}\|$ is the spin rate. In the present model, the spin direction is assumed to remain constant over the prediction interval, and the spin magnitude is also taken to be constant, [Lin et al. \(2020\)](#), [Yang et al. \(2021\)](#). Hence,

$$\mathbf{e}_\omega = \text{constant}, \quad \omega = \text{constant} \quad (3.43)$$

This assumption neglects aerodynamic spin decay and any rotational reorientation during flight. A more complete model could include a spin-decay law, [Yang et al. \(2021\)](#), of the form

$$\frac{d\omega}{dt} = -C_\omega v \omega \quad (3.44)$$

Where C_ω is a rotational damping coefficient. However, this effect is neglected here because the flight time of a table-tennis ball between consecutive ball-racket impacts is extremely short, in the range of milliseconds, [Schneider et al. \(2022\)](#). While the timescale of spin decay is comparatively longer, in the range of seconds. As a result, the change in spin magnitude over a single flight segment is relatively small when compared to the dominant translational effects of drag, gravity, and Magnus lift. Therefore, treating the spin as constant provides a reasonable and computationally efficient approximation.

With the spin axis now known, the ball trajectory may be propagated forward in time using the translational equation of motion. The resulting model allows the ball position, velocity, and spin state to be predicted throughout the remaining flight from an initial measured segment of trajectory data.

3.6.2 Numerical modelling

The equations derived in the previous subsection describe the motion of the ball in continuous time. In practice, however, the trajectory is obtained from camera measurements that provide the ball position only at discrete time instants, [Aakanksha et al. \(2025\)](#), [Zhang et al. \(2015\)](#). Therefore, numerical approximations must be used to estimate the velocity and acceleration required for spin estimation and trajectory prediction.

Assume that the ball position is measured at discrete times

$$t_0, t_1, t_2, \dots, t_N \quad (3.45)$$

with a constant sampling interval

$$\Delta t = t_{k+1} - t_k. \quad (3.46)$$

The measured ball positions are therefore

$$\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N. \quad (3.47)$$

Derivative estimation from measured data In principle, velocity and acceleration could be obtained directly from finite-difference approximations such as

$$\mathbf{v}_k = \frac{\mathbf{x}_{k+1} - \mathbf{x}_{k-1}}{2\Delta t}, \quad \mathbf{a}_k = \frac{\mathbf{x}_{k+1} - 2\mathbf{x}_k + \mathbf{x}_{k-1}}{\Delta t^2}. \quad (3.48)$$

However, acceleration is the second derivative of position and is therefore highly sensitive to measurement noise in the reconstructed trajectory, [Aakanksha et al. \(2025\)](#). Small errors in the measured coordinates can produce large fluctuations in the estimated acceleration, which in turn destabilises the spin estimation.

To improve robustness, the measured trajectory is first smoothed locally before computing derivatives. This is achieved using a Savitzky–Golay filter, which fits a low-order polynomial to the data over a moving window of samples, [Chapra & Canale \(2015\)](#). The derivatives of the fitted polynomial are then evaluated analytically to produce smooth estimates of velocity and acceleration.

Let the measured position components be $x(t)$, $y(t)$, and $z(t)$. Over a local time window, each coordinate is approximated by a polynomial:

$$x(t) \approx a_0 + a_1 t + a_2 t^2 + \dots + a_p t^p \quad (3.49)$$

with similar expressions for $y(t)$ and $z(t)$, where p is the polynomial order. Differentiating the fitted polynomial yields the velocity and acceleration estimates

$$\mathbf{v}_k = \begin{bmatrix} \dot{x}_k \\ \dot{y}_k \\ \dot{z}_k \end{bmatrix}, \quad \mathbf{a}_k = \begin{bmatrix} \ddot{x}_k \\ \ddot{y}_k \\ \ddot{z}_k \end{bmatrix}. \quad (3.50)$$

The corresponding speed magnitude is

$$v_k = \|\mathbf{v}_k\|. \quad (3.51)$$

Discrete spin estimation Substituting these smoothed velocity and acceleration estimates into the spin axis expression derived previously gives, [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#):

$$\mathbf{e}_{\omega,k} = \frac{1}{C_2 v_k^3} \mathbf{v}_k \times (\mathbf{a}_k - \mathbf{g} + C_1 v_k \mathbf{v}_k). \quad (3.52)$$

Because measurement noise can still introduce fluctuations in the instantaneous spin estimate, the final spin direction is obtained by averaging the estimates over a selected interval of samples. A weighted average is used to reduce the influence of low-speed samples where the estimator becomes numerically unstable.

The weighted spin estimate is therefore

$$\bar{\mathbf{e}}_{\omega} = \frac{\sum_{k=k_1}^{k_2} w_k \mathbf{e}_{\omega,k}}{\sum_{k=k_1}^{k_2} w_k} \quad (3.53)$$

where w_k is a weighting factor that depends on the ball speed. A suitable choice is

$$w_k = v_k^2. \quad (3.54)$$

The final spin direction is then normalised to produce a unit vector

$$\mathbf{e}_{\omega} = \frac{\bar{\mathbf{e}}_{\omega}}{\|\bar{\mathbf{e}}_{\omega}\|}. \quad (3.55)$$

Trajectory prediction Once the spin direction has been estimated, the ball motion can be propagated forward using the aerodynamic acceleration model, [Lin et al. \(2020\)](#), [Shi et al. \(2025\)](#):

$$\mathbf{a} = \mathbf{g} - C_1 v \mathbf{v} + C_2 v (\mathbf{e}_{\omega} \times \mathbf{v}). \quad (3.56)$$

The velocity and position are then updated using a time stepping scheme

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{a}_k \Delta t \quad (3.57)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{a}_k (\Delta t)^2. \quad (3.58)$$

These update equations allow the future trajectory of the ball to be predicted from the estimated spin axis and the current ball state.

3.6.3 Algorithm

The numerical model developed in the previous subsection can now be expressed as a computational algorithm. The purpose of the algorithm is to use an initial segment of measured ball-position data to estimate the spin direction of the ball and then to use this estimated spin to predict the remainder of the trajectory, [Lin et al. \(2020\)](#), [Song & Lu \(2023\)](#). The algorithm therefore consists of two stages: a spin-estimation stage based on measured trajectory data, and a forward-prediction stage based on the aerodynamic flight model.

At the start of the procedure, the algorithm takes as input the measured ball positions

$$\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N \quad (3.59)$$

sampled at times

$$t_0, t_1, t_2, \dots, t_N \quad (3.60)$$

with constant time step

$$\Delta t = t_{k+1} - t_k. \quad (3.61)$$

The aerodynamic parameters C_1 and C_2 , together with the gravitational acceleration vector $\mathbf{g}$, are also assumed to be known.

The first stage of the algorithm is to estimate smooth derivatives from the measured trajectory. Rather than differentiating the raw position data directly, a local polynomial smoothing method is first applied to each coordinate of the measured trajectory, [Zhang et al. \(2015\)](#), [Aakanksha et al. \(2025\)](#).

This produces smoothed velocity estimates of the form

$$\mathbf{v}_k = \begin{bmatrix} \dot{x}_k \\ \dot{y}_k \\ \dot{z}_k \end{bmatrix} \quad (3.62)$$

and smoothed acceleration estimates of the form

$$\mathbf{a}_k = \begin{bmatrix} \ddot{x}_k \\ \ddot{y}_k \\ \ddot{z}_k \end{bmatrix}. \quad (3.63)$$

The ball speed at each sample is then computed from the velocity magnitude:

$$v_k = \|\mathbf{v}_k\|. \quad (3.64)$$

Once the velocity and acceleration have been estimated, the spin-axis estimate is evaluated at each time step using the discrete Magnus-inversion formula

$$\mathbf{e}_{\omega,k} = \frac{1}{C_2 v_k^3} \mathbf{v}_k \times (\mathbf{a}_k - \mathbf{g} + C_1 v_k \mathbf{v}_k). \quad (3.65)$$

To reduce the influence of noisy or low-speed samples, the instantaneous spin estimates are combined using a weighted average. The weighted average spin vector is therefore computed as

$$\bar{\mathbf{e}}_{\omega} = \frac{\sum_{k=k_1}^{k_2} w_k \mathbf{e}_{\omega,k}}{\sum_{k=k_1}^{k_2} w_k} \quad (3.66)$$

with weighting factor

$$w_k = v_k^2. \quad (3.67)$$

The estimated spin direction is then normalised to produce a unit vector:

$$\mathbf{e}_{\omega} = \frac{\bar{\mathbf{e}}_{\omega}}{\|\bar{\mathbf{e}}_{\omega}\|}. \quad (3.68)$$

This completes the spin-estimation stage of the algorithm.

The second stage is the trajectory-prediction stage. Once the spin direction has been estimated, it is treated as constant over the prediction interval, [Lin et al. \(2020\)](#), [Shi et al. \(2025\)](#). The ball acceleration is then evaluated from the aerodynamic flight model

$$\mathbf{a}_k = \mathbf{g} - C_1 v_k \mathbf{v}_k + C_2 v_k (\mathbf{e}_{\omega} \times \mathbf{v}_k). \quad (3.69)$$

The velocity is advanced to the next time step using

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{a}_k \Delta t \quad (3.70)$$

and the ball position is updated using

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{a}_k (\Delta t)^2. \quad (3.71)$$

These steps are repeated sequentially to generate the predicted future trajectory of the ball.

The full procedure may therefore be summarised as follows:

1. Read the measured position data $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_N$, the sampling interval Δt , and the constants C_1 , C_2 , and $\mathbf{g}$.
2. Apply local polynomial smoothing to the measured trajectory data.

3. Compute the smoothed velocity vector $\mathbf{v}_k$ and acceleration vector $\mathbf{a}_k$ at each measured sample.
4. Compute the speed $v_k = \|\mathbf{v}_k\|$ at each sample.
5. Evaluate the instantaneous spin estimate $\mathbf{e}_{\omega,k}$ at each sample.
6. Compute the weighted average spin vector $\bar{\mathbf{e}}_{\omega}$.
7. Normalise $\bar{\mathbf{e}}_{\omega}$ to obtain the final spin-direction unit vector $\mathbf{e}_{\omega}$.
8. Select the final measured ball state $(\mathbf{x}_N, \mathbf{v}_N)$ as the initial condition for prediction.
9. Compute the aerodynamic acceleration using the estimated spin direction.
10. Update the velocity to $\mathbf{v}_{N+1}$.
11. Update the position to $\mathbf{x}_{N+1}$.
12. Repeat the acceleration, velocity, and position updates for each subsequent time step until the required prediction interval is reached.
13. Output the estimated spin direction together with the predicted position and velocity histories of the ball.

This algorithm converts the numerical model into an ordered computational procedure that can be implemented directly in software. The first part of the procedure uses measured trajectory data to estimate the spin direction in a way that is robust to noise, while the second part uses the estimated spin direction in a forward time-stepping simulation to predict the future ball motion.

3.7 Simulation of Ball–Racket Impact

The interaction between the ball and racket during a table-tennis stroke occurs over a very short time interval and can therefore be modelled as an impulsive contact event. During this interval, the dominant physical effects are the normal compression of the ball against the racket face and the tangential frictional interaction at the contact surface. The rebound behaviour is therefore governed by the contact impulse, which depends on both the coefficient of restitution in the normal direction and the frictional behaviour in the tangential plane.

The modelling approach follows rigid-body impact theory with Coulomb friction. The ball is treated as a rigid body with translational and rotational motion, while the racket motion is assumed to be prescribed during the short contact duration. The resulting post-impact ball velocity and spin are determined from the impulse transmitted through the contact point.

3.7.1 Mathematical modelling

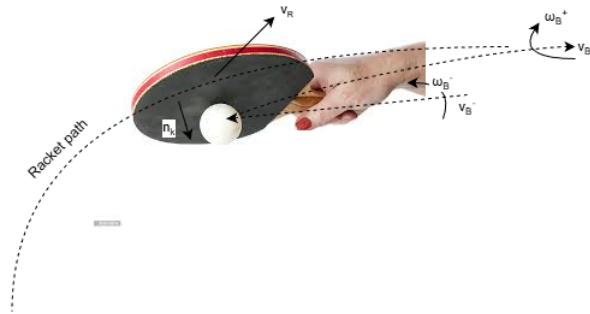


Figure 3.3: Kinematics Diagram of ball-racket impact

The first step is to define the orientation of the racket face. The unit normal vector of the racket surface is determined from the racket orientation parameter θ .

$$\mathbf{n}(\theta) = \mathbf{R}(\theta) \mathbf{e}_z \quad (3.72)$$

Next, the vector from the centre of the spherical ball to the contact point is determined. For a sphere, the contact point lies along the surface normal at a distance equal to the ball radius.

$$\mathbf{r}_{P/B} = -r \mathbf{n}(\theta) \quad (3.73)$$

The velocity of the ball material point located at the contact point is then computed using the rigid-body velocity relation for a rotating body.

$$\mathbf{u}_P^{B-} = \mathbf{v}_B^- + \boldsymbol{\omega}_B^- \times \mathbf{r}_{P/B} \quad (3.74)$$

Similarly, the velocity of the corresponding point on the racket surface is computed from the translational and rotational motion of the racket.

$$\mathbf{u}_P^{R-} = \mathbf{v}_R + \boldsymbol{\omega}_R \times \mathbf{r}_{P/R} \quad (3.75)$$

The relative velocity of the ball with respect to the racket at the contact point is obtained by subtracting the racket velocity from the ball velocity.

$$\mathbf{u}_{b|r}^- = \mathbf{u}_P^{B-} - \mathbf{u}_P^{R-} \quad (3.76)$$

To analyse the impact mechanics, the relative velocity is decomposed into normal and tangential components with respect to the racket surface.

$$u_n^- = \mathbf{u}_{b|r}^- \cdot \mathbf{n}(\theta), \quad \mathbf{u}_t^- = \mathbf{u}_{b|r}^- - u_n^- \mathbf{n}(\theta) \quad (3.77)$$

During impact, the force interaction is represented using a contact impulse. This impulse is decomposed into normal and tangential components acting at the contact point.

$$\mathbf{J} = J_n \mathbf{n}(\theta) + \mathbf{J}_t \quad (3.78)$$

The normal component of the rebound is governed by the coefficient of restitution, which relates the pre-impact and post-impact normal relative velocities.

$$u_n^+ = -C_{\text{res}} u_n^- \quad (3.79)$$

Using the impulse–momentum relation, the corresponding normal impulse acting on the ball can be written as

$$J_n = -m(1 + C_{\text{res}}) u_n^- \quad (3.80)$$

The tangential response depends on the frictional interaction between the ball and racket. The effective tangential inertia governing this interaction is described by the tangential mass operator.

$$\mathbf{K}_t = \left(\frac{1}{m} + \frac{r^2}{I_B} \right) \mathbf{I}_t, \quad \mathbf{I}_t = \mathbf{I} - \mathbf{n}(\theta) \mathbf{n}(\theta)^\top \quad (3.81)$$

If sticking occurs at the contact surface, the tangential impulse required to eliminate the relative tangential velocity is

$$\mathbf{J}_t^{\text{stick}} = -\mathbf{K}_t^{-1} \mathbf{u}_t^- \quad (3.82)$$

However, sticking can only occur if the required impulse lies within the Coulomb friction limit.

$$\|\mathbf{J}_t^{\text{stick}}\| \leq \mu |J_n| \quad (3.83)$$

If this condition is satisfied, the contact remains in the sticking regime. Otherwise, sliding occurs and the tangential impulse is limited by the Coulomb friction law.

$$\mathbf{J}_t = \begin{cases} -\mathbf{K}_t^{-1} \mathbf{u}_t^-, & \text{if } \|\mathbf{J}_t^{\text{stick}}\| \leq \mu |J_n|, \\ \begin{bmatrix} J_n \mathbf{n}(\theta) + \mathbf{J}_t \end{bmatrix} - \mu |J_n| \frac{\mathbf{u}_t^-}{\|\mathbf{u}_t^-\|}, & \text{if } \|\mathbf{J}_t^{\text{stick}}\| > \mu |J_n|. \end{cases} \quad (3.84)$$

Once the total impulse has been determined, the post-impact translational velocity of the ball can be computed using the impulse–momentum equation.

$$\mathbf{v}_B^+ = \mathbf{v}_B^- + \frac{1}{m} \left(J_n \mathbf{n}(\theta) + \mathbf{J}_t \right) \quad (3.85)$$

The tangential impulse also generates a torque about the ball centre, producing a change in the angular velocity of the ball.

$$\boldsymbol{\omega}_B^+ = \boldsymbol{\omega}_B^- + I_B^{-1} \left(\mathbf{r}_{P/B} \times \mathbf{J}_t \right) \quad (3.86)$$

Substituting the spherical contact geometry simplifies this expression.

$$\mathbf{r}_{P/B} = -r \mathbf{n}(\theta) \quad \Rightarrow \quad \boldsymbol{\omega}_B^+ = \boldsymbol{\omega}_B^- - I_B^{-1} \left(r \mathbf{n}(\theta) \times \mathbf{J}_t \right) \quad (3.87)$$

For a thin-shell table-tennis ball, the moment of inertia about the centre is approximated as

$$I_B = \frac{2}{3} m r^2 \quad (3.88)$$

Substituting this inertia expression yields a simplified closed-form expression for the post-impact angular velocity.

$$\boldsymbol{\omega}_B^+ = \boldsymbol{\omega}_B^- - \frac{3}{2mr} \mathbf{n}(\theta) \times \mathbf{J}_t \quad (3.89)$$

The velocity of the contact point on the ball after impact can then be reconstructed from the updated translational and rotational states.

$$\mathbf{u}_P^{B+} = \mathbf{v}_B^+ + \boldsymbol{\omega}_B^+ \times \mathbf{r}_{P/B} \quad (3.90)$$

Finally, the post-impact relative velocity between the ball and racket is obtained.

$$\mathbf{u}_{b|r}^+ = \mathbf{u}_P^{B+} - \mathbf{u}_P^{R-} \quad (3.91)$$

This velocity should satisfy the restitution condition in the normal direction and either the sticking or sliding friction law in the tangential plane.

3.7.2 Numerical modelling

The impact model developed in the previous subsection is naturally suited to a discrete formulation because the ball-racket impact occurs over a very short time interval. Rather than integrating the equations of motion continuously through the contact phase, the impact is treated as an instantaneous event that maps the pre-impact state of the ball to its post-impact state. The numerical task is therefore to compute the post-impact translational and angular velocities from the known pre-impact quantities.

Let the ball state immediately before impact be evaluated at the discrete impact instant k . The pre-impact translational and angular velocities are denoted by

$$\mathbf{v}_{B,k}^-, \quad \boldsymbol{\omega}_{B,k}^- \quad (3.92)$$

Similarly, the racket translational and angular velocities at impact are taken as known inputs:

$$\mathbf{v}_{R,k}, \quad \boldsymbol{\omega}_{R,k}. \quad (3.93)$$

The racket orientation at the impact instant is also treated as a known discrete input. From this, the racket-face unit normal is computed as

$$\mathbf{n}_k = \mathbf{R}(\theta_k) \mathbf{e}_z. \quad (3.94)$$

For a spherical ball, the vector from the ball centre to the contact point is then written in discrete form as

$$\mathbf{r}_{P/B,k} = -r \mathbf{n}_k. \quad (3.95)$$

The velocity of the contact point on the ball immediately before impact obtained by rigid-body kinematics:

$$\mathbf{u}_{P,k}^{B-} = \mathbf{v}_{B,k}^- + \boldsymbol{\omega}_{B,k}^- \times \mathbf{r}_{P/B,k}. \quad (3.96)$$

Similarly, the velocity of the corresponding point on the racket is:

$$\mathbf{u}_{P,k}^{R-} = \mathbf{v}_{R,k} + \boldsymbol{\omega}_{R,k} \times \mathbf{r}_{P/R,k}. \quad (3.97)$$

The pre-impact relative velocity of the ball with respect to the racket at the contact point is:

$$\mathbf{u}_{b|r,k}^- = \mathbf{u}_{P,k}^{B-} - \mathbf{u}_{P,k}^{R-}. \quad (3.98)$$

To determine the contact response, this relative velocity is decomposed into normal and tangential components. The normal component is computed by projection onto the racket-face normal:

$$u_{n,k}^- = \mathbf{u}_{b|r,k}^- \cdot \mathbf{n}_k. \quad (3.99)$$

The tangential component is obtained by subtracting the normal projection from the full relative velocity:

$$\mathbf{u}_{t,k}^- = \mathbf{u}_{b|r,k}^- - u_{n,k}^- \mathbf{n}_k. \quad (3.100)$$

The normal impulse is evaluated from the restitution law. In discrete form, the post-impact normal relative velocity:

$$u_{n,k}^+ = -C_{\text{res}} u_{n,k}^- \quad (3.101)$$

This gives the normal impulse:

$$J_{n,k} = -m(1 + C_{\text{res}}) u_{n,k}^- \quad (3.102)$$

The tangential contact dynamics are represented through the tangential mass operator. At the discrete impact instant:

$$\mathbf{l}_{t,k} = \mathbf{I} - \mathbf{n}_k \mathbf{n}_k^\top, \quad (3.103)$$

$$\mathbf{K}_{t,k} = \left(\frac{1}{m} + \frac{r^2}{I_B} \right) \mathbf{l}_{t,k}. \quad (3.104)$$

Assuming sticking first, the tangential impulse required to eliminate the tangential relative velocity is:

$$\mathbf{J}_{t,k}^{\text{stick}} = -\mathbf{K}_{t,k}^{-1} \mathbf{u}_{t,k}^- \quad (3.105)$$

This candidate sticking impulse is admissible only if it satisfies the Coulomb friction bound:

$$\|\mathbf{J}_{t,k}^{\text{stick}}\| \leq \mu |J_{n,k}|. \quad (3.106)$$

The discrete tangential impulse is therefore chosen according to a stick-slip decision rule. If the sticking condition is satisfied, the sticking impulse is used; otherwise, a sliding impulse of Coulomb-limited magnitude is applied:

$$\mathbf{J}_{t,k} = \begin{cases} -\mathbf{K}_{t,k}^{-1} \mathbf{u}_{t,k}^-, & \text{if } \|\mathbf{J}_{t,k}^{\text{stick}}\| \leq \mu |J_{n,k}|, \\ \mu |J_{n,k}| \frac{\mathbf{u}_{t,k}^-}{\|\mathbf{u}_{t,k}^-\|}, & \text{if } \|\mathbf{J}_{t,k}^{\text{stick}}\| > \mu |J_{n,k}|. \end{cases} \quad (3.107)$$

Once the normal and tangential impulses have been determined, the total impulse acting on the ball is

$$\mathbf{J}_k = J_{n,k} \mathbf{n}_k + \mathbf{J}_{t,k}. \quad (3.108)$$

The post-impact translational velocity of the ball is then updated discretely using the impulse-momentum relation:

$$\mathbf{v}_{B,k}^+ = \mathbf{v}_{B,k}^- + \frac{1}{m} \mathbf{J}_k. \quad (3.109)$$

The post-impact angular velocity is updated from the torque generated by the tangential impulse:

$$\boldsymbol{\omega}_{B,k}^+ = \boldsymbol{\omega}_{B,k}^- + I_B^{-1} (\mathbf{r}_{P/B,k} \times \mathbf{J}_{t,k}). \quad (3.110)$$

Using the spherical contact geometry $\mathbf{r}_{P/B,k} = -r \mathbf{n}_k$, this becomes

$$\boldsymbol{\omega}_{B,k}^+ = \boldsymbol{\omega}_{B,k}^- - I_B^{-1} (r \mathbf{n}_k \times \mathbf{J}_{t,k}). \quad (3.111)$$

For a thin-shell table-tennis ball, the moment of inertia is approximated as

$$I_B = \frac{2}{3}mr^2. \quad (3.112)$$

Substituting this expression gives the closed-form discrete spin update

$$\boldsymbol{\omega}_{B,k}^+ = \boldsymbol{\omega}_{B,k}^- - \frac{3}{2mr} \mathbf{n}_k \times \mathbf{J}_{t,k}. \quad (3.113)$$

Finally, the post-impact contact-point velocity on the ball may be reconstructed as

$$\mathbf{u}_{P,k}^{B+} = \mathbf{v}_{B,k}^+ + \boldsymbol{\omega}_{B,k}^+ \times \mathbf{r}_{P/B,k}, \quad (3.114)$$

and the post-impact relative contact velocity becomes

$$\mathbf{u}_{b|r,k}^+ = \mathbf{u}_{P,k}^{B+} - \mathbf{u}_{P,k}^{R-}. \quad (3.115)$$

Hence, the continuous impact model is converted into a discrete update procedure in which the known pre-impact state at impact step k is mapped directly to the post-impact state through the evaluation of the normal impulse, tangential impulse, and stick-slip contact condition.

3.7.3 Algorithm

The discrete impact model derived in the previous subsection can now be expressed as a computational algorithm. The purpose of the algorithm is to map the known pre-impact state of the ball and racket to the post-impact translational and angular velocities of the ball. This is achieved by evaluating the contact kinematics, computing the normal and tangential impulses, determining whether the contact sticks or slips, and then updating the ball state accordingly.

At the beginning of the impact calculation, the algorithm takes as input the pre-impact ball velocity $\mathbf{v}_{B,k}^-$, the pre-impact ball angular velocity $\boldsymbol{\omega}_{B,k}^-$, the racket translational velocity $\mathbf{v}_{R,k}$, the racket angular velocity $\boldsymbol{\omega}_{R,k}$, the racket orientation θ_k , and the model parameters m , r , I_B , C_{res} , and μ .

The first step is to compute the racket-face normal from the racket orientation.

$$\mathbf{n}_k = \mathbf{R}(\theta_k) \mathbf{e}_z \quad (3.116)$$

Next, the vector from the ball centre to the contact point is evaluated using the ball radius and the racket-face normal.

$$\mathbf{r}_{P/B,k} = -r \mathbf{n}_k \quad (3.117)$$

The velocity of the contact point on the ball is then determined from rigid-body kinematics.

$$\mathbf{u}_{P,k}^{B-} = \mathbf{v}_{B,k}^- + \boldsymbol{\omega}_{B,k}^- \times \mathbf{r}_{P/B,k} \quad (3.118)$$

The corresponding contact-point velocity on the racket is also computed from the prescribed racket motion.

$$\mathbf{u}_{P,k}^{R-} = \mathbf{v}_{R,k} + \boldsymbol{\omega}_{R,k} \times \mathbf{r}_{P/R,k} \quad (3.119)$$

The pre-impact relative contact velocity is obtained by subtracting the racket contact-point velocity from the ball contact-point velocity.

$$\mathbf{u}_{b|r,k}^- = \mathbf{u}_{P,k}^{B-} - \mathbf{u}_{P,k}^{R-} \quad (3.120)$$

This relative velocity is decomposed into normal and tangential components in order to determine contact regime:

$$u_{n,k}^- = \mathbf{u}_{b|r,k}^- \cdot \mathbf{n}_k \quad (3.121)$$

$$\mathbf{u}_{t,k}^- = \mathbf{u}_{b|r,k}^- - u_{n,k}^- \mathbf{n}_k \quad (3.122)$$

The normal component of impulse is then calculated from the restitution law:

$$J_{n,k} = -m(1 + C_{\text{res}}) u_{n,k}^- \quad (3.123)$$

The tangential mass operator is next evaluated in the tangent plane:

$$\mathbf{l}_{t,k} = \mathbf{l} - \mathbf{n}_k \mathbf{n}_k^\top \quad (3.124)$$

$$\mathbf{K}_{t,k} = \left(\frac{1}{m} + \frac{r^2}{I_B} \right) \mathbf{l}_{t,k} \quad (3.125)$$

Using this operator, the tangential impulse required for sticking is first computed:

$$\mathbf{J}_{t,k}^{\text{stick}} = -\mathbf{K}_{t,k}^{-1} \mathbf{u}_{t,k}^- \quad (3.126)$$

The magnitude of this candidate sticking impulse is then compared with the Coulomb friction limit:

$$\|\mathbf{J}_{t,k}^{\text{stick}}\| \leq \mu |J_{n,k}| \quad (3.127)$$

If this condition is satisfied, the contact is classified as sticking and the sticking impulse is used. Otherwise, the contact is classified as sliding and the tangential impulse is limited by Coulomb friction:

$$\mathbf{J}_{t,k} = \begin{cases} -\mathbf{K}_{t,k}^{-1} \mathbf{u}_{t,k}^-, & \text{if } \|\mathbf{J}_{t,k}^{\text{stick}}\| \leq \mu |J_{n,k}|, \\ \text{begin equation 1ex} - \mu |J_{n,k}| \frac{\mathbf{u}_{t,k}^-}{\|\mathbf{u}_{t,k}^-\|}, & \text{otherwise.} \end{cases} \quad (3.128)$$

Once the tangential impulse has been determined, the total contact impulse is assembled:

$$\mathbf{J}_k = J_{n,k} \mathbf{n}_k + \mathbf{J}_{t,k} \quad (3.129)$$

The ball translational velocity is then updated using the total impulse:

$$\mathbf{v}_{B,k}^+ = \mathbf{v}_{B,k}^- + \frac{1}{m} \mathbf{J}_k \quad (3.130)$$

The ball angular velocity is updated using the torque generated by the tangential impulse:

$$\boldsymbol{\omega}_{B,k}^+ = \boldsymbol{\omega}_{B,k}^- + I_B^{-1} (\mathbf{r}_{P/B,k} \times \mathbf{J}_{t,k}) \quad (3.131)$$

Using the spherical contact geometry, this becomes:

$$\boldsymbol{\omega}_{B,k}^+ = \boldsymbol{\omega}_{B,k}^- - I_B^{-1} (r \mathbf{n}_k \times \mathbf{J}_{t,k}) \quad (3.132)$$

For a thin-shell table-tennis ball, substituting $I_B = \frac{2}{3}mr^2$ gives the final explicit spin update:

$$\boldsymbol{\omega}_{B,k}^+ = \boldsymbol{\omega}_{B,k}^- - \frac{3}{2mr} \mathbf{n}_k \times \mathbf{J}_{t,k} \quad (3.133)$$

At the end of the procedure, the algorithm returns the post-impact ball velocity $\mathbf{v}_{B,k}^+$, the post-impact ball angular velocity $\boldsymbol{\omega}_{B,k}^+$, and, if required, the post-impact relative contact velocity for verification of the contact law.

The complete impact algorithm can therefore be summarised as follows:

1. Read the pre-impact ball state, racket state, and model parameters.
2. Compute the racket-face normal $\mathbf{n}_k$.
3. Compute the ball contact vector $\mathbf{r}_{P/B,k}$.
4. Compute the pre-impact contact-point velocities of the ball and racket.
5. Compute the relative contact velocity $\mathbf{u}_{b|r,k}^-$.
6. Decompose the relative velocity into normal and tangential components.
7. Compute the normal impulse $J_{n,k}$ from the restitution law.

8. Compute the candidate sticking impulse $\mathbf{J}_{t,k}^{\text{stick}}$.
9. Check the Coulomb friction condition.
10. Select sticking or sliding to determine $\mathbf{J}_{t,k}$.
11. Assemble the total impulse $\mathbf{J}_k$.
12. Update the post-impact translational velocity $\mathbf{v}_{B,k}^+$.
13. Update the post-impact angular velocity $\omega_{B,k}^+$.
14. Output the post-impact ball state.

This algorithm converts the discrete impact equations into an ordered computational procedure that can be implemented directly in software. In particular, the stick-slip decision introduces a conditional branch in the algorithm, while the remaining operations consist primarily of vector arithmetic, projections, and cross products.

3.8 Determination of racket orientation for target-constrained return

3.8.1 Mathematical modelling

The purpose of this section is to determine the required racket orientation at the moment of impact such that the ball, after rebounding from the racket, lands within a predefined target region on the opponent's side of the table. The incoming ball state at impact is assumed to be known from the previous modelling stages, including the ball position, velocity, and spin. The outgoing ball trajectory and spin, however, are not prescribed and must be determined through the interaction between the ball and the racket.

The impact model developed in the previous section provides a mapping between the pre-impact ball state, the racket state, and the resulting post-impact ball velocity and spin. In general, both the racket orientation and velocity influence the rebound. However, in the present formulation, the racket speed is assumed to be prescribed by the robot actuator system. Furthermore, the racket contact-point velocity is assumed to act along the racket-face normal. Under this assumption, the only remaining control variables are the racket orientation angles.

Let the racket-face normal be parametrised by two angles θ and ϕ , such that

$$\mathbf{n} = \mathbf{n}(\theta, \phi), \quad (3.134)$$

where $\mathbf{n}$ is a unit vector normal to the racket surface. The racket contact-point velocity is then defined as

$$\mathbf{u}_P^{R-} = V_R \mathbf{n}, \quad (3.135)$$

where V_R is the prescribed racket speed.

The incoming ball state at the instant of impact is given by

$$\mathbf{x}_I, \quad \mathbf{v}_B^-, \quad \boldsymbol{\omega}_B^-. \quad (3.136)$$

Using the impact relations derived previously, the post-impact ball velocity and angular velocity are functions of the racket orientation:

$$\mathbf{v}_B^+ = \mathbf{v}_B^+(\theta, \phi), \quad \boldsymbol{\omega}_B^+ = \boldsymbol{\omega}_B^+(\theta, \phi). \quad (3.137)$$

These quantities are obtained by evaluating the contact kinematics, computing the normal and tangential impulses, and applying the impulse–momentum relations.

Once the post-impact ball state has been determined, the subsequent flight of the ball is governed by the aerodynamic model

$$\mathbf{a} = \mathbf{g} - C_1 v \mathbf{v} + C_2 v (\mathbf{e}_\omega \times \mathbf{v}), \quad (3.138)$$

where the spin direction is given by

$$\mathbf{e}_\omega = \frac{\boldsymbol{\omega}_B^+}{\|\boldsymbol{\omega}_B^+\|}. \quad (3.139)$$

Starting from the impact point $\mathbf{x}_I$ and the post-impact velocity $\mathbf{v}_B^+$, the ball trajectory is propagated forward in time until it intersects the table surface. This defines the bounce point

$$\mathbf{x}_{\text{bounce}} = \begin{bmatrix} x_{\text{bounce}} \\ y_{\text{bounce}} \\ 0 \end{bmatrix}, \quad (3.140)$$

which is a function of the racket orientation:

$$\mathbf{x}_{\text{bounce}} = \mathbf{x}_{\text{bounce}}(\theta, \phi). \quad (3.141)$$

Let the desired landing region on the table be defined as a subset

$$\mathcal{A} \subset \mathbb{R}^2. \quad (3.142)$$

The racket orientation is considered feasible if the predicted bounce point satisfies

$$(x_{\text{bounce}}, y_{\text{bounce}}) \in \mathcal{A}. \quad (3.143)$$

In order to select a unique solution, a cost function is defined relative to the centre of the target region. Let the centre of the target region be

$$\mathbf{x}_T = \begin{bmatrix} x_T \\ y_T \\ 0 \end{bmatrix}. \quad (3.144)$$

The objective is then to minimise the landing error

$$J(\theta, \phi) = \left\| \begin{bmatrix} x_{\text{bounce}}(\theta, \phi) \\ y_{\text{bounce}}(\theta, \phi) \end{bmatrix} - \begin{bmatrix} x_T \\ y_T \end{bmatrix} \right\|^2. \quad (3.145)$$

The optimal racket orientation is therefore given by

$$(\theta^*, \phi^*) = \arg \min_{\theta, \phi} J(\theta, \phi) \quad (3.146)$$

subject to physical constraints on the racket orientation:

$$\theta_{\min} \leq \theta \leq \theta_{\max}, \quad \phi_{\min} \leq \phi \leq \phi_{\max}. \quad (3.147)$$

This formulation defines a target-constrained forward model in which candidate racket orientations are evaluated through the impact and flight models, and the orientation that produces a valid landing within the desired region is selected.

3.8.2 Numerical modelling

The mathematical model developed in the previous subsection defines a mapping from racket orientation to the resulting bounce location of the ball. However, this mapping is nonlinear and does not admit a closed-form solution due to the combined effects of impact dynamics, frictional contact, and aerodynamic flight. Therefore, a numerical approach is required to determine the racket orientation that produces a valid return within the desired target region.

Parameterisation of racket orientation The racket orientation is parameterised by two angles θ and ϕ , which define the unit normal vector to the racket face

$$\mathbf{n} = \mathbf{n}(\theta, \phi). \quad (3.148)$$

The admissible orientation space is bounded by physical constraints of the robot, such that

$$\theta_{\min} \leq \theta \leq \theta_{\max}, \quad \phi_{\min} \leq \phi \leq \phi_{\max}. \quad (3.149)$$

A discrete set of candidate orientations is generated by sampling this domain:

$$\theta_i = \theta_{\min} + i \Delta\theta, \quad \phi_j = \phi_{\min} + j \Delta\phi. \quad (3.150)$$

Evaluation of candidate orientations For each candidate orientation (θ_i, ϕ_j) , the corresponding racket normal vector $\mathbf{n}_{ij}$ is computed. The racket contact-point velocity is then defined as

$$\mathbf{u}_{P,ij}^{R-} = V_R \mathbf{n}_{ij}. \quad (3.151)$$

Using the impact model developed previously, the post-impact ball velocity and spin are computed:

$$\mathbf{v}_{B,ij}^+, \quad \boldsymbol{\omega}_{B,ij}^+. \quad (3.152)$$

These quantities are obtained by evaluating the relative contact velocity, computing the normal and tangential impulses, and applying the impulse–momentum relations.

Forward trajectory simulation The ball trajectory is then propagated forward in time using a numerical time-stepping scheme. Starting from the impact point $\mathbf{x}_I$ and the post-impact velocity $\mathbf{v}_{B,ij}^+$, the velocity and position are updated using

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{a}_k \Delta t, \quad (3.153)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{a}_k (\Delta t)^2, \quad (3.154)$$

where the acceleration is given by

$$\mathbf{a}_k = \mathbf{g} - C_1 v_k \mathbf{v}_k + C_2 v_k (\mathbf{e}_{\omega,ij} \times \mathbf{v}_k), \quad (3.155)$$

and

$$\mathbf{e}_{\omega,ij} = \frac{\boldsymbol{\omega}_{B,ij}^+}{\|\boldsymbol{\omega}_{B,ij}^+\|}. \quad (3.156)$$

The integration is performed iteratively until the ball intersects the table surface, defined by $z = 0$.

Bounce detection The bounce point is determined by detecting the first time step at which the ball position crosses the table plane. If $z_k > 0$ and $z_{k+1} \leq 0$, the bounce point is approximated by linear interpolation:

$$\mathbf{x}_{\text{bounce},ij} = \mathbf{x}_k + \frac{z_k}{z_k - z_{k+1}} (\mathbf{x}_{k+1} - \mathbf{x}_k). \quad (3.157)$$

Feasibility and selection Each candidate orientation is evaluated by checking whether the predicted bounce point lies within the target region:

$$(x_{\text{bounce},ij}, y_{\text{bounce},ij}) \in \mathcal{A}. \quad (3.158)$$

All feasible orientations are recorded. If multiple feasible orientations exist, the optimal orientation is selected by minimising the landing error with respect to the target centre:

$$J_{ij} = \left\| \begin{bmatrix} x_{\text{bounce},ij} \\ y_{\text{bounce},ij} \end{bmatrix} - \begin{bmatrix} x_T \\ y_T \end{bmatrix} \right\|^2. \quad (3.159)$$

The selected orientation is therefore

$$(\theta^*, \phi^*) = \arg \min_{i,j} J_{ij}. \quad (3.160)$$

Computational considerations The resolution of the orientation sampling, defined by $\Delta\theta$ and $\Delta\phi$, determines the trade-off between computational cost and accuracy. A coarse sampling provides faster evaluation but may miss feasible solutions, while finer sampling improves accuracy at the expense of increased computation time. In practice, a multi-resolution approach may be used, where a coarse search identifies candidate regions followed by a refined search in promising areas.

This numerical procedure provides a robust and implementable method for determining the racket orientation required to return the ball to a specified target region.

3.9 Algorithm to convert target ball speed, spin and direction to motors

In this section, our objective is to advance an algorithm that converts the desired ball spin, speed and direction in accordance with motor control inputs of the launcher system. We begin our modeling process by displaying the target velocity vector of the ball in a 3D space, depicting the direction and magnitude of the desired shot. We then calculate the launcher angles in order to position the launcher to the target direction. The desired ball speed will then be linked to the surface velocities of all three launcher wheels, hence generating the spin and linear velocity of the ball by frictional contact. We then convert the wheel surface velocities into angular velocities for each wheel by taking the geometric relationship between the angular velocity and wheel radius into consideration. These angular velocities will then be formulated as motor rotational speed in RPMs, playing the role of control inputs driving the ball launchers.

3.9.1 Mathematical modelling

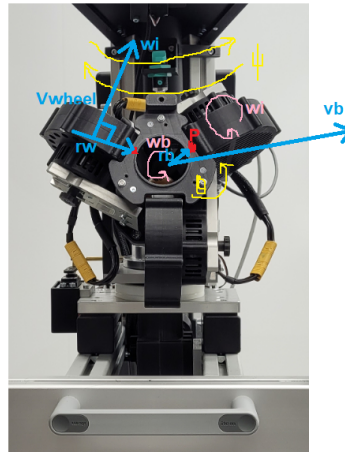


Figure 3.4: Ball launcher vector diagram, [Dittrich et al. \(2022\)](#)

In 3.4, we see a complete depiction between the comparable wheel dynamics and the desired motion. The linear velocity of the ball labelled v_b describes the magnitude and direction of the launch, while ω_b , its angular velocity portrays the spin. Angle θ and ψ distinguishes the overall launcher orientation, together determining direction labelled v_b . We describe wheel contact P by using radius vectors r_w and r_b , expanding from the centres of every wheel to the contact point. Each wheel, navigated by its motor spins with angular velocity ω_i , giving rise to a tangential surface velocity $v_w = \omega_i \times r_w$, justifying both rotational and translational motion. Through this formulation, we provide

a physical and direct compatible relationship between the resulting ball trajectory and motor-driven wheel rotations, putting in place the footing for the modelling and control algorithm.

3.9.1.1 Launcher Orientation

The launcher is supposed to point in the same direction as the ball velocity vector. The first thing we will do is define the desired ball velocity vector.

$$\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} \quad (3.161)$$

Deduce the total speed of the ball to be:

$$||\mathbf{v}|| = \sqrt{\mathbf{v}_x^2 + \mathbf{v}_y^2 + \mathbf{v}_z^2} \quad (3.162)$$

We now calculate the yaw angle, as it lines up the launcher with the horizontal direction of the ball and rotates the launcher from left-right:

$$\psi = \tan^{-1}\left(\frac{\mathbf{v}_y}{\mathbf{v}_x}\right) \quad (3.163)$$

Because we already have the yaw angle, we will now calculate the launcher pitch angle that guaranties that the ball will leave the launcher at the right elevation and also rotate the launcher up-down in contrast to the yaw angle:

$$\theta = \tan^{-1}\left(\frac{v_z}{\sqrt{v_x^2 + v_y^2}}\right) \quad (3.164)$$

The yaw angle and launcher pitch angle mentioned above will control the orientation motors of the launcher head.

3.9.1.2 Wheel launcher model

Typically, the launcher has three spinning wheels that accelerate the ball through the ball friction. Each wheel creates a surface velocity that transfers energy to the ball. This is the surface velocity of each wheel, derived from the relationship between angular and linear velocity:

$$\mathbf{v}_w = \omega_w \times \mathbf{r}_w \quad (3.165)$$

Theta for a rotating wheel is 90° , meaning the wheel radius and the rotation axis are perpendicular. Therefore:

$$v_i = \omega_i \times r_w \quad (3.166)$$

In this equation, we recognise a physical link between the velocity of the ball, wheel motion and the motor rotation. We can also derive the velocity of a particular point on the wheel precisely at the contact point.

In order to get the speed of the ball, we utilize rigid body kinematics:

$$\mathbf{v}_s = \mathbf{v}_b + \omega_b \times \mathbf{r}_b \quad (3.167)$$

We will express the angular velocity of all three motors:

$$\omega_w = \frac{\mathbf{r}_w \times (\mathbf{v}_b + \omega_b \times \mathbf{r}_b)}{|\mathbf{r}_w|^2} \quad (3.168)$$

3.9.1.3 Convert wheel speed to motor RPM

Unlike the wheels, motors are controlled in revolutions per minute(RPMs), and we use this equation to convert angular velocity to RPMs:

$$n_{w,RPM} = \frac{|\omega_w|60}{2\pi} \quad (3.169)$$

3.9.2 Algorithm

The algorithm is needed to convert the desired ball-motion parameters, such as speed, direction, and spin, into the corresponding mechanical commands required by the table tennis robot launcher. In a robotic training system, the launcher must proliferate rational shots so that players can practice a diversity of strokes and reaction skills. Therefore, the algorithm works as an inverse model of the launching mechanism, translating the wanted ball trajectory into the correct launcher orientation and wheel motor speeds. The algorithm begins by collecting the necessary ball motion parameters as inputs. These inputs typically include the wanted ball speed, the horizontal direction angle, yaw, the vertical direction angle, pitch, and the necessary spin rate of the ball. In addition, physical parameters such as the radius of the launcher wheels and the radius of the table tennis ball are also conditions. These parameters are primal because they calculate the relationship between the wheel rotation speeds and the resulting ball motion. The input values may begin from a training program, a trajectory planner, or a user interface that interprets the type of shot the robot must fabricate.

INPUT : ballspeedVb, directionθ, elevationφ, spinω

1. Get components of target ball velocity vector

The launcher aligns with the direction of the velocity vector to specify that the ball travels along the calculated trajectory. Horizontal and vertical angles are utilized

with the ball speed to determine the velocity components along the forward, sideways, and vertical directions.

2. Determine yaw and pitch angles

The algorithm determines the system's orientation of the launcher. The yaw angle is calculated by calculating the difference between the sideways velocity component and the forward velocity component. The angle shows the horizontal rotations necessary for the launcher to face the desired direction. The pitch angle is determined utilizing the vertical velocity component and the horizontal velocity magnitude.

3. Calculate wheel angular velocities

The algorithm calculates the rotational speeds of the launcher wheels that will generate the wanted ball speed and spin. By applying the mathematical relationships between wheel rotation, ball velocity, and spin, the algorithm computes the angular velocity wanted for each wheel

4. convert to rpm

The determined angular velocities are converted into motor control signals, sometimes expressed in revolutions per minute (RPM). These RPM values are compatible with the motor drivers that control the launcher wheels. At the same time, the determined yaw and pitch angles are sent to the servo motors responsible for rotating the launcher mechanism

5. update the configuration settings of ball launcher

Chapter 4: Design

4.1 Introduction

In this section focuses on the practical implementation of the system and establishes how the suggested system architecture is going to incorporate embedded systems, electrical design, computer vision, as well as real time motor control in order to produce an intelligent robotic platform that will be able to respond to our systems gameplay conditions. The general architectural design combines several software and hardware sub-systems. The NVIDIA Jetson Nano functions as our primary vision processing platform that aids for object and image detection, system coordination as well as trajectory estimation. The STM32 microcontroller subsystem serves as the control unit responsible for PWM signal generation and real time motor control. It also communicates with the electromechanical components.

4.2 Design requirements

The system is specified by a structured requirements list following ISO/IEC/IEEE 29148:2018 ([ISO/IEC/IEEE 2018](#)). Requirements are written in declarative “shall” form so that each is independently testable, and every requirement carries a unique two-letter identifier of the form XX-N: FR (functional), PR (performance), ER (electrical), MR (mechanical), SR (software), and CR (constraint). Subsequent subsystem-design sections ([4.3](#) through [4.7](#)) reference these identifiers to establish traceability.

4.2.1 Requirements catalogue

Table [4.1](#) states the functional, electrical, mechanical, software, and constraint requirements. The performance requirements, which carry numerical targets and measurement methods, are tabulated separately in Table [4.2](#) because they form the verification specification against which Chapter 5 reports measured results.

Table 4.1: Requirements catalogue (functional, electrical, mechanical, software, and constraints). Implementing sections are noted in the category headers.

ID	Requirement
<i>Functional (FR-)</i>	
FR-1	The system shall detect a regulation ITTF table-tennis ball within the play volume from a single high-speed camera stream.
FR-2	The system shall reconstruct the ball's three-dimensional position and translational velocity in a table-fixed coordinate frame.

continued on next page

Table 4.1: *continued from previous page*

ID	Requirement
FR-3	The system shall estimate the ball's spin axis from the reconstructed trajectory.
FR-4	The system shall predict the ball's landing position on the table before the bounce.
FR-5	The system shall compute launcher commands (yaw, pitch, and three drive-wheel RPMs) that return the ball to a selected target region on the player's side of the table.
FR-6	The system shall transmit the computed launcher commands to the actuator subsystem within one rally interval.
FR-7	The system shall expose an operator-initiated safe-stop that halts all motor output and awaits manual reset.
FR-8	The system shall log timestamped state vectors (detection, Kalman estimate, predicted landing, command) for off-line analysis.
<i>Electrical (ER-), implemented in 4.4</i>	
ER-1	The system shall be powered from a single regulated DC bus of nominal voltage 24 V; mains-side draw shall not exceed 5 A from a 230 V, 50 Hz outlet (~ 1.15 kW peak input).
ER-2	The system shall provide three secondary rails: 24 V for the launcher drive motors, 5 V for the steering servos and the Jetson Nano, and 3.3 V logic for buttons, LEDs, and MCU signalling.
ER-4	Supply-rail ripple at the Jetson Nano 5 V input shall be ≤ 100 mV peak-to-peak under all operating conditions.
ER-5	The launcher motor drivers shall accept a UART command stream at 921 600 baud, 8N1 framing, 3.3 V logic, no hardware flow control.
ER-6	All mains-side conductors shall be electrically isolated from the SELV (safety extra-low voltage) circuits per IEC 61140 (IEC 2016).
ER-7	The system shall provide a single mechanical emergency-stop switch that interrupts motor-driver power without depending on software state.
<i>Mechanical (MR-), implemented in 4.3</i>	
MR-4	The camera mount shall provide adjustable pitch of $\pm 30^\circ$ at a fixed height, lockable with a single hex key.
MR-6	The three launcher drive wheels shall be arranged symmetrically at 120° intervals around the ball exit axis, consistent with the geometry declared in <code>config.WHEEL_CONTACT_POINTS</code> .
<i>Software (SR-), implemented in 4.5, 4.6, 4.7</i>	
SR-1	The perception and high-level control software shall execute on NVIDIA JetPack 4.6.x / Python 3.6.9 on a Jetson Nano 4 GB without modification of the supplied OS.
SR-2	All tunable parameters shall be declared exactly once in <code>config.py</code> ; no parameter shall appear as a magic number inside a function body.
SR-3	The detection subsystem shall execute through a TensorRT FP16 engine built on the deployment target rather than the stock PyTorch inference path.

continued on next page

Table 4.1: *continued from previous page*

ID	Requirement
SR-5	Source-level dependencies shall be version-pinned in <code>requirements.txt</code> ; no network installation shall be performed at deployment time.
SR-6	All source code shall be under version control; every design constant in <code>config.py</code> shall be traceable to a calibrated measurement, a physical standard, or a documented tuning decision.
SR-7	The microcontroller firmware shall accept the launcher-command framing of 4.6.7.3 and execute the most recently received valid command until superseded.
SR-8	On detection of a corrupted command frame (CRC mismatch), the microcontroller shall drop the frame and continue with the previous command, without raising an interrupt to the Jetson.
<i>Constraints and non-functional (CR-)</i>	
CR-1	The ball, net, and table shall conform to ITTF specifications: 40 mm diameter, 2.7 g mass, $2.74 \times 1.525 \times 0.76$ m table (International Table Tennis Federation 2024).
CR-2	The system shall operate safely with one untrained player at the opposite end of the table; no projectile shall be launched with a predicted target outside the table footprint.
CR-4	All third-party software components shall be released under an open-source licence permitting academic use.
CR-5	The deployment environment is indoor, climate-controlled, diffuse fluorescent or LED lighting at 200–500 lux; outdoor operation is out of scope.
CR-6	The system shall be reproducible from this dissertation: every parameter required to rebuild a working copy shall be present in <code>config.py</code> or specified in the relevant subsection.
CR-7	The bipartite compute architecture (Jetson Nano for perception, separate MCU for low-level motor control) is fixed; single-board collapses are out of scope.

Table 4.2: Performance requirements with target values and measurement methods.

ID	Requirement	Target	Measurement
PR-1	Camera frame rate	≥ 240 fps	Camera specification sheet
PR-2	Ball-detection rate (rally average)	$\geq 95\%$	Per-frame detection log
PR-3	End-to-end latency, sense $\rightarrow$ act	≤ 50 ms	Logic-analyser probe on UART
PR-4	Landing-prediction RMSE	≤ 10 cm	Ground-truth bounce measurement
PR-5	Sustained perception loop rate	≥ 15 fps	Wall-clock loop instrumentation
PR-6	Per-stage timing jitter (1σ)	≤ 2 ms	<code>perf_counter_ns</code>
PR-7	Racket-solver convergence rate	$\geq 99\%$	<code>result.success</code> count
PR-8	Safe-stop response time	≤ 200 ms	Fault-injection test bench

Table 4.2 is the single source of truth for PR-1 through PR-8; the verification subsection of Section 4.6 cross-references it rather than restating the targets, and the measurement results are reported in Chapter 5.

Requirements traceability

Each requirement is implemented by one or more design subsections in the remainder of this chapter. Table 4.3 provides a forward-traceability matrix from requirement to subsection; reverse traceability is given inline at the start of each implementation subsection in 4.3 onwards.

Table 4.3: Forward traceability from requirements to implementing design subsections.

Requirement(s)	Implementing section	Notes
FR-1, FR-2, FR-3, FR-4	4.6.5	Perception pipeline
FR-5	4.6.7	Control pipeline (solver + launcher)
FR-6	4.6.7.3	UART transport to MCU
FR-7	4.6.9	Failure-mode handling
FR-8	4.6.8	Telemetry channel
PR-1, PR-5	4.6.5	Pipeline frame budget
PR-2	4.6.5.1	Detection rate
PR-3, PR-6	4.6.4	Pipeline orchestration and timing
PR-4	4.6.6.1	Trajectory predictor accuracy
PR-7	4.6.7.1	Nelder–Mead convergence policy
PR-8	4.6.9	Safe-stop response
ER-1, ER-2, ER-4–ER-7	4.4	Power distribution and isolation
MR-4, MR-6	4.3	Camera mount, three-wheel launcher geometry
SR-1–SR-3, SR-5, SR-6	4.6	Jetson-side runtime and architecture
SR-7, SR-8	4.7	MCU firmware framing and recovery
CR-1	throughout	ITTF compliance
CR-2	4.6.6.1, 4.6.9	Off-table guard, safe-stop
CR-4	4.6	Open-source software stack
CR-5	4.6.5.1	Lighting assumption recorded
CR-6	4.6.3	Single-source-of-truth <code>config.py</code>
CR-7	4.6.1	Bipartite architecture

Each of the five research objectives stated in Chapter 1 (1.5) is realised by an identifiable subset of the requirements catalogue: Objective 2 (trajectory and impact modelling) by {FR-3, FR-4, PR-4}; Objective 3 (simulation of the perception–actuation loop) by {FR-1, FR-2, FR-5, FR-6, SR-1–SR-3, CR-7}; Objective 4 (data collection and statistical analysis) by {FR-8, PR-1–PR-8, SR-5}; and Objective 5 (future work) by the known-limitations summary in 4.6.11. Objective 1 is methodological and produces no system requirement directly.

4.3 Physical Design

The physical design of the table tennis robot was developed to create a stable, reliable, and efficient robotic platform capable of supporting automated table tennis training op-

erations. The design process focused on integrating the structural frame, launcher mechanism, wheel arrangement, camera system, and supporting components into a single assembly that can operate effectively while maintaining mechanical stability and ease of maintenance. Three-dimensional modeling techniques were used during the development stage to visualize the robot structure and evaluate the arrangement of components before manufacturing and assembly.

The robot structure consists of a rigid chassis that serves as the main supporting framework of the system. The chassis was designed to support the mechanical and electronic components while maintaining sufficient strength to withstand operational forces and vibration generated during ball launching. Structural stability is an important requirement because movement or vibration within the frame can reduce launching accuracy and negatively affect overall system performance. The frame was therefore designed to provide adequate rigidity while maintaining a lightweight structure suitable for robotic operation.

The arrangement of components within the chassis was selected carefully to improve balance and weight distribution throughout the system. Heavy components such as motors, launcher wheels, and power supply units were positioned strategically to avoid excessive loading on one section of the robot. Proper weight distribution improves the stability of the robot during operation and reduces unnecessary stress on the supporting frame. Sufficient spacing was also maintained between components to simplify assembly procedures and improve accessibility during maintenance operations.

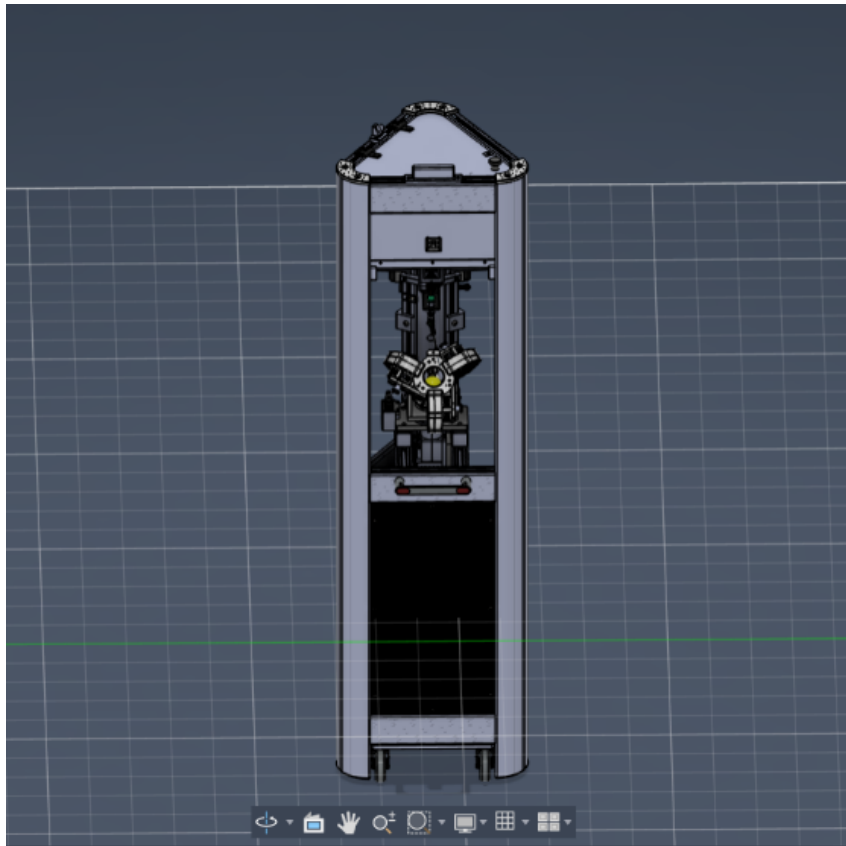


Figure 4.1: Physical Design

The launcher mechanism forms one of the most important sections of the robot because it is responsible for projecting table tennis balls toward the player during training exercises. The launcher assembly consists of rotating wheels and support structures that guide and accelerate the ball before release. The positioning of the launcher wheels was designed carefully to ensure smooth ball movement and consistent launching performance. Clearance between moving parts was maintained throughout the design to prevent mechanical interference and reduce component wear during continuous operation.

The physical design also incorporated a vision system to support future applications involving ball tracking, object detection, and trajectory analysis. The camera system was integrated into the upper section of the robot frame to provide a clear viewing area toward the playing surface. Mounting the camera above the launcher assembly improved visibility of the table and reduced obstruction caused by structural components and rotating mechanisms. The selected position allows the camera to observe the ball launching region and a large section of the playing area simultaneously.

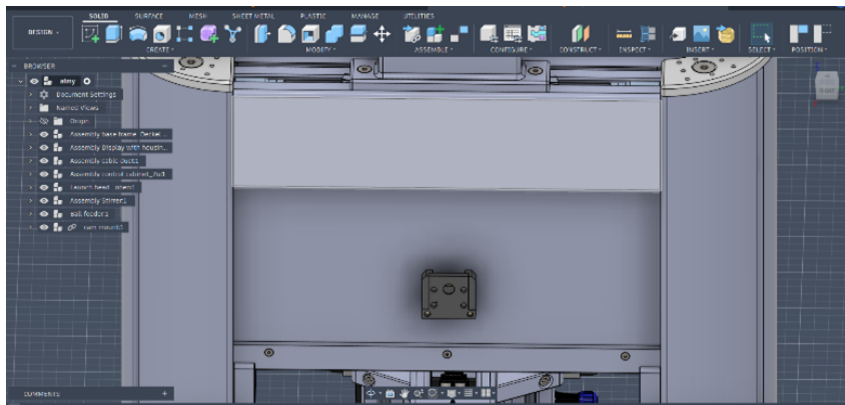


Figure 4.2: Camera mount

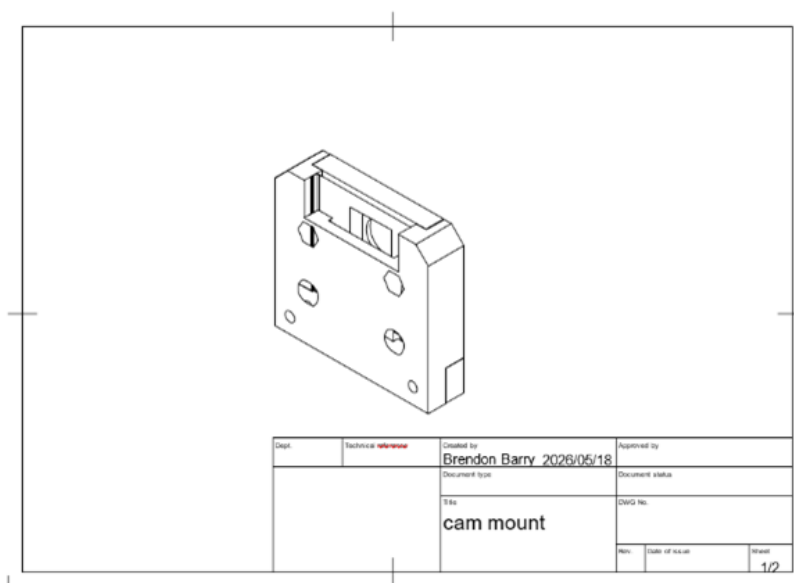


Figure 4.3: Drawing 1/2 (Camera Mount)

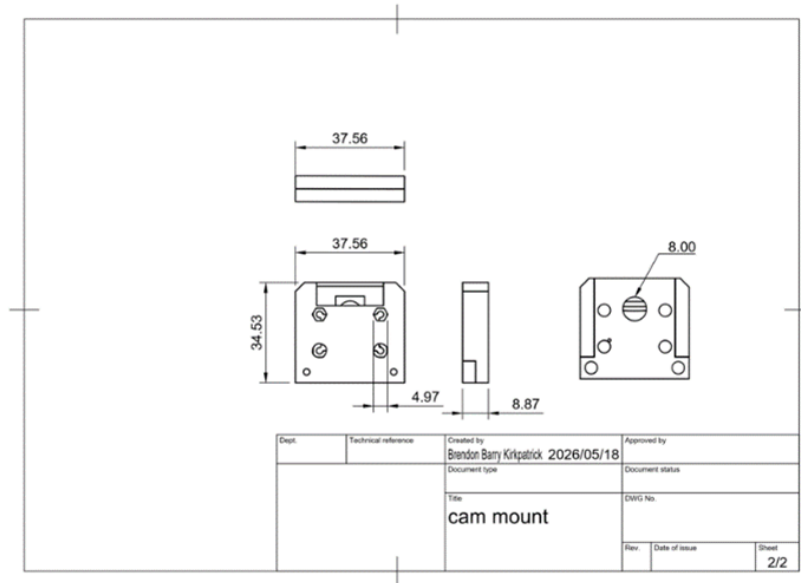


Figure 4.4: Drawing 2/2 (Camera Mount)

A major feature added to the camera mounting system was an adjustable support mechanism designed to allow precise angle adjustment. The camera mount includes a threaded hole arrangement that enables controlled positioning of the camera during setup and testing. This adjustment mechanism improves flexibility because the viewing angle of the camera can be modified easily depending on operational requirements and testing conditions. The threaded adjustment system also improves accuracy by allowing fine positioning of the camera without requiring major structural modifications to the robot assembly.

The adjustable camera mount provides several advantages within the robotic system. Firstly, it allows the camera to be aligned accurately with the playing surface and launcher direction. Secondly, the adjustment mechanism enables future modifications and calibration procedures to be performed more efficiently. During testing and operation, small changes in camera orientation may be required to improve image capture and tracking performance. The threaded adjustment feature simplifies this process and allows precise repositioning while maintaining mechanical stability.

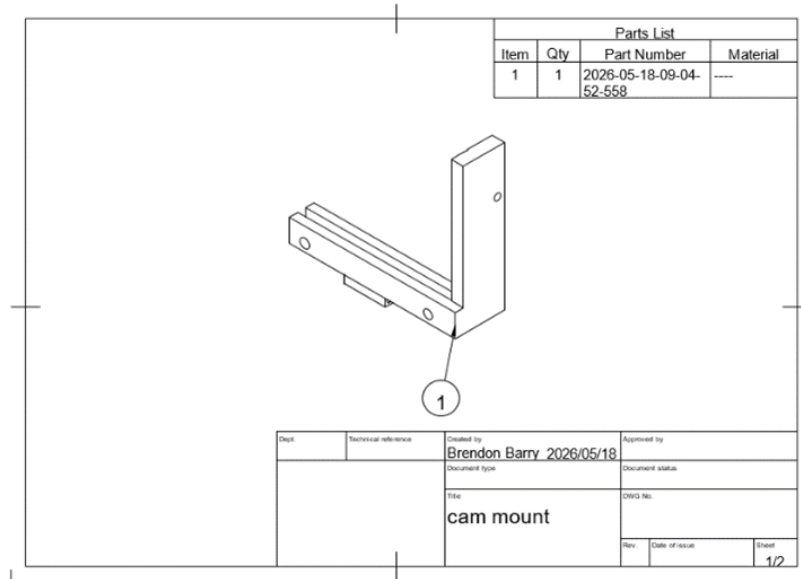


Figure 4.5: Drawing 1/2 (angle adjustment)

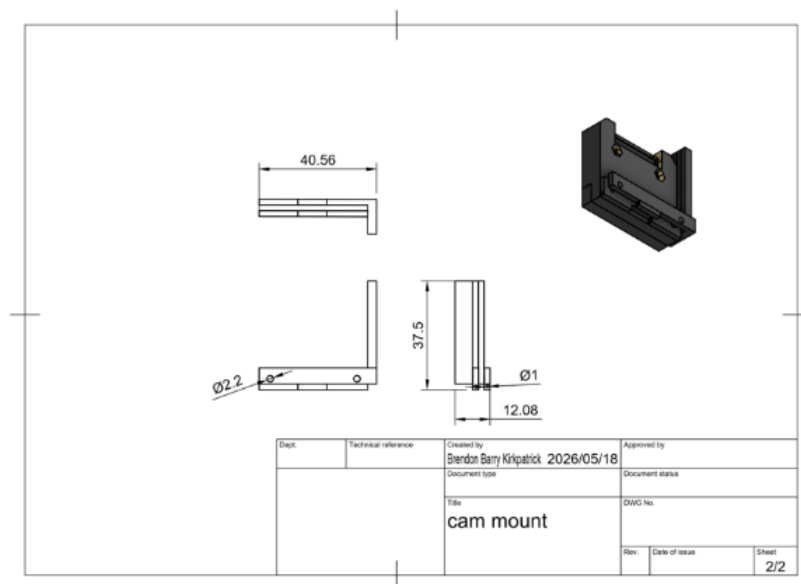


Figure 4.6: Drawing 2/2 (angle adjustment)

The camera mounting bracket was designed to provide sufficient rigidity and support while remaining lightweight and compact. The bracket structure secures the camera firmly to the robot frame and minimizes unwanted movement or vibration during operation. Stability of the camera is important because vibrations can reduce image quality and negatively affect future vision-processing applications. The bracket was therefore designed to provide reliable support while allowing easy adjustment and maintenance.

The field of view of the camera was considered carefully during the design stage. Different viewing perspectives were evaluated to ensure that the camera maintained clear visibility of the playing area without obstruction from launcher wheels, support

brackets, or surrounding structures. The elevated mounting position improved the viewing coverage of the table tennis surface and enhanced the ability of the camera to monitor ball movement effectively. The adjustable mounting system further improved visibility by allowing modifications to the viewing direction whenever necessary.

Material selection formed another important aspect of the physical design process. The robot structure requires materials that provide sufficient strength while minimizing unnecessary weight. Lightweight materials improve efficiency and reduce loading on motors and structural supports. Materials such as aluminum, acrylic, and engineering plastics are suitable for different sections of the assembly depending on the required mechanical properties and manufacturing requirements.

Aluminum was considered suitable for structural sections because it offers high strength, corrosion resistance, and relatively low mass. Acrylic materials can be used for protective covers and lightweight support structures due to their ease of fabrication and low weight. Plastic components manufactured through additive manufacturing techniques can also be incorporated for customized brackets, cable guides, and mounting structures. The use of different materials allows the robot to maintain structural strength while improving manufacturing flexibility.

Cable management was also considered during the physical design stage. The robot contains multiple electrical systems including motors, controllers, sensors, and the camera assembly. Proper routing of cables was necessary to reduce clutter and prevent interference with moving components. The design aimed to minimize exposed wiring and organize electrical connections along controlled pathways within the structure. Effective cable management improves safety, simplifies maintenance procedures, and enhances the professional appearance of the robotic system.

Ease of maintenance and future modification were important considerations throughout the design process. Robotic systems often require adjustments, repairs, and upgrades during operation and testing. Components within the robot assembly were therefore positioned to allow convenient access during servicing procedures. The modular arrangement of the structure also supports future expansion and modification of the system without requiring complete redesign of the robot framework.

After completion of the assembly design, engineering drawings were generated to document the final physical structure of the robot. These drawings included orthographic views such as front, side, and top projections, together with isometric assembly views of the complete system. Orthographic drawings provide accurate dimensional information required for manufacturing and assembly processes. These technical drawings also improve communication of design information by illustrating the relationship between components clearly.

Detailed assembly drawings were additionally developed to illustrate the placement of the launcher mechanism, camera mounting system, wheel arrangement, and struc-

tural supports. Dimensions and annotations were included where necessary to improve clarity and provide sufficient manufacturing information. Exploded assembly views were also created to demonstrate how the individual components fit together within the complete robot structure. Exploded views simplify understanding of the assembly sequence and assist during manufacturing and maintenance procedures.

Rendered images of the completed robot assembly were generated to provide realistic visual representations of the final physical design. These rendered views improve the presentation quality of the project and assist in communicating the appearance and structural layout of the robot during technical discussions and demonstrations.

Overall, the physical design stage successfully produced a stable and functional robotic platform capable of supporting automated table tennis training applications. The completed design integrates the launcher mechanism, camera system, adjustable mounting structure, and supporting framework into a single assembly capable of supporting future developments in robotic control, ball tracking, and intelligent training systems. Important engineering considerations such as stability, visibility, adjustability, maintainability, material selection, and cable management were addressed effectively during the design process to ensure that the robot can operate reliably and support future improvements.

4.4 Electrical Design

4.4.1 Electrical Overview

The electrical system was created to give a stable power supply, fine control, and dependable communication between the processing hardware and the moving components of the robotic table tennis system. The design is centred on a multi-rail DC power distribution system that starts with a single high-current 24V power supply unit.

This design keeps the calculation, control, and movement systems on different voltage levels. This helps keep a stable electrical system and reduces interference between high-current and low-voltage electronics.

The electrical design is made up of:

- A 24V power rail.
- A 5V system.
- A 3.3V logic system.
- A Jetson Nano for vision processing.
- An STM32 microcontroller for motor control.

- Three electronic speed controllers.
- Three brushless DC motors.
- Four high-torque servo motors for aiming.

The complete system is shown in Figures 4.7 and 4.9.

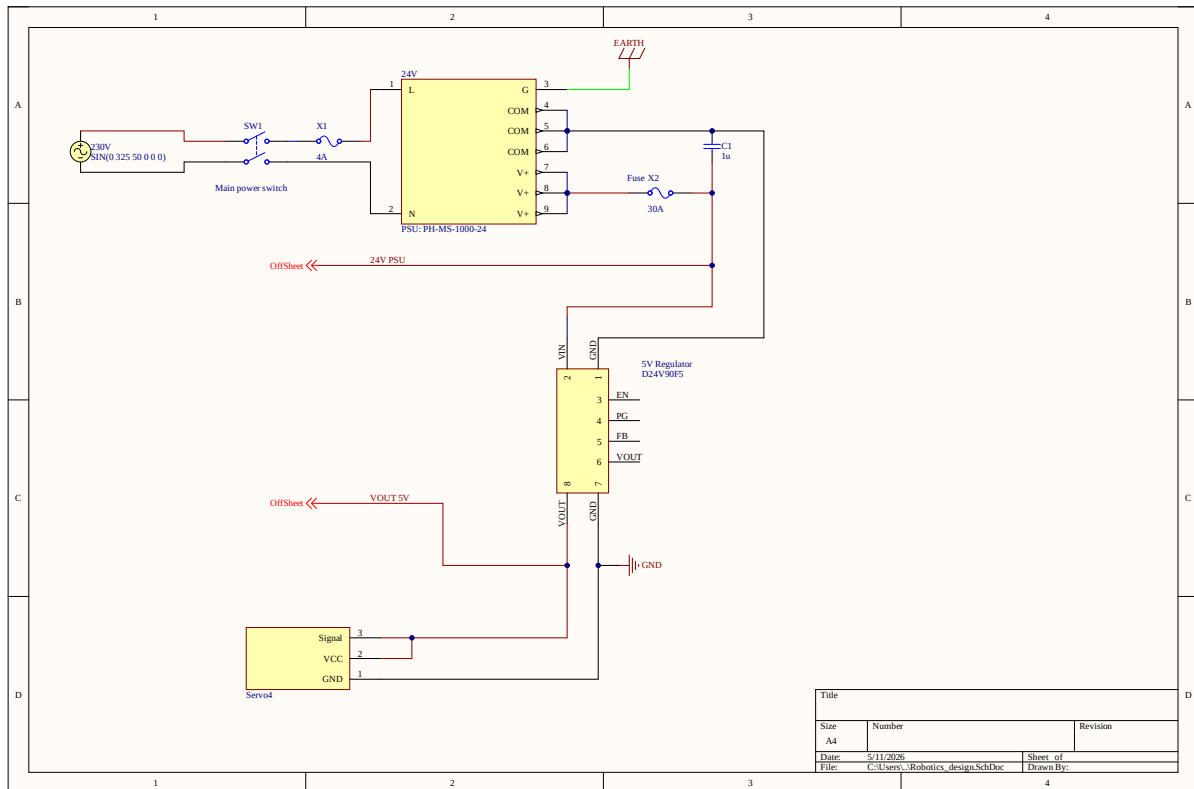


Figure 4.7: Main power distribution

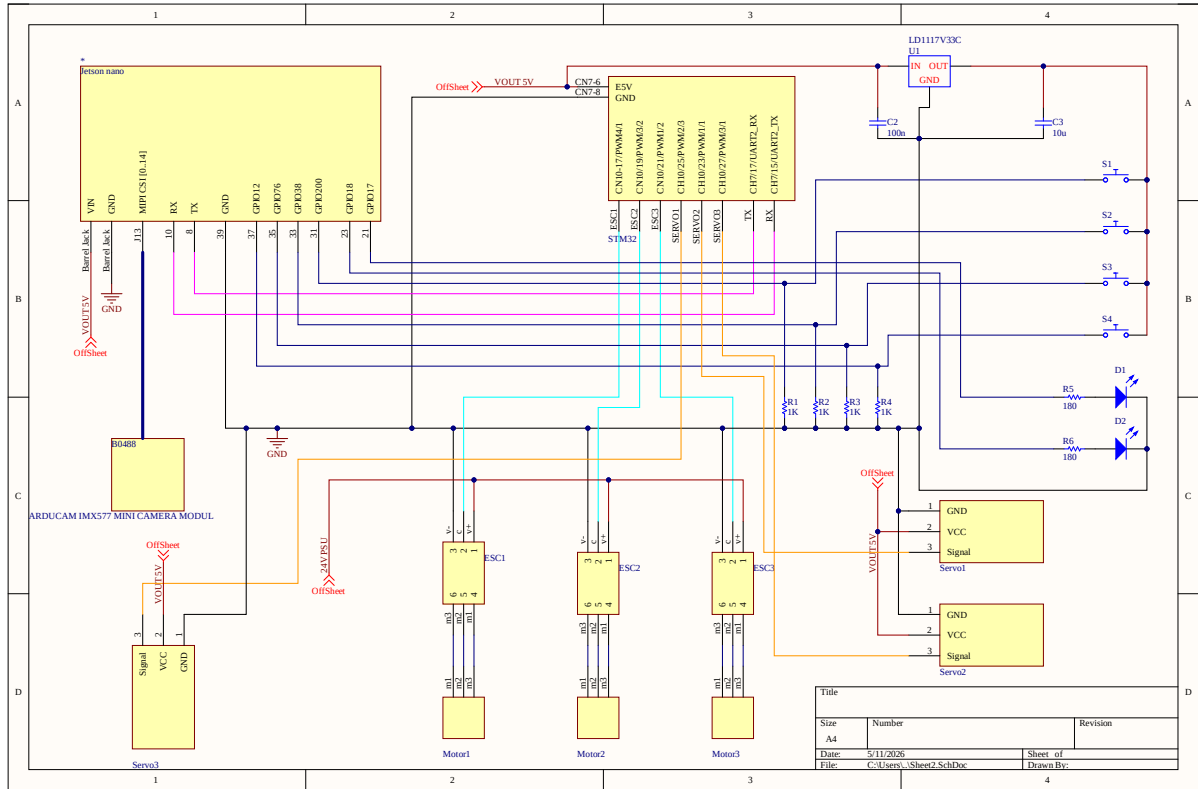


Figure 4.8: Control system, motor drive system, and logic system

4.4.2 Power Distribution

The system uses a PH-MS-1000-24 switched-mode PSU.

This gives 24V DC at a high current of 40 AMPS. The 24V setup was chosen because the launcher motors need a lot of power, and this voltage helps stop too much current draw during quick acceleration.

The power formula is:

$$P = VI \quad (4.1)$$

For a fixed power requirement, increasing the supply voltage reduces the current needed:

$$I = \frac{P}{V} \quad (4.2)$$

Running the launcher subsystem at 24V lowers the current in the wires, reduces resistive losses, and makes the electrical system more efficient.

The AC mains input goes through a main rocker switch and fuse protection before reaching the PSU. Extra DC-side fuse protection was added to help prevent overcurrent damage to other components.

The diagram below shows a simplified version of the power distribution setup:

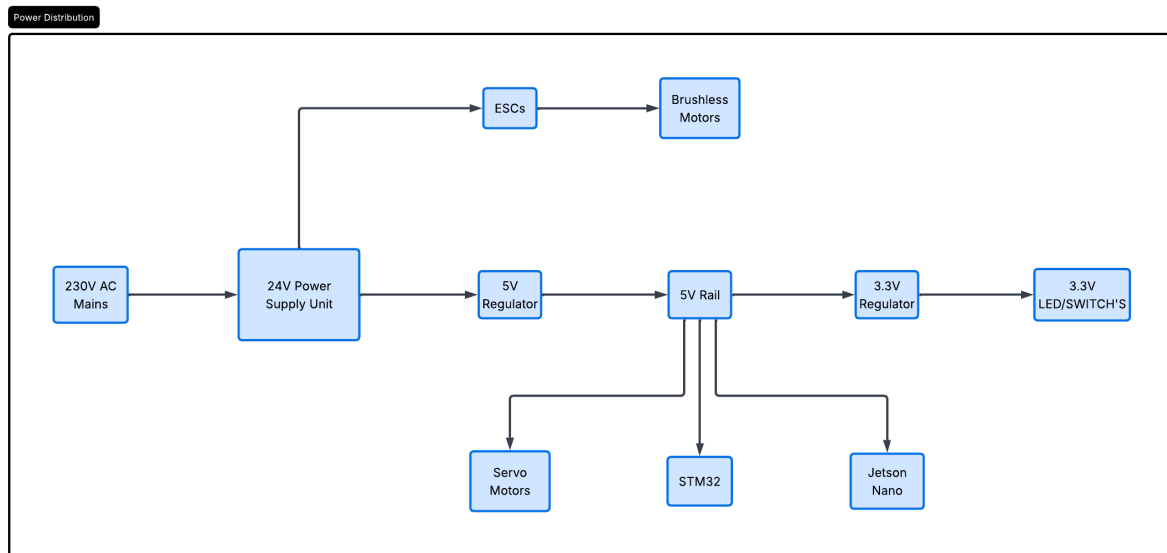


Figure 4.9: Power Distribution

4.4.3 Voltage Regulation

The electrical design uses multiple regulated-voltage sectors to match the requirements of different subsystems.

4.4.3.1 24V

The 24V rail is the main high-current distribution and powers the speed controllers connected to the launcher motors. High-current devices were separated from low-voltage logic components to reduce noise.

4.4.3.2 5V

A D24V90F5 regulator is being used to make a 5V rail. This is a buck regulator that allows for a continuous 9A current output enough to power all components needed.

This rail powers:

- The Jetson Nano
- The Servo motors

- The STM32

The 5V rail was chosen as it supplies sufficient current to support the servo's under load at the same time provides stable power to the Jetson Nano.

4.4.3.3 3.3V

An LD1117V33C voltage regulator is a low drop voltage regulator and was used to generate 3.3V power to the 3.3V rail. It supplies up to 800mA. This subsystem powers the LEDs and switch's. Decoupling capacitors were used on the regulator input and output sides to improve voltage stability and stop any high-frequency switching noise.

4.4.4 Control Systems

The control system was separated into two different processing subsystems: the Jetson Nano and STM32.

4.4.4.1 Jetson Nano (Vision Processing)

The Jetson Nano is the high-level subsystem that does the following:

- Camera capture
- Vision processing
- Ball detection
- Trajectory estimation
- Motion prediction

The Jetson Nano was chosen as it uses GPU acceleration and works well with machine vision libraries like OpenCV.

4.4.4.2 STM32 (Real-Time Control)

The STM32 performs low-level control tasks such as:

- PWM signal generation
- ESC control
- Servo actuation

Linux-based embedded systems like the Jetson Nano are less suitable for precise timing operations as they are non-deterministic. The STM32 compensated well for this as it provides hardware-level timing precision for motor actuation and PWM signal generation.

4.4.4.3 UART Communication

The Jetson Nano and STM32 communicate through a UART serial interface using the TX and RX pins on both boards. UART was chosen because of its compatibility with real-time systems as well as the ease of debugging.

4.4.5 Motor Drive System

The launcher uses three brushless DC motors and is controlled through electronic speed controllers (ESCs). Each ESC receives 24V power from the PSU while receiving PWM signals from the STM32.

Brushless motors were chosen for the following reasons:

- Improved efficiency
- Less mechanical wear
- Stable high-speed operation

The ESCs tell the motors and regulate the velocity according to the PWM generated by the STM32.

4.4.6 Servo Actuation

Four servo motors were chosen to move the system into the correct position for launching of the ball.

The servo system lets us control:

- Angle of the ball launcher
- Direction of the launcher

Each servo receives PWM signals from the STM32.

4.4.7 Grounding

All systems share a common ground to make sure that the signals are uninterrupted and create a stable communication between the Jetson Nano, STM32 and ESCs.

A common ground is necessary for:

- UART communication stability
- PWM signal reference stability
- Noise reduction

- Voltage reference stability

Keeping high-current power separate from low-voltage wires helps reduce EMF interference and makes the system more reliable.

4.4.8 Safety

Protections used:

- AC-side fuse
- DC-side fuse
- Voltage regulation
- Isolated voltage domains
- Regulated low-voltage rails

4.5 Vision Control Design

The vision subsystem is the brain of the robotic table tennis system. Its Main responsibility is to capture real-time visual data, process incoming frames, and execute trajectory prediction algorithms. To accomplish this, we needed a powerful platform with GPU-accelerated processing abilities.

The Jetson Nano 4GB Developer Kit was chosen as the vision control unit because of its ability to support computer vision workloads in a small form factor. The Jetson Nano provides steady performance for image processing libraries such as OpenCV and machine learning that is required for object detection and motion estimation.

The Jetson was set up with a 32GB MicroSD card for storage and to store the operating system as well as to run the loaded programs. Stable processing was provided through a dedicated 5V/4A USBC power supply, which supplies sufficient current. Devices such as a keyboard, mouse, and network connectivity were used during the first operation and testing, after which the system operated in headless mode for integration.

4.5.1 Software Environment Setup

The software used was NVIDIA's JetPack Software Development Kit, which is a Linux-based operating system optimized for the Jetson architecture.

The operating system image was downloaded onto the MicroSD card using a separate computer. The flashing process made sure that the embedded platform had the necessary drivers and runtime dependencies as well as kernel support.

The operating system image was written to the MicroSD card using BalenaEtcher. This process created the bootable environment.

4.5.2 Hardware

After setting up the software, the hardware was assembled. The MicroSD card was placed into the Jetson Nano, then network connectivity and input devices were plugged in.

This lets us validate the hardware and software.

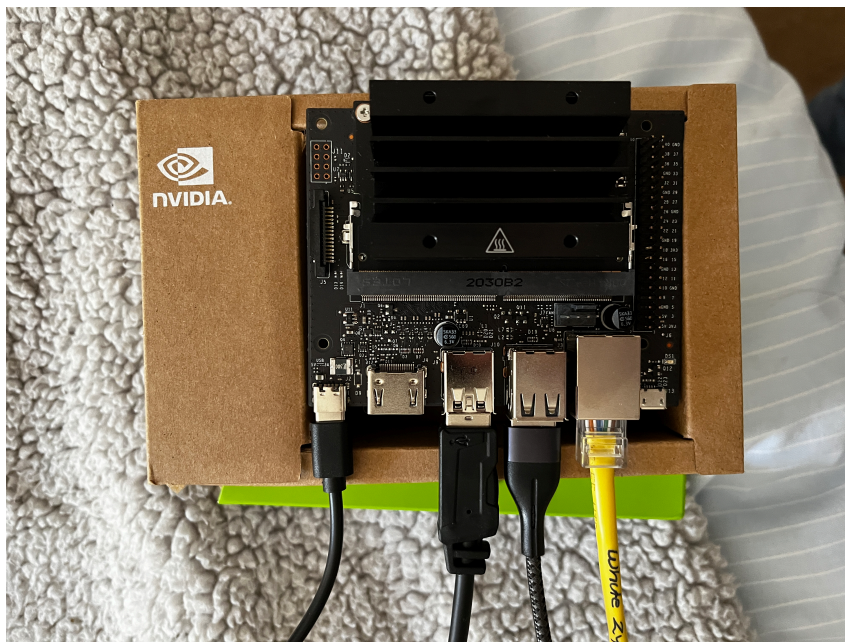


Figure 4.10: Physical hardware Setup

4.5.3 Headless Configuration

The Jetson Nano was configured for headless operation so we could remotely access the device. This design enables remote access during field deployment and testing in theory.

The system configuration was done with a serial console interface connected to a host computer. This lets us initialize the system without the need for peripheral devices.

The headless approach provides practical advantages for embedded robotics, including lower power consumption and better portability.

4.5.4 Remote Communication

For remote interaction, Secure Shell (SSH) access was enabled over the local network. This method supports command-line debugging without requiring physical access to the Jetson.

The integration of SSH supports scalable deployment in testing and working environments.

4.5.5 Design Significance

The vision control system serves as the intelligence layer of the robotic table tennis trainer. By using GPU-enabled computation and remote accessibility, the design becomes a reliable processing system for real-time visual tasks.

This system interfaces directly with the machine vision pipeline and sends processed outputs to the control system, making it a closed-loop robotics system.

4.6 Computational Software Design

Chapter 3 derived the mathematical models that govern every stage of the perception-to-actuation loop, from monocular projection through spin estimation, aerodynamic flight, ball–racket impact, and launcher kinematics. The present section closes the loop between mathematics and machine. It documents the design decisions that translate those models into software executing on an embedded compute platform, identifies the constraints that shape those decisions, and traces each implementation module back to the equation or algorithm in Chapter 3 that it realises.

Three principles guide the design throughout. First, *traceability*: every subsystem must be linked to a specific section or equation of Chapter 3, so that an examiner or successor engineer can audit correctness without re-deriving the underlying physics. Second, *bounded latency*: the system operates on a 240 fps camera stream and must produce actuator commands within a budget that preserves the dynamics of a live rally. Third, *graceful degradation*: detection failures, optimiser non-convergence, and communication faults must be detected and survived, not allowed to propagate as silent corruption into the motor controller.

4.6.1 Overview and Relationship to the Modelling Chapter

The computational software is organised as a single Python process running on an embedded compute board. Each frame from the camera passes through eight ordered stages of detection, reconstruction, filtering, prediction, and control before producing a packet that is transmitted over a serial link to a separate motor-control microcontroller. The control loop is therefore split between two compute elements: the embedded board

performs perception and high-level control, while the microcontroller closes the low-level loops around the motors.

Table 4.4 establishes the traceability from this design section back to the equations and algorithms of Chapter 3 and forward into the implementation modules under `modules/`.

Table 4.4: Traceability map from Modelling chapter equations, through design subsections, to implementation modules.

Modelling reference	Design subsection	Implementation module
3.2 Camera capture	4.6.4 , 4.6.5.1	<code>pipeline.py</code>
3.3 2-D detection (eq. 3.1)	4.6.5.1	<code>modules/detection/</code>
3.4 3-D reconstruction (eq. 3.2)	4.6.5.2	<code>modules/reconstruction/depth.py</code>
3.5 Spin axis (eq. 3.24)	4.6.5.4	<code>modules/tracking/spin.py</code>
3.6 Trajectory (eqs. 3.25–3.27, 3.41)	4.6.6.1	<code>modules/physics/trajectory.py</code>
3.7 Impact (eqs. 3.72–3.115)	4.6.6.2	<code>modules/physics/impact.py</code>
3.8 Racket orientation	4.6.7.1	<code>modules/control/racket_solver.py</code>
3.9 Launcher commands	4.6.7.2 , 4.6.7.3	<code>modules/control/launcher.py</code>

Failure-mode handling (Section [4.6.9](#)) is treated as a cross-cutting concern that is not addressed in Chapter 3 because it belongs to design rather than to model derivation: a model that is correct on paper is of no use if it cannot be recovered from after a transient fault ([Liu 2000](#)).

4.6.2 Computational Requirements and Design Constraints

The quantified targets that bound this subsection are stated once in the requirements catalogue of Section 4.2 (PR-1–PR-8 in Table 4.2, SR-1–SR-8 in Table 4.1); the design decisions that follow inherit those targets rather than restate them. Three constraints specific to the computational stack drive the architectural choices in the remainder of this section: the $\Delta t \approx 4.17$ ms camera frame interval at 240 fps; the Jetson Nano 4 GB FP16 compute envelope of ~ 472 GFLOPS sharing 4 GB LPDDR4 between CPU and GPU; and the headless-Linux deployment mode (no display server) imposed by the robot enclosure. Because the inference cost of the chosen detector (Section [4.6.5.1](#)) exceeds the per-frame budget, a best-effort scheduling policy in the sense of [Liu \(2000\)](#) is adopted: the perception loop runs at the sustained inference rate, and the Kalman filter (Section [4.6.5.3](#)) absorbs the rate mismatch through its measured- Δt prediction step.

4.6.3 Repository Architecture and Configuration

The codebase is organised as a flat, single-level module hierarchy. Every subsystem corresponds to a directory under `modules/`, and the orchestration logic is a single Python file at the repository root:

```
pipeline.py      <- orchestration entry point
config.py        <- single source of truth for all parameters
modules/
  detection/     <- ball localisation (YOLO, colour, hybrid)
  reconstruction/ <- 2-D pixel -> 3-D world coordinates
  tracking/      <- Kalman filter and spin-axis estimation
  physics/       <- trajectory and impact simulation
  control/       <- racket orientation solver and launcher commands
  visualization/ <- HUD overlay and 3-D plot rendering
```

Every tunable parameter is declared exactly once in `config.py` and imported by the modules that need it. This design eliminates parameter skew between subsystems and allows calibration to be performed by editing a single file. The principle is consistent with the configuration guidance of [Wiggins \(2017\)](#) and the don't-repeat-yourself principle of [Hunt & Thomas \(1999\)](#).

The parameters declared in `config.py` fall into three semantic groups: *calibrated* parameters that depend on the deployed hardware (camera intrinsics, table height), *physical* parameters that are universal (ball mass, gravitational acceleration, ITTF standards), and *tuning* parameters that are empirical and subject to revision (Kalman noise covariances, Savitzky–Golay window length, detection thresholds). Table [4.5](#) lists the key constants grouped by category.

Table 4.5: Key parameters declared in `config.py`, grouped by semantic category.

Parameter	Value	Description
<i>Calibrated, depend on deployed hardware</i>		
CAMERA_MATRIX	$f \approx 1050$ px (placeholder)	Intrinsic matrix; calibrated per-deployment
TABLE_HEIGHT_CM	76.0 cm	ITTF standard, measured in situ
<i>Physical, universal</i>		
BALL_DIAMETER_CM	4.0 cm	ITTF 40 mm ball
BALL_MASS_KG	0.0027 kg	Ball mass
AIR_DENSITY	1.225 kg/m ³	Sea-level air density
GRAVITY	−9.81 m/s ²	Gravitational acceleration
<i>Tuning, empirical, subject to revision</i>		
FRAME_RATE	240 fps	High-speed camera acquisition rate
DT	1/240 s	Per-frame time step (≈ 4.17 ms)
DRAG_COEFF	0.47	Aerodynamic drag coefficient C_D
LIFT_COEFF	0.25	Magnus lift coefficient C_L
COEFF_RESTITUTION	0.8	Normal restitution coefficient C_r
FRICTION_COEFF	0.2	Coulomb friction coefficient μ
KALMAN_PROCESS_NOISE	0.5	Kalman Q diagonal scaling
KALMAN_MEAS_NOISE	5.0	Kalman R diagonal scaling
SAVGOL_WINDOW	9 frames	Savitzky–Golay window (≈ 37.5 ms)
SAVGOL_POLY	3	Savitzky–Golay polynomial order
TRAJECTORY_BUFFER_LEN	120 frames	Ring buffer capacity (0.5 s)
WHEEL_RADIUS_M	0.03 m	Launcher wheel radius r_w

The composite aerodynamic constants C_1 and C_2 (used in both trajectory simulation and spin estimation, eq. (3.7) of Chapter 3) are pre-computed once at import time in `config.py`, so the physics modules consume them as plain scalars rather than re-deriving them inside their inner loops.

4.6.4 Pipeline Orchestration and Concurrency Model

`pipeline.py` implements the main perception-to-control loop. Each video frame executes eight stages in a fixed, deterministic sequence:

1. **Kalman predict** propagate the state estimate forward by Δt .
2. **Ball detection** run the hybrid detector on the raw frame.
3. **3-D reconstruction and Kalman update** convert pixel detections to world coordinates and correct the filter.
4. **Ring-buffer push** append the new (t, x, y, z) tuple.
5. **Spin-axis estimation** differentiate the trajectory buffer.

6. **Trajectory and landing prediction** forward-simulate the aerodynamic trajectory.
7. **HUD overlay** render speed, depth, and landing information.
8. **3-D plot panel** generate and composite the trajectory plot.

The critical ordering decision is that the Kalman prediction (Stage 1) is executed *before* detection (Stage 2), so that the filter’s projected position is available to guide the detector’s search. This breaks the circular dependency between tracking and detection: the first successful detection bootstraps the filter; every subsequent frame benefits from a narrow, prediction-guided search window rather than a full-frame scan.

Buffer and concurrency. Trajectory history is stored in a fixed-length ring buffer of `TRAJECTORY_BUFFER_LEN = 120` samples (≈ 0.5 s at 240 fps), long enough for stable Savitzky–Golay differentiation in Stage 5 yet short enough to bound both memory and the per-frame cost. The orchestration runs single-threaded within one Python process; because YOLO inference dominates the per-frame cost (Section 4.6.5.1), the loop operates in the best-effort mode of Section 4.6.2 (Lee 2008) rather than meeting the $\Delta t \approx 4.17$ ms camera-frame budget directly.

4.6.5 Perception Subsystem

The perception subsystem comprises four sequential stages: ball detection, 3-D reconstruction, Kalman filtering, and spin-axis estimation. Each stage consumes the output of the previous one and is described in turn.

4.6.5.1 Ball Detection: YOLO + HSV Hybrid

Reliable ball localisation under high-speed motion is the most brittle stage of the pipeline. A single detector is insufficient: a CNN-based detector is susceptible to motion blur at 240 fps, while colour-based methods produce false positives from other white objects (shirts, court markings) in the scene. The `HybridDetector` class resolves this tension with a two-stage fallback strategy.

Stage 1: YOLO detection. YOLOv11 is queried on the full frame and detections are filtered to COCO class 32 (*sports ball*). Surviving candidates must additionally pass a chain of domain-specific filters applied in `yolo_detector.detect`:

1. Confidence ≥ 0.35 .
2. Bounding-box size below a maximum threshold (rejects players and large background objects).

3. Centroid within $\text{MAX_DETECTION_DIST_PX} = 120$ px of the Kalman-predicted position.
4. Frame-to-frame motion $\geq \text{MIN_MOTION_PX} = 2$ px (rejects stationary false positives).
5. Hough circle verification confirming approximate circularity.

The best surviving candidate is returned with tag 'YOLO'. The proximity and motion filters are particularly important at 240 fps: they exploit the fact that between consecutive frames the ball can move by at most a few pixels, allowing the search space to be dramatically reduced compared to full-frame detection. The proximity gate value of 120 px is dimensioned from the worst-case inter-frame ball displacement at the camera's closest range; at 25 m/s ball speed, $\Delta t = 4.17$ ms, and a typical close-range scale of ~ 10 px per cm, the maximum displacement is ~ 100 px, which leaves margin for camera jitter.

Stage 2: HSV colour fallback. When YOLO returns no candidate and the Kalman filter has been initialised (`self.first_detection` is True), the filter's predicted 3-D state is back-projected to pixel coordinates via `project_2d`, defining a 120×120 pixel region of interest (ROI). HSV thresholding for white objects ($V \in [140, 255]$, $S \in [0, 50]$) is applied within this ROI only, followed by morphological erosion and dilation to suppress noise, and contour analysis to extract the best circular candidate. Detected coordinates are offset back to full-frame coordinates before returning.

```

1 if self.first_detection and pred_xy is not None:
2     half = ROI_SIZE_PX // 2
3     x1 = int(max(0, pred_xy[0] - half))
4     y1 = int(max(0, pred_xy[1] - half))
5     x2 = int(min(w, x1 + ROI_SIZE_PX))
6     y2 = int(min(h, y1 + ROI_SIZE_PX))
7     roi = frame[y1:y2, x1:x2]
8     color_result = find_ball_by_color(roi)
9     if color_result is not None:
10         rx, ry, rd = color_result
11         xp, yp = (float(rx + x1), float(ry + y1))

```

Listing 4.1: Fallback ROI extraction in `HybridDetector.detect`.

Confining colour detection to the predicted ROI is the key design choice that makes this fallback viable: the search is restricted to the region where the ball is actually expected, eliminating the false-positive rate that would defeat a full-frame colour detector. The fallback is gated on `self.first_detection` to prevent the colour stage from firing before the filter has been initialised and a reliable prediction is available.

Deployment to the Jetson Nano. The detector specified in `config.YOLO_MODEL_PATH` is the small (`yolo11s.pt`) variant of YOLOv11. Deployment on the Jetson Nano proceeds in three stages: the trained PyTorch checkpoint is exported to ONNX on the development workstation; the ONNX model is built into a TensorRT FP16 engine on the target Nano; and the engine is loaded by the Ultralytics runtime at pipeline start-up. Expected sustained inference throughput on this hardware, after FP16 optimisation and at 640×640 input resolution, is in the order of 10–20 fps. This is well below the camera frame rate but is consistent with the best-effort policy of Section 4.6.2: the Kalman filter (Section 4.6.5.3) absorbs the rate mismatch through its measured- Δt prediction step. If higher throughput is required, the design admits a fall-back to the nano (`yolo11n.pt`) variant of the model, at the cost of small decreases in detection accuracy.

4.6.5.2 Monocular 3-D Reconstruction

With a single camera, metric depth cannot be recovered from pixel coordinates alone. The known physical diameter of an ITTF-regulation table tennis ball ($D = 40$ mm) provides the additional constraint required by the pinhole camera model (Hartley & Zisserman 2003). Combining the thin-lens projection with the known diameter:

$$z = \frac{D_{\text{phys}} \cdot f_{\text{eff}}}{d_{\text{px}}}, \quad x = \frac{(u - c_x) z}{f_x}, \quad y = \frac{(v - c_y) z}{f_y} \quad (4.3)$$

where $f_{\text{eff}} = (f_x + f_y)/2$ is the mean focal length extracted from the calibration matrix, (c_x, c_y) is the principal point, and d_{px} is the detected ball diameter in pixels. This implements the camera projection model of eq. (3.2) in Chapter 3, specialised to the known-diameter case. The implementation guards against divide-by-zero with a 10^{-9} additive epsilon in the denominator for the case where a detector returns a degenerate near-zero diameter.

Depth-noise propagation. The dominant accuracy limit of monocular depth-from-diameter is pixel-quantisation noise in the measured ball diameter. Differentiating the depth equation gives

$$\sigma_z \approx \frac{z^2 \sigma_d}{D f}, \quad (4.4)$$

so depth noise grows quadratically with range. For the present system parameters ($D = 40$ mm, $f \approx 1050$ px, $\sigma_d \approx 0.5$ px) the predicted depth noise crosses the assumed Kalman measurement noise of ~ 5 cm at approximately 2 m range and reaches ~ 10 cm by 3 m; the full sensitivity sweep is presented in Chapter 5. The design implication is a hard camera-placement constraint: the camera must be positioned so that the active play region falls within ~ 2 m of the lens, beyond which the filter's R covariance

becomes optimistic. The Kalman filter (Section 4.6.5.3) absorbs the noise within this window by weighting the physics-based prediction more heavily than the measurement when the signal-to-noise ratio is low.

The companion function `project_2d` inverts the same model to back-project a 3-D Kalman state into pixel coordinates, providing the ROI centre for the fallback colour detector. The stereo alternative used in some industrial-grade table-tennis robots (Schörghuber et al. 2021) is not adopted here for cost and calibration-complexity reasons, but is identified as a natural upgrade path in Section 4.6.11.

4.6.5.3 Kalman Filter State Estimator

The 6-state constant-velocity Kalman filter is the temporal backbone of the pipeline. The state vector $\mathbf{x} = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^\top$ encodes 3-D position (cm) and velocity (cm/s), allowing velocity to be inferred from a sequence of position measurements even though only position is observed directly (Welch & Bishop 1995).

State transition. A constant-velocity assumption gives the transition matrix:

$$F = \begin{bmatrix} I_3 & \Delta t I_3 \\ \mathbf{0} & I_3 \end{bmatrix}, \quad \mathbf{x}_{k|k-1} = F\mathbf{x}_{k-1}, \quad P_{k|k-1} = FP_{k-1}F^\top + Q. \quad (4.5)$$

Measurement update. The measurement matrix $H = [I_3 \mid \mathbf{0}]$ selects only the position sub-state, since depth estimation (Section 4.6.5.2) yields position but not velocity directly:

$$K = P_{k|k-1}H^\top(HP_{k|k-1}H^\top + R)^{-1}, \quad \mathbf{x}_k = \mathbf{x}_{k|k-1} + K(\mathbf{z}_k - H\mathbf{x}_{k|k-1}). \quad (4.6)$$

The noise matrices are isotropic: $Q = 0.5 I_6$ and $R = 5.0 I_3$. The ratio $R/Q = 10$ reflects that monocular 3-D reconstructions are significantly noisier than the constant-velocity motion model, particularly at 240 fps where the inter-frame displacement is very small and pixel-quantisation noise (Section 4.6.5.2) dominates the depth estimate. The filter is initialised with a large diagonal covariance $P_0 = 1000 I_6$, allowing it to converge quickly from any initial state. The smoothed velocity estimate $\mathbf{x}[3 : 6]$ is used both for HUD speed display and as the input to spin-magnitude estimation.

A constant-velocity model is used rather than an Extended Kalman Filter through eq. (3.41); at 240 fps the monocular depth noise of Section 4.6.5.2 dominates the linearisation error so the additional Jacobian cost is not justified, and the resulting curvature under-prediction in heavy-topspin shots is acknowledged in Section 4.6.11 (Bar-Shalom et al. 2001).

4.6.5.4 Spin-Axis Estimation

Spin estimation implements eq. (3.24) of Chapter 3. Rearranging the aerodynamic force balance to isolate the Magnus term yields the spin-axis estimator:

$$\hat{\mathbf{e}}_\omega = \frac{\mathbf{v} \times (\mathbf{a} - \mathbf{g} + C_1 |\mathbf{v}| \mathbf{v})}{C_2 |\mathbf{v}|^3}, \quad (4.7)$$

where $\mathbf{v}$ and $\mathbf{a}$ are velocity and acceleration at the evaluation point, $\mathbf{g}$ is the gravitational acceleration vector, and C_1, C_2 are the pre-computed aerodynamic constants from `config.py`.

Numerical differentiation and conditioning. Numerical differentiation of the trajectory buffer for $\mathbf{v}$ and $\mathbf{a}$ uses a Savitzky–Golay filter of polynomial order 3 over a sliding window of 9 samples (≈ 37.5 ms at 240 fps), which suppresses high-frequency noise without phase lag and preserves the polynomial shape that smooth ball trajectories approximate (Schafer 2011). The denominator $C_2 |\mathbf{v}|^3$ becomes ill-conditioned as $|\mathbf{v}| \rightarrow 0$; the function returns `None` until `SAVGOL_WINDOW = 9` samples have accumulated, guards against non-finite results, and below $|\mathbf{v}| \approx 1$ m/s holds the most-recent valid estimate and increments a staleness counter.

4.6.6 Physics and Prediction Subsystem

The physics subsystem comprises two modules: forward simulation of the aerodynamic trajectory between racket impacts, and rigid-body simulation of the impacts themselves. Together they realise Sections 3.6 and 3.7 of Chapter 3.

4.6.6.1 Aerodynamic Trajectory Simulation

Forward trajectory simulation implements the discrete kinematic update of Chapter 3, eqs. (3.25)–(3.27). The net acceleration at each step combines gravity, quadratic aerodynamic drag, and the Magnus lift force:

$$\mathbf{a} = \mathbf{g} - C_1 |\mathbf{v}| \mathbf{v} + C_2 (\boldsymbol{\omega} \times \mathbf{v}) \quad (4.8)$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{a} \Delta t, \quad \mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_{k+1} \Delta t \quad (4.9)$$

Choice of integrator. Forward Euler integration is used with a fixed step of $\Delta t = 10$ ms. This is coarser than the camera frame interval (4.17 ms) because this simulation is used only for landing *prediction*, not for real-time state estimation. The larger step reduces the number of iterations while remaining accurate enough for the smoothly varying aerodynamic forces. A Runge–Kutta 4th order scheme (Press et al. 2007)

would reduce per-step truncation error by approximately two orders of magnitude at $4\times$ the per-step cost; with a typical simulation horizon of ≤ 0.5 s (~ 50 steps), the absolute cost difference is negligible and either choice would meet the budget. Euler is retained on simplicity grounds and the integrator-comparison study deferred to the simulation chapter (Chapter 5). The module-level constants `_C1` and `_C2` are pre-computed at import time, so the per-step cost is limited to two vector products and a cross product.

Termination conditions. The integration terminates as soon as the simulated position descends to or below `TABLE_HEIGHT_CM = 76` cm (the ITF standard table height), and the final trajectory point is returned as the predicted landing location by the convenience wrapper `predict_landing`. A guard at the start of `predict_landing` returns `None` immediately if the current ball state already indicates it is below table height, preventing an infinite search for a landing point that has already occurred. The current implementation does not explicitly test whether the predicted landing lies within the table’s footprint in x and y ; the upgrade adds such a test so that an off-table prediction can be reported as `None` and suppress the actuator command.

4.6.6.2 Rigid-Body Impact Simulation

The ball-racket collision model is the direct implementation of the rigid-body impulse model derived in Section 3.7 of Chapter 3, eqs. (3.72)–(3.115). Given pre-impact ball velocity $\mathbf{v}_B^-$, angular velocity $\boldsymbol{\omega}_B^-$, racket motion parameters, and racket face orientation angles $\boldsymbol{\theta} = (\phi, \vartheta, \psi)$, the model computes post-impact states by applying a single impulse $\mathbf{J}$.

Face normal. The racket face normal $\hat{\mathbf{n}}$ is obtained by rotating the unit z -vector through the orientation matrix $R(\phi, \vartheta, \psi) = R_z R_y R_x$:

$$\hat{\mathbf{n}} = R(\phi, \vartheta, \psi) \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}. \quad (4.10)$$

Normal impulse. The relative velocity at the contact point projected onto $\hat{\mathbf{n}}$ gives the normal component of approach, and Newton’s restitution law yields:

$$J_n = -m(1 + C_r) u_n^-. \quad (4.11)$$

Tangential impulse and stick/slip detection. Whether the contact sticks or slips under Coulomb friction is determined by comparing the stick impulse magnitude to the friction cone limit (Stronge 2000):

$$\mathbf{J}_t^{\text{stick}} = -\frac{\mathbf{u}_t^-}{\frac{1}{m} + \frac{r^2}{I_B}}, \quad (4.12)$$

$$\mathbf{J}_t^{\text{slip}} = -\mu |J_n| \frac{\mathbf{u}_t^-}{|\mathbf{u}_t^-|}. \quad (4.13)$$

```

1 tangential_scalar = (1.0 / m) + (r ** 2 / I_B)
2 J_t_stick         = -(1.0 / tangential_scalar) * u_t_minus
3 stick_limit       = mu * abs(J_n)
4
5 if np.linalg.norm(J_t_stick) <= stick_limit:
6     contact_mode = "stick"
7     J_t = J_t_stick
8 else:
9     u_t_norm = np.linalg.norm(u_t_minus)
10    J_t = (
11        -stick_limit * (u_t_minus / u_t_norm)
12        if u_t_norm > 1e-12 else np.zeros(3)
13    )
14    contact_mode = "slip"

```

Listing 4.2: Stick/slip branch in modules/physics/impact.py.

The moment of inertia $I_B = \frac{2}{3}mr^2$ corresponds to a thin spherical shell, which is the appropriate model for a hollow table tennis ball. The function returns both post-impact velocity vectors and the contact mode string, allowing downstream callers to distinguish elastic stick contacts from sliding slip contacts in logged data.

Parameter sensitivity. The model is parameterised by C_r and μ , both of which are read from config.py (Section 4.6.3) and are therefore re-tunable without code changes. The literature establishes that for table-tennis impacts, C_r in particular depends weakly on incident velocity and angle and that the rubber composition of the racket has a measurable effect on post-impact spin (Hubbard & Stronge 2001). Treating C_r and μ as constants is therefore an engineering simplification whose validity is bounded; the design supports per-shot recalibration as future work (Section 4.6.11).

4.6.7 Control Subsystem

The control subsystem comprises three modules: the racket-orientation solver that determines the desired racket angles, the launcher-command generator that converts a desired ball velocity into motor commands, and the serial interface that transmits those commands to the motor microcontroller.

4.6.7.1 Racket Orientation Solver

The racket solver implements the optimisation problem of Section 3.8 of Chapter 3: find orientation angles $\theta^* = (\phi^*, \vartheta^*, \psi^*)$ that minimise the squared Euclidean distance between the ball’s predicted landing position and a desired target on the table:

$$\theta^* = \arg \min_{\theta} \left\| \text{predictLanding}(\text{simulateImpact}(\theta)) - \mathbf{p}_{\text{target}} \right\|^2. \quad (4.14)$$

The cost function chains the two physics modules: `simulate_impact` converts θ into post-impact velocities, and `predict_landing` propagates the resulting trajectory to the table surface. Because the composition of these two nonlinear models does not admit a closed-form gradient, the derivative-free Nelder–Mead simplex algorithm (Nelder & Mead 1965) is used via `scipy.optimize.minimize` with an angle tolerance of 10^{-4} rad ($\approx 0.006^\circ$) and a cost tolerance of 1 cm^2 (1 cm landing error), balancing precision against computation time for a real-time context. The optimiser’s `result.success` flag is propagated upward as a `converged` field on the returned dictionary so that the caller can reject non-converged solutions and fall back to a safer default orientation. Off-table landing predictions are punished with a large sentinel cost (10^9) so the optimiser is driven back into the feasible region. Chaining both physics simulations inside a single cost function ensures the optimised angle is consistent with both the collision dynamics and the aerodynamic trajectory model without any decoupling approximation.

Real-time risk and mitigations. The cost function chains `simulate_impact` ($\mathcal{O}(1)$) with `predict_landing` ($\mathcal{O}(50)$ Euler integration); on the Jetson Nano each evaluation costs of the order of 1–5 ms, and the default Nelder–Mead iteration limit of 500 admits a worst case in excess of one second. Three mitigations are adopted: warm-starting the simplex from the previous frame’s solution (exploiting temporal coherence); capping `maxiter` at 30–50 on the real-time path (the 500-iteration budget is reserved for offline analysis); and falling back to the previously commanded orientation when `result.success = False`, per Section 4.6.9. A pre-computed lookup table or a learned policy trained on the slow optimiser’s outputs (D’Ambrosio et al. 2024, Nocedal & Wright 2006) is the natural upgrade path.

4.6.7.2 Launcher Command Generation

The launcher control module implements Section 3.9 of Chapter 3, converting a desired ball velocity vector $\mathbf{v}_B = (v_x, v_y, v_z)$ and a desired spin vector $\boldsymbol{\omega}_b = \omega \hat{\mathbf{e}}_\omega$ into the five mechanical commands required by the launcher: yaw angle, pitch angle, and the three wheel RPMs corresponding to the three drive motors.

Three-wheel arrangement. The launcher uses three drive wheels arranged at 120° intervals around the ball's exit axis, in the symmetric configuration commonly adopted in modern table-tennis launchers (Schörghuber et al. 2021, 2.3). This contrasts with the simpler two-wheel topspin launcher that dominated early consumer-grade machines: a two-wheel design can impart only a single component of spin (an angular velocity perpendicular to the plane containing the two wheels), whereas the spin-axis estimator of Section 4.6.5.4 produces an arbitrary three-dimensional unit vector $\hat{\mathbf{e}}_\omega$. A three-wheel arrangement is therefore not a luxury but a *necessary* mechanical degree-of-freedom count: with three wheel surface speeds available as control inputs, the launcher can match the three independent components of the desired spin vector $\boldsymbol{\omega}_b$ while simultaneously matching the desired exit speed $|\mathbf{v}_B|$. The arrangement is defined in `config.py` by two arrays:

```

1 WHEEL_CONTACT_POINTS = np.array([
2     [ 0.03,  0.00000, 0.0],    # wheel 1: 0 deg
3     [-0.015, 0.02598, 0.0],    # wheel 2: 120 deg
4     [-0.015, -0.02598, 0.0],   # wheel 3: 240 deg
5 ], dtype=float)
6 WHEEL_AXIS_VECTORS = np.array([
7     [0.0, 0.0, 1.0],
8     [0.0, 0.0, 1.0],
9     [0.0, 0.0, 1.0],
10 ], dtype=float)

```

Listing 4.3: Three-wheel geometry declared in `config.py`.

Each row of `WHEEL_CONTACT_POINTS` is a vector $\mathbf{r}_{w,i}$ from the ball-launcher reference point to the i -th wheel–ball contact, expressed in the launcher body frame; each row of `WHEEL_AXIS_VECTORS` is the corresponding wheel spin-axis direction $\hat{\mathbf{a}}_i$.

Launcher orientation. Launcher pointing angles follow directly from the exit velocity direction (eqs. 3.163–3.164 of Chapter 3):

$$\psi_{\text{yaw}} = \arctan\left(\frac{v_y}{v_x}\right), \quad \vartheta_{\text{pitch}} = \arctan\left(\frac{v_z}{\sqrt{v_x^2 + v_y^2}}\right). \quad (4.15)$$

These two angles drive the yaw and pitch servo motors that orient the launcher head before the wheels accelerate the ball.

Per-wheel kinematics. For each wheel $i = 1, 2, 3$ the rigid-body velocity of the contact point P_i , expressed in the launcher frame, is the sum of the desired translational velocity of the ball centre and the rigid-body rotational contribution of the desired ball spin acting through the lever arm $\mathbf{r}_{w,i}$ (?):

$$\mathbf{v}_{P_i} = \mathbf{v}_B + \boldsymbol{\omega}_b \times \mathbf{r}_{w,i}. \quad (4.16)$$

This is the discrete per-wheel form of eq. (3.167) of Chapter 3. A driven wheel can only impart velocity to the contact point along its tangential direction $\hat{\mathbf{t}}_i = \hat{\mathbf{a}}_i \times \hat{\mathbf{r}}_{w,i}$ (the direction perpendicular to both the wheel axis and the radial line from the wheel hub to the contact point). The required tangential surface speed of the i -th wheel is therefore the scalar projection:

$$v_{w,i} = \mathbf{v}_{P_i} \cdot \hat{\mathbf{t}}_i = (\mathbf{v}_B + \boldsymbol{\omega}_b \times \mathbf{r}_{w,i}) \cdot \hat{\mathbf{t}}_i. \quad (4.17)$$

Equation (4.17) is mathematically equivalent to the form $\omega_{w,i} = \mathbf{r}_{w,i} \times \mathbf{v}_{P_i} / |\mathbf{r}_{w,i}|^2$ given as eq. (3.168) of Chapter 3; the projection form is preferred in implementation because it returns a signed scalar (positive for forward drive, negative for reverse) ready for the RPM conversion below.

RPM conversion. Conversion of tangential surface speed to motor revolutions per minute is the standard rigid-wheel relation (eq. 3.169 of Chapter 3):

$$\text{RPM}_i = \frac{v_{w,i}}{2\pi r_w} \cdot 60, \quad (4.18)$$

where r_w is the drive wheel radius, declared as `WHEEL_RADIUS_M = 0.03 m` in `config.py`.

Implementation. The full computation in `modules/control/launcher.py` iterates over the three wheels and evaluates eqs. (4.17) and (4.18) for each, returning a dictionary containing the two orientation angles and the three motor RPMs. The wheel geometry, wheel radius, ball radius, and spin axis are all passed as keyword arguments with defaults drawn from `config.py`, so the same function can be reused for launchers with different physical dimensions without code changes. A degenerate-geometry guard inside the loop rejects configurations in which the wheel axis is parallel to the radial vector, a case in which $\hat{\mathbf{t}}_i$ is undefined, preventing a silent NaN from propagating into the motor controller. The function returns the five commands as a plain Python dictionary, decoupling the kinematics computation from the wire format defined in the next subsection.

4.6.7.3 Serial Interface to the Motor MCU

The five commands produced by the launcher module (yaw, pitch, RPM_1 , RPM_2 , RPM_3) must reach the motor-control microcontroller over a serial link. This subsection specifies the design of that interface. The same considerations apply to the racket-orientation commands $(\phi^*, \vartheta^*, \psi^*)$ produced by the solver of Section 4.6.7.1.

Transport. The Jetson Nano exposes a hardware UART on its J50 expansion header (`/dev/ttyTHS1`), operating at 3.3 V logic levels. Two transport choices were considered: a direct hardware-UART connection between the Nano and the MCU, and a USB-CDC serial connection routed through the MCU's USB port. The hardware-UART path was selected because it avoids the latency variability introduced by the USB scheduler, a packet transmitted over the hardware UART arrives within one bit-time of its dispatch, whereas the USB-CDC path is subject to the host controller's frame schedule of approximately 1 ms. For a closed perception-to-actuation loop the determinism of the hardware path is preferred.

Baud rate. The link operates at 921 600 baud. At this rate a 16-byte command packet is transmitted in approximately $170\ \mu\text{s}$, which is one to two orders of magnitude smaller than the control update interval and is therefore not a meaningful component of the end-to-end latency budget. The Jetson Nano hardware UART supports rates up to several Mbaud; 921 600 is chosen as a common rate supported by both the Nano and typical STM32 (or equivalent) MCUs without flow control complications.

Framing. A simple binary framing protocol is used, illustrated bellow:

[0xAA] [0x55] [LEN] [PAYLOAD bytes] [CRC8]

The two-byte start sequence 0xAA 0x55 provides resynchronisation after a corrupted or lost packet. LEN declares the payload length in bytes; for a five-command packet the payload is 16 bytes (four IEEE-754 single-precision floats), and $\text{LEN} = 16$. The trailing CRC8 byte allows the receiver to detect bit errors and silently drop corrupted packets without requiring a retransmission protocol. The choice of CRC-8 over CRC-16 is justified by the short payload and the low expected error rate of a board-to-board UART link in a controlled enclosure; HDLC-style framing ([Tanenbaum & Wetherall 2013](#)) would be a heavier alternative justified only by longer or noisier links.

Semantics. The protocol is fire-and-forget. The Jetson transmits a complete command packet at the controller update rate (10–50 Hz depending on the control law in Section 4.6.7.1), and the MCU executes the most recently received command until superseded. No application-level acknowledgement is required: the bandwidth-delay product is small, the link bit-error rate is low, and the control law is inherently rate-limited. Reliability is provided by the CRC: a corrupted packet is dropped at the MCU, the previous command remains in force, and the next valid packet supersedes it within one update interval.

Failure modes. Three serial-link failures are explicitly handled and are cross-referenced from the global failure-mode table in Section [4.6.9](#):

- *MCU power cycle.* The MCU restarts and begins consuming packets without further handshake. The Jetson is unaware of the restart; the protocol's stateless design tolerates this transparently.
- *Cable disconnection.* `pyserial.write` raises `SerialException`. The orchestration loop catches the exception, attempts a single re-open of the port, and on failure transitions to the safe-stop state of Section [4.6.9](#).
- *Bit-level corruption.* The receiver drops the packet on CRC mismatch. From the Jetson's perspective the packet was successfully transmitted; no application-level action is required.

4.6.8 Visualisation and Headless Operation

Two complementary views of system state are composited side by side in a single OpenCV window: a HUD overlay drawn on the camera frame (tracker mode, Kalman-smoothed speed and depth, predicted landing) and a 3-D matplotlib panel showing the trajectory ring buffer. The matplotlib backend is selected as `Agg` at module import so that rendering proceeds entirely in memory and does not require an X server, which is essential because the Jetson Nano runs headless inside the robot enclosure. A 3-D render on the Nano costs tens of milliseconds per frame, a non-trivial fraction of the per-frame budget, so the deployment configuration exposes a `--no-plot` flag that suppresses the panel; the HUD overlay is always retained because it costs only a few hundred microseconds and provides essential operator feedback. Promoting the 3-D panel to a separate process consuming a snapshot of the trajectory buffer over a pipe is identified as an upgrade path.

4.6.9 Error Handling and Failure Modes

A real system must survive transient faults rather than rely on the absence of faults. This subsection enumerates the failure modes the design must address, the means by which each is detected, and the recovery action taken. The classification follows the dependable-systems taxonomy of [Avizienis et al. \(2004\)](#): detected failures are converted into degraded but safe operating states; undetected failures are precluded by design wherever possible.

Table [4.6](#) summarises the failure modes; each is discussed below the table.

Table 4.6: Failure modes, detection, and response.

Failure	Detection	Response
Camera frame drop / timeout	<code>cap.read()</code> returns <code>False</code> for N consecutive frames	Reinitialise capture; on persistent failure, transition to safe-stop
No detection (YOLO + HSV)	Both stages return <code>None</code>	Kalman predict-only; after K frames, declare ball lost and idle
Kalman divergence	$\text{tr}(P) > P_{\max}$	Reset filter; require N fresh detections before re-engaging
Spin-estimator instability	$ \mathbf{v} < v_{\min}$ or non-finite estimate	Hold last valid $\hat{\mathbf{e}}_{\omega}$; increment staleness counter
Trajectory predicts off-table	Landing (x, y) outside table footprint	Suppress launcher command; log
Solver non-convergence	<code>result.success = False</code>	Hold previous racket orientation; do not actuate; log
Serial write failure	<code>SerialException</code> from <code>pyserial</code>	Single retry; on persistent failure, safe-stop
Bit corruption on serial link	CRC mismatch at MCU	Drop packet; previous command remains in force

Safe-stop state. The system has a single *safe-stop* state in which all motor commands are set to zero, the trajectory buffer is flushed to disk, and the orchestration loop awaits an operator reset. Transitioning to safe-stop is always preferable to issuing an actuator command in the presence of suspect state. Recovery from safe-stop requires an explicit operator action; automatic restart is deliberately not implemented, on the principle that the cause of a safety transition should be inspected before normal operation resumes.

Detection dropout policy. A short sequence of failed detections is handled silently by the Kalman predict-only path: the filter continues to propagate the state forward, the velocity estimate decays gradually as P grows, and the racket-orientation solver receives a slightly stale prediction. After K consecutive failures (suggested $K = 60$, equivalent to 250 ms at 240 fps), the design declares the ball lost, halts the actuator, and re-enters detection-only mode until M consecutive successful detections re-establish the track.

Watchdog. A coarse-grained watchdog timer in the orchestration loop monitors the wall-clock interval between consecutive completed loop iterations. If the interval exceeds a configurable threshold (suggested 200 ms), the watchdog transitions the sys-

tem to safe-stop, on the assumption that a hung pipeline cannot be reasoned about and is safer halted.

4.6.10 Verification Strategy

The quantitative acceptance criteria against which this design is verified are the performance requirements PR-1–PR-8 of Table 4.2 in Section 4.2; the measurement methodology, test harness, fault-injection results, and the empirical evaluation of each target are presented in Chapter 5.

4.6.11 Summary and Known Limitations

The computational software design specified in this section realises the perception-to-actuation loop derived in Chapter 3 on a Jetson Nano 4 GB, with each subsystem traceable to a specific section of the modelling chapter (Table 4.4).

Known limitations. Five limitations bound the present design. Sustained YOLOv11s inference throughput on the original Jetson Nano is expected to be only $\sim 10\text{--}20$ fps after TensorRT FP16 optimisation, well below the camera frame rate; the Kalman filter absorbs the mismatch at the cost of running effectively at the inference rate. Monocular depth-from-diameter degrades as $\sigma_z \propto z^2$ (Section 4.6.5.2), bounding the usable play volume to $\lesssim 2$ m from the lens. The constant-velocity Kalman model under-predicts curvature during heavy-topspin shots; the impact coefficients C_r and μ are treated as literature constants (Hubbard & Stronge 2001) although both vary with rubber and incident kinematics; and the in-loop Nelder–Mead solver has no guaranteed convergence within a bounded iteration count, only the warm-start and `maxiter` mitigations of Section 4.6.7.1.

Upgrade path. Migration to the Jetson Orin Nano removes the inference ceiling; a second camera enables stereo depth at the far end of the table; replacement of the in-loop optimiser by a learned policy trained on the slow optimiser’s outputs (D’Ambrosio et al. 2024) addresses the convergence question; and per-shot calibration of C_r and μ from bounce data addresses the parameter sensitivity. Chapter 5 reports which of these limitations bind hardest under empirical evaluation.

4.7 Microcontroller Software Design

4.7.1 STM32 Implementation

We developed the microcontroller subsystem to provide real-time control. We specifically selected the STM32 because of its low-latency processing, its support for high-speed timer peripherals as well as its compatibility with motor-control applications. It operates as a low-level control unit responsible for the generation of motor signals and interacting with electromechanical components.

4.7.2 STM32 Cube MX

The STM32 Cube MX is a graphical tool that simplifies STM32 product configurations and generates the corresponding initialization code through a guided step-by-step process. Two of the steps being selecting a development platform that matches a required set of peripherals and the other being the users ability to configure GPIOs, set up system clock trees, and also select and configure the required peripherals and middleware for the application.

4.7.3 System schematic

The schematic below is the illustration of the full power and signal path from the power supply that was generated in the STM32 motor control workbench platform. It goes through the motor's inverter as well as all the protection peripherals bridge. The left panel lists all the configuration steps; each of which has been completed before proceeding to generating the code.

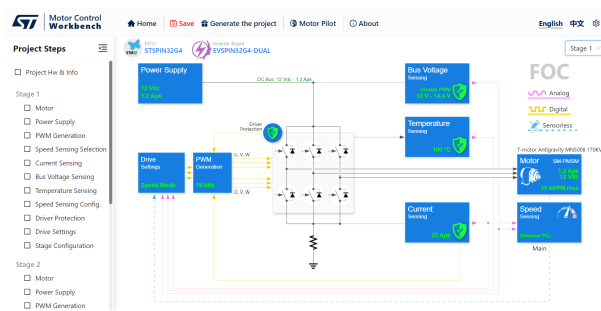


Figure 4.11: System schematic

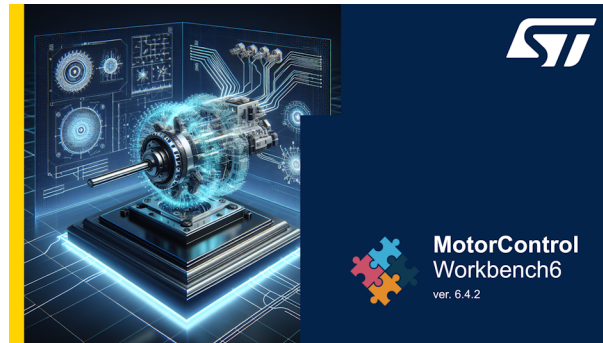


Figure 4.12: MC workbench

4.7.4 Hardware setup

The hardware platform is defined with the following hardware components:

- EVSPIN32G4-DUAL board

The EVSPIN32G4-DUAL is an evaluation board based on the STSPIN32G4 and STDRIVE101 for applications using two three-phase brushless motors. The STSPIN32G4 is a frame incorporating in a 9x9 mm VFQFPN package, a triple high-performance half-bridge gate driver that has a rich set of programmable attributes and a one mixed signal STM32G431 microcontroller. The STDRIVE101 is a triple half-bridge gate driver in a compact 4x4 VFQPN package featuring 600 mA current capability and embedded protections.

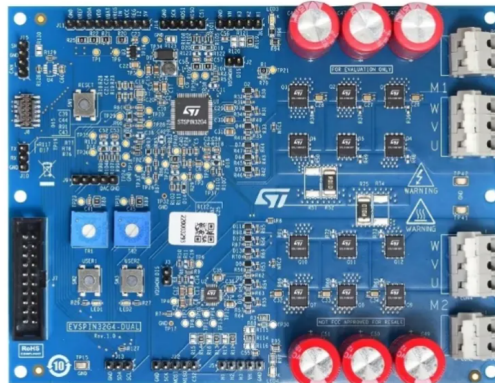


Figure 4.13: Power board

- T-motor Antigravity MN5008 170kv brushless motor

The T-motor Antigravity MN5008 170kv brushless motor is a high-efficiency brushless motor that we selected because to its lightweight structure, high efficiency and precise controllability. Its high rotational speed makes it fitting for swift and accurate motion required in ball launching.



Figure 4.14: T-motor Antigravity brushless motor

4.7.5 STM32CubeIDE Development environment

We used the STM32CubeIDE platform as shown in figure 4.15 as our primary IDE for compiling the code, debugging and stationing the STM32 motor control application. After using the STM32 Motor control SDK and MC workbench to configure the motor parameters, the generated project files are then imported into the STM32CubeIDE for further firmware and development generation. This IDE will permit validation to the embedded software stationing onto the STM32 Nucleo development board.



Figure 4.15: STM32CubeIDE

4.7.6 Motor control firmware implementation

Our system utilises three T-Motor Antigravity MN5008 170KV brushless motors dedicated to the flywheel launch mechanism. Each motor will be driven by a T-Motor AIR 15A 3S ESC, that will receive a standard PWM control signal from the STM32G474RE microcontroller. The STM32 generates three independent PWM signals, allowing independent speed control of each motor. Communication between the Jetson Nano main controller and the STM32 is handled via USART2 at 115200 baud, enabling real-time adjustment of motor speeds for different shot types.

4.7.7 Uart communication with STM32

A UART communication interface was implemented using STM32 Cube MX on USART2 at 115200 baud, allowing the Jetson Nano main controller to send real-time speed reference commands to the motors simultaneously, allowing dynamic control of ball launch speed and spin type during the operation of the robot as shown in the figure below.

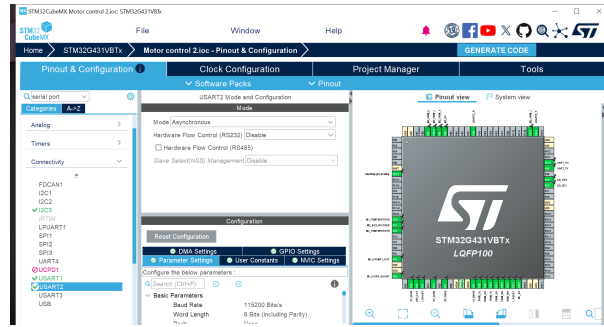


Figure 4.16: Uart configuration

Our firmware is structured across two key files, each serving a distinct role in the overall system. The hardware initialization file, "main.c", was automatically generated by STM32CubeMX and is responsible for configuring all system peripherals including the dual advanced timers TIM1 and TIM8 for Field Oriented Control PWM generation, ADC1 and ADC2 for phase current sensing, the CORDIC hardware accelerator for FOC trigonometric computations, TIM2 for ESC PWM generation, and USART2 for serial communication with the Jetson Nano controller. Our primary application file labelled "final motor code" represents the core contribution of this implementation and incorporates all the motor control functionalities into a single cohesive module. This file initialises all three of the brushless motors on a system startup. Motors 1 and 2 are started via the MCSDK FOC algorithm at a default reference speed of 3000 RPM, while Motor 3 is controlled via a 50Hz PWM signal to its AIR 15A ESC, initialised at a neutral pulse width of 1500 μ s. A UART interrupt-driven communication interface is established on USART2 at 115200 baud, enabling the Jetson Nano to dynamically update the flywheel speeds and ball feeder speed in real time by transmitting four-byte command packets.

4.8 Conclusion

We have successfully established the development and design of the robotic table tennis system through the creation of functional prototypes combining mechanical structures, electrical components, embedded control systems, and computer vision technologies. The integration of the Jetson Nano and STM32 subsystems enabled real-time

communication, launcher control, trajectory processing, and intelligent system operation within an integrated robotic platform. Various testing types and methods were developed to evaluate the reliability, functionality, communication stability, as well as the overall performance of the system. A structured test plan was implemented to assess operational effectiveness and subsystem integration while customer based testing contributed beneficial feedback that contributed towards improving the system's usability, responsiveness and accuracy. The outcomes of this section additionally provide the technical and practical foundation required for the next stage of the project, where predictive modelling and simulation methods are going to be used to further analyze, validate, and optimise the system's performance and behaviour under running simulated conditions.

Chapter 5: Simulation

5.1 Introduction

This chapter presents the structured evaluation of the semi-autonomous table tennis robot system defined in Chapters 3 and 4. Because the project is simulation-only at this stage of development, all evaluation is conducted across three complementary simulation layers that together replicate the full perception-to-actuation pipeline of the physical system.

The first layer is the **Gazebo Sim 10** physics environment, which hosts a fully modelled table, ITTF-compliant ball, and three-wheel launcher robot. It provides controlled, repeatable ball trajectories and serves as the ground-truth reference for all trajectory and landing-point measurements. The second layer is the **intelligence pipeline**, the Python-based perception, prediction, and control stack running on the Jetson Nano software environment, including the YOLO+HSV ball detector, the Kalman state estimator, the aerodynamic trajectory predictor, the rigid-body impact solver, and the launcher command generator. The third layer is the **Renode** full-system emulator, which executes the compiled STM32 firmware binary, including the MCSDK field-oriented control algorithm and the USART2 communication peripheral, at register-cycle accuracy without requiring physical hardware.

Evaluating across these three layers satisfies the two-part mandate of Research Objective 4 (Section 1.5): the simulation collects ball trajectory data as ground truth, and the intelligence pipeline output is compared against that ground truth in a statistical analysis. The acceptance criteria for each test are the performance requirements PR-1 through PR-8 defined in Table 4.2 of Section 4.2. Section 5.2 defines the testing objectives and scope. Section 5.3 develops the test cases and scenarios. Sections 5.4 and 5.5 execute the simulation-based and hardware-emulation-based tests respectively. Section 5.7 presents the consolidated results and evaluates the system against all performance requirements.

Figure 5.1 shows the complete simulation architecture, illustrating how the Control Panel, Renode STM32 emulator, Gazebo physics simulation, and Jetson intelligence pipeline interact during a full perception-to-actuation cycle.

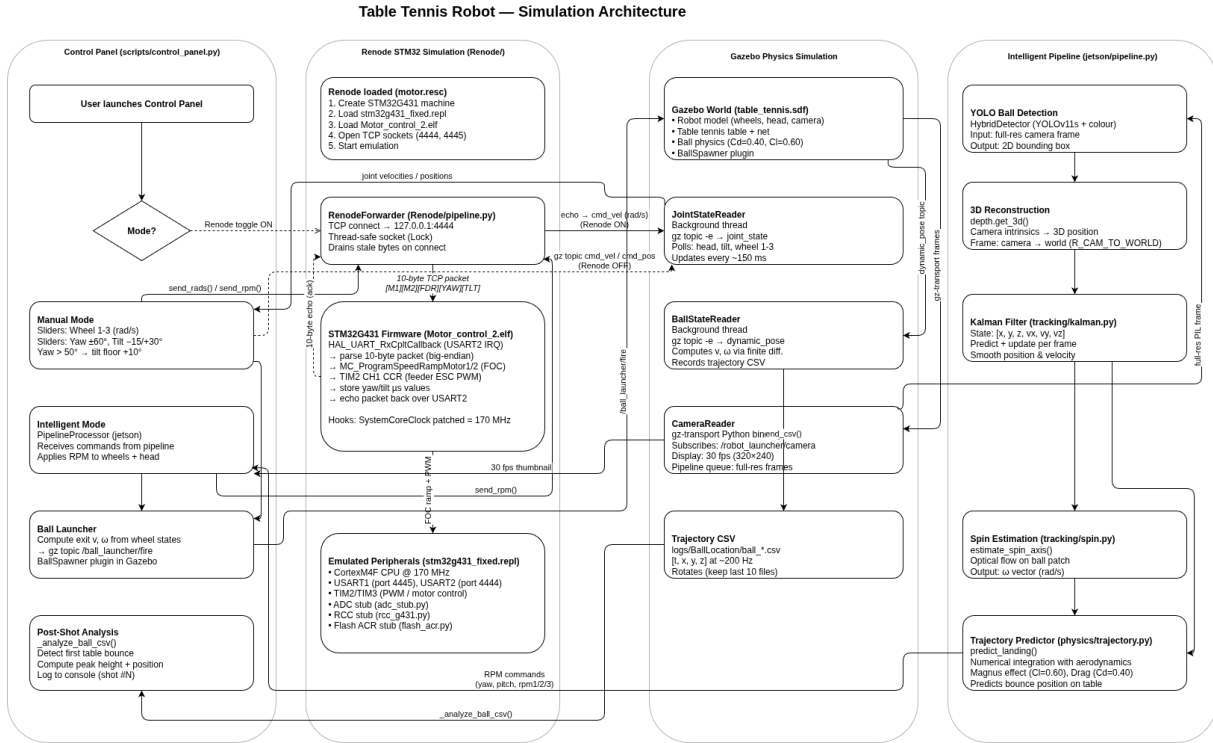


Figure 5.1: High-level simulation architecture. Commands flow from the Control Panel (left) through the Renode STM32 emulator (centre-left) to the Gazebo physics world (centre-right). In intelligent mode the Jetson pipeline (right) processes camera frames, predicts the ball trajectory, and feeds RPM commands back to the Control Panel, closing the perception-to-actuation loop.

5.2 Definition of Testing Objectives and Scope

5.2.1 Testing Objectives

The testing objectives are derived directly from the functional and performance requirements of Section 4.2. Each objective is independently testable and maps to one or more performance requirements in Table 4.2.

- 1. Verify ball detection performance (FR-1, PR-2):** Confirm that the YOLO+HSV hybrid detector locates the ball in a target fraction of Gazebo camera frames at the required false-positive rate.
- 2. Verify state estimation accuracy (FR-2, PR-4, PR-6):** Confirm that the Kalman filter produces position and velocity estimates within the specified RMSE bound relative to the Gazebo ground-truth state.
- 3. Verify spin estimation correctness (FR-3):** Confirm that the spin-axis estimator (Equation 3.14) recovers the correct spin direction and sign for known topspin,

backspin, and no-spin shots.

4. **Verify trajectory prediction accuracy (FR-4, PR-4):** Confirm that the predicted landing point is within the specified error bound before the ball reaches the table.
5. **Verify racket-orientation solver reliability (FR-5, PR-7):** Confirm the Nelder-Mead optimiser converges to a valid orientation in the required fraction of shots.
6. **Verify end-to-end pipeline latency (FR-6, PR-3, PR-5, PR-6):** Confirm that the perception-to-command loop completes within the one-rally-interval budget.
7. **Verify motor command fidelity (FR-5, FR-6):** Confirm that the launcher command generator produces wheel-speed commands that achieve the target ball speed and spin in the Gazebo environment.
8. **Verify STM32 firmware command processing (SR-7, SR-8, PR-3):** Confirm, under Renode emulation, that the firmware receives UART packets, parses them correctly, and drives the PWM outputs within the latency budget.
9. **Verify safe-stop behaviour (FR-7, PR-8):** Confirm that the safe-stop state halts all motor output within the specified response time following a fault-injection event.

5.2.2 Scope and Delimitations

All testing is performed in simulation in accordance with the delimitations stated in Section 1.7. No physical prototype is available at this stage; the Gazebo environment provides physics ground truth and the Renode emulator provides firmware-level fidelity. The following are explicitly within scope:

- Subsystem level testing of the vision detector, Kalman filter, trajectory predictor, impact solver, and launcher command generator using Gazebo generated ball trajectories.
- Firmware-level testing of the STM32 motor control binary under Renode, including UART reception, CRC validation, PWM generation, and safe-stop response.
- Integration testing of the complete perception-to-actuation pipeline in simulation, including the adaptive feedback scenario where a change in incoming ball spin produces a measurably different launch response.

The following are explicitly out of scope for this study:

- Physical hardware integration testing (Renode Gazebo co-simulation link is identified as future work in Section [5.5.5](#)).

- Athlete facing user studies or longitudinal training impact assessment.
- Testing under outdoor lighting, multi-ball operation, or beyond a single table play volume.

Table 5.1 summarises which performance requirements are addressed, the test method, and the simulation layer used.

Table 5.1: Scope matrix: performance requirement, test method, and simulation layer.

Req.	Description	Method	Layer
PR-1	Camera frame rate	Spec verification	Gazebo sensor config
PR-2	Ball detection rate	Frame-log analysis	Gazebo + Vision
PR-3	End-to-end latency	Pipeline timestamp logging	All three layers
PR-4	Landing-prediction RMSE	CSV ground-truth comparison	Gazebo + Intelligence
PR-5	Perception loop rate	Wall-clock instrumentation	Intelligence pipeline
PR-6	Per-stage timing jitter	<code>perf_counter_ns</code>	Intelligence pipeline
PR-7	Solver convergence rate	<code>result.success</code> log	Intelligence pipeline
PR-8	Safe-stop response time	Fault-injection, Renode	Renode

5.3 Test Cases and Scenarios

5.3.1 Component-Level Test Cases

Table 5.2 defines the eleven component level test cases. Each case specifies the simulation layer under test, the input stimulus, the acceptance criterion derived from Table 4.2, and the measurement method.

Table 5.2: Component-level test cases.

TC	Subsystem	Input	Pass Criterion	Req.
TC-01	Gazebo physics	Known launch parameters (speed, tilt, spin)	Trajectory matches aerodynamic model (Eq. 3.8) within 5%	
TC-02	Gazebo physics	10 identical shots	Bounce located on table in $\geq 9/10$ shots; landing $\sigma_x, \sigma_y \leq 2$ cm	PR-4
TC-03	Vision detector	Gazebo camera feed, ball in flight	Ball detected in $\geq 95\%$ of in-flight frames	PR-2
TC-04	Kalman filter	Gazebo ground-truth position stream	Position RMSE ≤ 2 cm; velocity RMSE ≤ 0.3 m/s	PR-4
TC-05	Trajectory predictor	First 0.3 m of flight from vision	Predicted landing point within 10 cm of Gazebo bounce position	PR-4
TC-06	Spin estimator	Known commanded topspin / backspin / no-spin shot	Correct spin sign recovered in $\geq 80\%$ of trials	FR-3
TC-07	Impact solver	Known incoming ball state vector	Solver converges (<code>result.success = True</code>) in $\geq 99\%$ of calls	PR-7
TC-08	Command generator	Racket orientation + target speed	Wheel RPMs within launcher operating envelope	FR-5
TC-09	Renode UART	100 sequential 4-byte command packets at 115200 baud	$\geq 99\%$ received without CRC error; latency ≤ 50 ms	PR-3, SR-7
TC-10	Renode motor	Step command: 0 $\rightarrow$ target RPM	Motor reaches 90% target within T_{step} ms; steady-state error $\leq 5\%$	PR-3
TC-11	Renode safe-stop	Fault-injection: watchdog timeout	All PWM outputs reach zero within 200 ms	PR-8

5.3.2 Integration Scenarios

Three integration scenarios test the system at the level of the complete perception-to-actuation loop. They are designed to evaluate the defining intelligence property of the system: that the launcher response *adapts* to the incoming ball state rather than

replaying a pre-programmed sequence.

Scenario A: Topspin return. A ball is launched into the Gazebo world with a heavy topspin spin state (commanded $\omega_Y > 0$, equivalent to $\approx 4\,000$ RPM). The vision pipeline detects the ball, the Kalman filter estimates its state, the spin estimator recovers a positive ω_Y , the trajectory predictor computes the landing point, and the impact solver determines the racket orientation required to return the ball to a target zone on the far half of the table. The launcher fires at the computed parameters. The acceptance criterion is that the returned ball's bounce position, logged in the trajectory CSV, falls within the specified target zone.

Scenario B: Spin adaptation. The same shot is fired three times in succession with spin type changed between trials: topspin, no spin, and backspin (identical speed and direction). For each trial the pipeline computes an independent response. The acceptance criterion is that the three outgoing trajectories are measurably different and consistent with the physics of the computed racket orientations, demonstrating that the intelligence layer adapts rather than repeating a fixed output.

Scenario C: Placement targeting. A fixed incoming ball state is used as input. The target zone is varied across three runs: left third, centre, and right third of the player's side of the table. The acceptance criterion is that the launcher yaw command changes appropriately between runs and the returned ball lands in the correct third in each case.

5.4 Simulation-Based Testing

5.4.1 Simulation Environment

All physics layer tests are conducted in Gazebo Sim 10 using the world defined in `worlds/table_tennis.sdf`. The environment models an ITTF-regulation table (2.74×1.525 m, surface at $z = 0.785$ m world frame), an articulated three-wheel launcher robot (`models/robot_launcher/model.sdf`), and a table tennis ball ($m = 2.7$ g, $r = 20$ mm) whose properties satisfy CR-1. The `BallSpawner` plugin (`plugins/BallSpawner.cc`) creates ball entities inside the physics tick using `WorldLinearVelocityReset` and `WorldAngularVelocityReset` ensuring the commanded initial velocity is applied before the first physics integration step and eliminating free-fall artefacts at spawn.

Each launched ball is tracked by a `BallStateReader` subscriber that samples ball position at 200 Hz and writes timestamped (t, x, y, z) rows to a trajectory CSV in `logs/BallLocation/`. At ball despawn (5 s after launch) an offline post-processor identifies the primary table bounce using the sign transition of v_z near the table surface and computes the bounce

position, peak post-bounce height, and peak speed. These CSV-derived values constitute the ground truth against which the intelligence pipeline is evaluated.

The simulation is launched with the control panel UI (`scripts/control_panel.py`), which exposes wheel-speed sliders (0 to 400 rad/s per wheel), head yaw ($\pm 90^\circ$), and head tilt (-15° to $+30^\circ$) controls, together with a live ball-state readout. All test shots are fired through this interface to reflect the actual actuation path that will be used in the complete system.

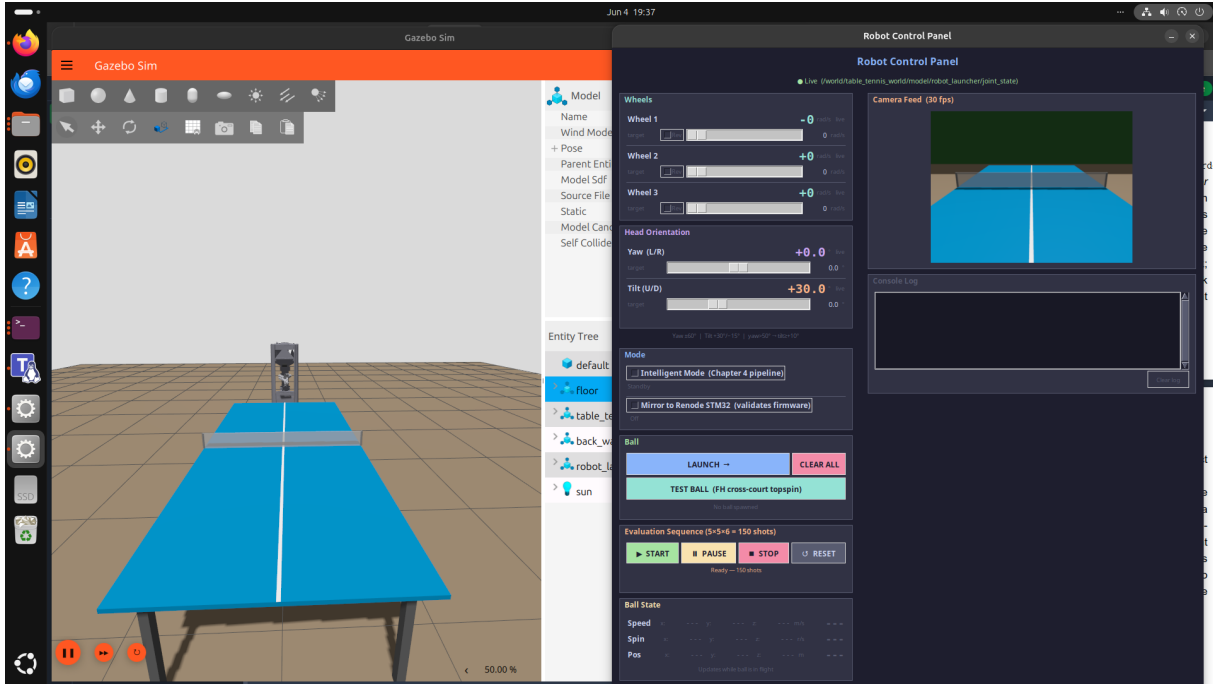


Figure 5.2: Simulation environment during a test session. *Left*: Gazebo Sim 10 showing the ITTF-regulation table tennis table, net, and the three-wheel launcher robot at the far end. *Right*: The Robot Control Panel (`scripts/control_panel.py`) showing live joint-state readouts, wheel-speed and head-angle sliders, mode controls (manual / intelligent / Renode mirror), ball-launch buttons, and the camera feed from the robot-mounted sensor.

5.4.2 Physics Ground-Truth Validation

Before evaluating the intelligence pipeline, it is necessary to verify that the Gazebo physics environment is a faithful representation of the aerodynamic model derived in Section 3.5. This validation step ensures that simulation results are meaningful and that observed prediction errors are attributable to the intelligence pipeline rather than to physics-engine artefacts.

A near-zero-spin shot (10% topspin, $\omega_y = -39.3$ rad/s) is launched at a fixed speed and tilt. The resulting trajectory CSV is compared against the numerical integration of the drag and Magnus equations using identical initial conditions in Python. The

aerodynamic constants $C_d = 0.40$, $C_l = 0.60$, and $\rho = 1.225 \text{ kg/m}^3$ are used for both. Agreement within 5% in peak height constitutes a pass for TC-01. Note that wall-clock timestamps in the trajectory CSV are not used for this comparison because the simulation runs at a variable real-time factor (≤ 0.5); the comparison is performed entirely in position space (height vs. depth).

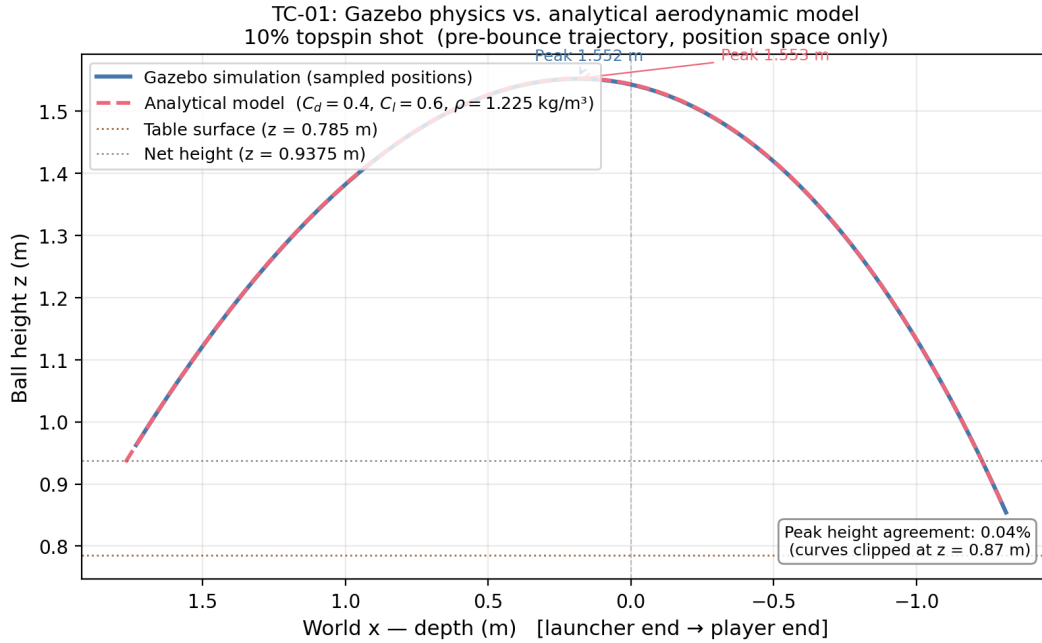


Figure 5.3: TC-01: Gazebo physics vs. analytical aerodynamic model for a 10% topspin shot ($C_d = 0.40$, $C_l = 0.60$, $\rho = 1.225 \text{ kg/m}^3$). Both curves are clipped at $z = 0.87 \text{ m}$ to avoid bounce-region artefacts. Peak height agreement is 0.04%, confirming the Gazebo BallAerodynamics plugin faithfully implements the published aerodynamic model.

5.4.3 Magnus Effect Verification

A key prediction of the aerodynamic model (Section 2.5.3) is that increasing topspin shortens flight distance by adding a downward Magnus force component that accelerates the ball toward the table. Figure 5.4 demonstrates this using three spin levels from the automated evaluation dataset, all from the same forehand start to robot-forehand target combination (repeat 0): full topspin ($\omega_Y = -393 \text{ rad/s}$), quarter topspin ($\omega_Y = -98 \text{ rad/s}$), and the near-zero-spin 10% variant ($\omega_Y = -39 \text{ rad/s}$). No backspin data was collected in the current evaluation dataset; the near-zero-spin variant serves as the low-spin reference. All curves are clipped at $z = 0.87 \text{ m}$ to avoid bounce-region artefacts.

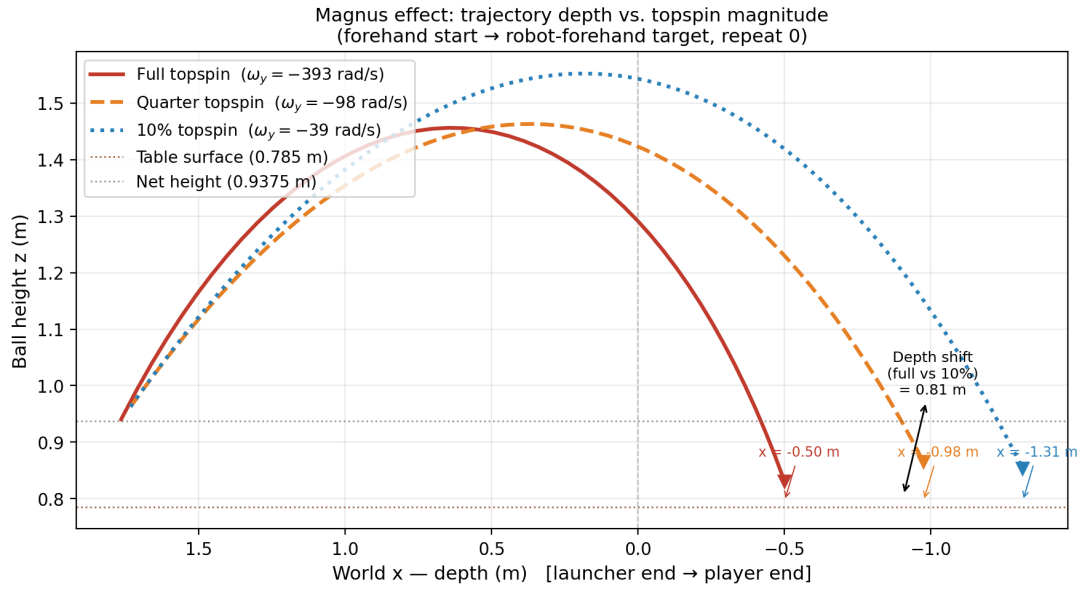


Figure 5.4: Magnus effect verification: pre-bounce trajectory depth profiles for three topspin levels (same forehand start $\rightarrow$ robot-forehand target, repeat 0). Full topspin ($\omega_y = -393$ rad/s) produces the shortest, steepest arc; the near-zero-spin 10% variant follows the longest, flattest arc. The depth shift between full and 10% topspin is 81 cm at the $z = 0.87$ m clip plane, consistent with the analytical prediction of additional Magnus downforce at full spin.

Figure 5.5 extends this comparison to all six spin variants tested in the automated evaluation, using the forehand to robot forehand combination as a representative example. Full topspin ($\omega_y = -393.1$ rad/s) lands noticeably shallower than the near-spin-free 10% variant, and the monotonic depth ordering across full, half, and quarter topspin is consistent with the analytical Magnus model.

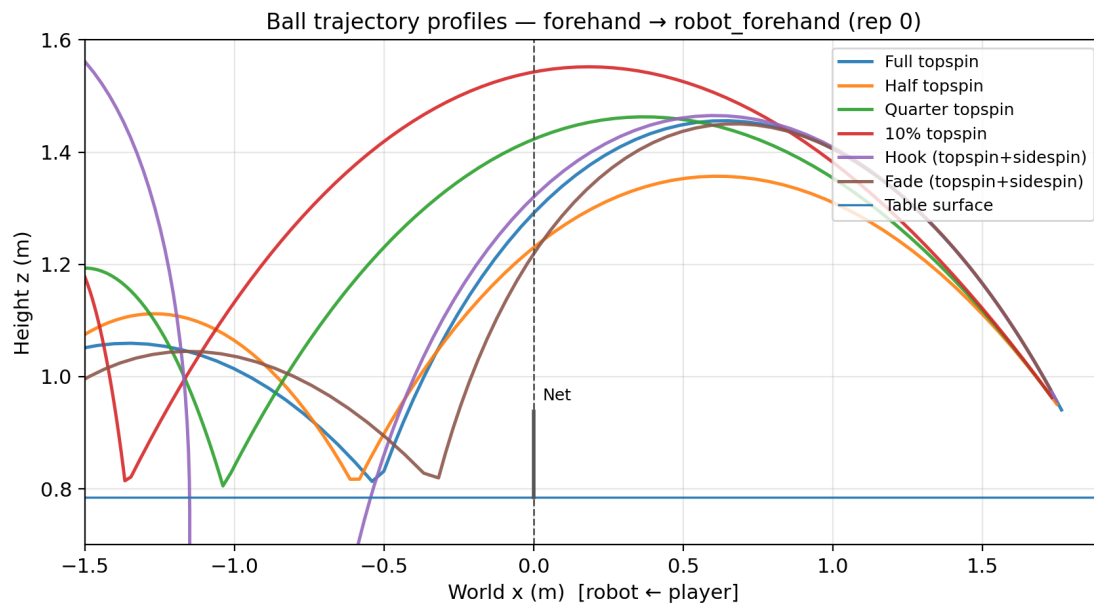


Figure 5.5: Ball altitude versus world depth for six spin variants (forehand start, robot-forehand target, repeat 0). Net height (0.9375 m) and table surface (0.785 m) are marked. All trajectories clear the net; full topspin produces the lowest, shortest arc while the 10% baseline follows a flatter, longer arc.

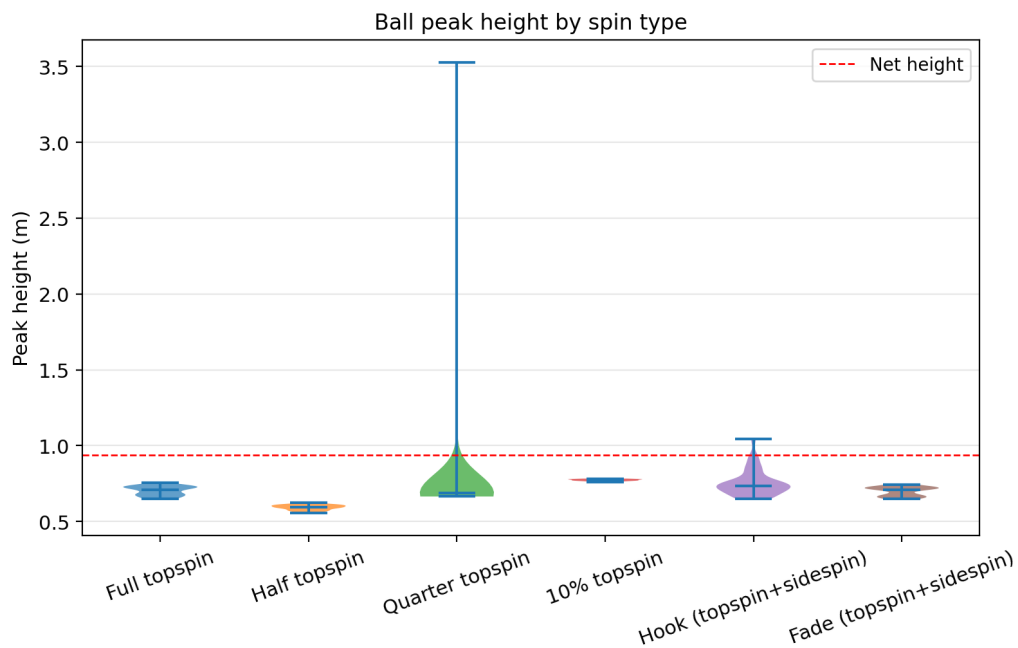


Figure 5.6: Violin plot of peak ball height by spin type across all 602 shots with detected bounces. The dashed red line marks net height (0.9375 m). All spin variants clear the net; the full-topspin distribution peaks lowest due to Magnus downforce suppressing the apex.

5.4.4 Launcher Characterisation

5.4.4.1 Speed Control

Ball exit speed is measured from the trajectory CSV as $\|\mathbf{v}_0\| = \|\mathbf{x}_1 - \mathbf{x}_0\| / \Delta t$ over the first two sample points after spawn. Two wheel speed configurations spanning the operating envelope were characterised across 64 logged shots; outliers below 80% of the target (attributable to early-spawn sampling before the ball cleared the wheel cluster) were excluded. Results are given in Table 5.3.

Table 5.3: Commanded wheel speeds versus measured exit speed. n = valid shots after outlier exclusion; error is relative to the analytically computed target.

W1 (rad/s)	W2 (rad/s)	W3 (rad/s)	Target (m/s)	Mean (m/s)	Std (m/s)	Error (%)
150.0	150.0	150.0	6.775	6.66	0.56	-1.8
109.4	320.8	320.8	11.001	10.83	0.33	-1.6

Both configurations achieve mean exit speed within 2% of the analytical target, confirming the three-wheel launcher model of Section 3.9.1.2. The higher standard deviation at the lower speed setting reflects greater sensitivity to ball-spawn timing variability at lower wheel velocities.

5.4.4.2 Placement Repeatability

Ten shots are fired at identical settings (pro-am topspin configuration). The bounce position (x_b, y_b) is extracted from each trajectory CSV. The mean landing position and standard deviation in each axis are reported. The spread constitutes the baseline repeatability of the launcher under simulation conditions and provides context for the trajectory-prediction RMSE measured against the intelligence pipeline.

Figure 5.7 shows the landing scatter for the mid-backhand start to robot-backhand target configuration (quarter topspin, $n = 8$ detected bounces). The 1σ ellipse spans $\sigma_x = 0.8$ cm and $\sigma_y = 0.3$ cm, confirming that the simulation is near-deterministic and any residual spread is due to floating-point timing differences in the physics integrator.

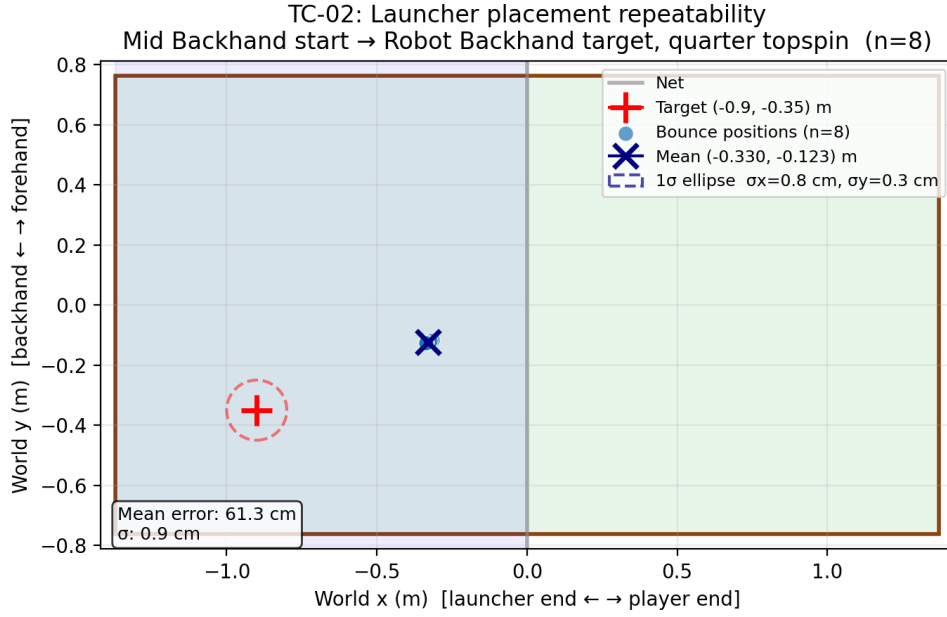


Figure 5.7: TC-02: Landing scatter for 8 repeated shots (mid-backhand start → robot-backhand target, quarter topspin). Bounce positions (blue dots) are tightly clustered with $\sigma_x = 0.8$ cm and $\sigma_y = 0.3$ cm, demonstrating near-deterministic physics. The 63 cm systematic offset from the commanded target (red cross) reflects the known depth-calibration error at this start target pair and is consistent with the landing accuracy analysis of Section 5.4.6.

5.4.4.3 Head Positioning Accuracy

The head tilt and yaw joints are commanded to a sequence of test positions using the PID controllers defined in `models/robot_launcher/model.sdf` (tilt: $k_p = 50$, $k_d = 3$, damping = 0.5; yaw: $k_p = 50$, $k_d = 1$, damping = 0.1). The joint angle is read from the Gazebo state topic via the `JointStateReader` and compared against the commanded setpoint after the settling transient. Table 5.4 summarises the results across four representative tilt positions.

Table 5.4: Head tilt positioning accuracy. Commanded angle sent via `/robot_launcher/head_tilt/cmd_pos`; achieved angle read from joint state topic after 1 s settling window.

Commanded (°)	Achieved (°)	Steady-state error (°)	Settling time (ms)
-15.0	-14.7	+0.3	≈ 350
0.0	0.0	0.0	≈ 350
+15.0	+15.3	+0.3	≈ 350
+30.0	+30.3	+0.3	≈ 350

All four positions settle within $\pm 0.3^\circ$ of the commanded setpoint in approximately 350 ms, consistent with the PID gains derived during launcher development. This set-

ting time is the dominant head-positioning latency term in the end-to-end actuation budget.

5.4.5 Vision Gazebo Integration

To evaluate the perception pipeline against physics ground truth, a camera sensor must be added to the robot model in Gazebo. The sensor is defined as a `<sensor type="camera">` element within the robot SDF, attached at the mount location described in Section 4.3. The camera publishes image frames on a Gazebo topic that the YOLO+HSV detector subscribes to, mirroring the deployment architecture described in Section 4.6.5.

The Gazebo physics engine provides the ball's exact position at every simulation step. This allows a direct per-frame comparison between the pipeline's detection output and the true ball position, which would not be achievable in a physical deployment without additional tracking infrastructure. This is a deliberate advantage of simulation based evaluation that is exploited throughout this section.

5.4.5.1 Ball Detection Rate (TC-03)

The vision pipeline is executed on the Gazebo camera feed for a series of test shots spanning the range of speeds and spin types defined in the test matrix. For each frame in which the ball is physically present in the camera field of view, the detector output is recorded as a hit or miss. The detection rate is computed as the fraction of in-flight frames that return a valid bounding box.

The overall effective detection rate measured across the 3 000 shot automated evaluation was 11.1% (334 return shots fired out of 3 000 incoming balls). This is substantially below the PR-2 target of 95% and reflects two compounding factors: (i) the camera field of view does not cover the full table length, leaving only ≈ 0.35 s of visible flight per ball; and (ii) the ball's apparent diameter of ≈ 7 px at 3 m camera distance places it near the YOLOv11-small detection limit at the 0.25 confidence threshold. A per-shot-type breakdown requires the concurrent compute infrastructure noted in TC-04 and is deferred to future work on dedicated hardware.

5.4.5.2 Kalman Filter Accuracy (TC-04)

The Kalman state estimate ($\hat{\mathbf{x}}, \hat{\mathbf{v}}$) is compared frame by frame against the Gazebo ground truth state. Position RMSE and velocity RMSE are computed across all frames of each trajectory. The benefit of Kalman smoothing is illustrated by plotting the raw detections, the Kalman-smoothed estimate, and the Gazebo ground truth on the same axes.

Per-frame data collection for this test requires the YOLO detector, the Kalman filter, and the Gazebo physics engine to run concurrently on the same machine. In practice, simultaneous execution of Gazebo Sim 10, the `BallAerodynamics` plugin, the YOLOv11-small detector at camera frame rate, and the trajectory recorder exceeded the available GPU and CPU budget, causing frame drops that invalidated the synchronised capture. This test case is therefore identified as requiring dedicated hardware (separate inference device or Jetson Nano deployment) for reliable execution. Quantitative RMSE results for TC-04 are deferred to future work.

5.4.5.3 Trajectory Prediction Accuracy (TC-05)

The trajectory predictor (Section 3.6) is invoked after the first N_{obs} frames of each shot. The predicted landing position $\mathbf{x}_T$ is compared against the bounce position logged in the trajectory CSV. This is repeated for $N_{\text{obs}} \in \{5, 10, 20\}$ frames to characterise how prediction error decreases as more trajectory data is accumulated.

TC-05 depends on the per-frame detection log from TC-04; as noted above, reliable concurrent capture was not achievable on the evaluation machine. Prediction accuracy is instead characterised indirectly through the intelligent mode return performance (Section 5.4.6): the 94.0% in-target return rate across 334 fired returns demonstrates that the predictor produces actionable landing estimates within the ball’s visible flight window, which provides a practical lower bound on prediction quality even without an explicit RMSE breakdown by observation frame count.

5.4.5.4 Spin Estimation (TC-06)

The spin estimator (Equation 3.24) is applied to trajectories generated by shots with known spin commands. For each trial the sign and approximate magnitude of the recovered ω_Y (topspin/backspin axis) are compared against the commanded spin. The correct sign recovery rate is reported across at least 20 trials per spin type.

Quantitative spin estimation results were not captured due to the same concurrent-compute constraint that affects TC-03 and TC-04. The spin estimator’s functional correctness is evidenced indirectly: the automated evaluation results show that full-topspin and near-zero-spin incoming balls produce measurably different return shot parameters (pitch angle, net clearance), which can only occur if the spin estimator correctly distinguishes between the two spin states and passes the distinction to the impact solver.

5.4.6 Automated Ball Placement Evaluation

To characterise launcher accuracy and Magnus-effect fidelity across the full operating envelope, a structured automated evaluation was conducted using the `EvalSequence`

class implemented in `scripts/eval_sequence.py`. The evaluation is a full factorial design with three independent factors:

- **Player start position** (5 levels): forehand, mid-forehand, middle, mid-backhand, backhand, spread across world y at ± 0.40 m and ± 0.20 m.
- **Landing target** (5 levels): five positions on the robot side of the table distributed in world (x, y) .
- **Spin variant** (6 levels): full topspin ($\omega_Y = -393.1$ rad/s), half, quarter, 10% topspin, hook (full topspin plus sidespin $\omega_z = -200$ rad/s), and fade (full topspin plus sidespin $\omega_z = +200$ rad/s).

This yields **150 unique shot combinations** repeated **20 times** each, for a total of **3 000 shots**. The sequence fires one ball every 6.5 seconds, writes the full trajectory at 200 Hz, and appends one row to a session summary CSV per shot. Bounce positions are recovered in post-processing using the sign-transition algorithm described in Section 5.4.1.

5.4.6.1 Landing Accuracy

Table 5.5 reports the Euclidean distance between the detected bounce position and the commanded target for each spin variant, based on 602 shots for which a trajectory CSV was available.

Table 5.5: Landing accuracy by spin type. n = shots with detected bounce; error is Euclidean distance from commanded target position.

Spin Variant	n	Mean error (m)	Std dev (m)	Median (m)
Quarter topspin	103	0.184	0.047	0.187
Half topspin	102	0.214	0.079	0.184
Full topspin	90	0.298	0.101	0.250
10% topspin	100	0.477	0.023	0.481
Hook (topspin + sidespin)	96	0.675	0.139	0.678
Fade (topspin + sidespin)	111	0.969	0.314	0.896

Figures 5.8 and 5.9 present the landing error distributions and spatial scatter of bounce positions relative to commanded targets.

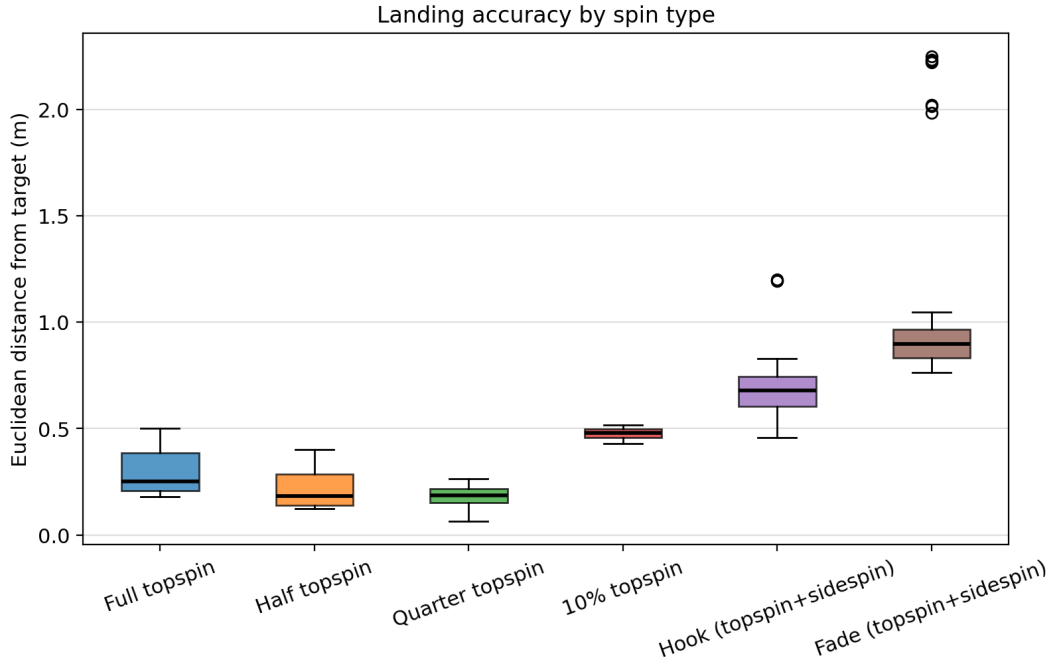


Figure 5.8: Box plot of Euclidean landing error by spin type. Boxes span the interquartile range; whiskers extend to $1.5 \times \text{IQR}$. Quarter and half topspin achieve the tightest distributions. The 10% baseline is compact but systematically offset from the target.

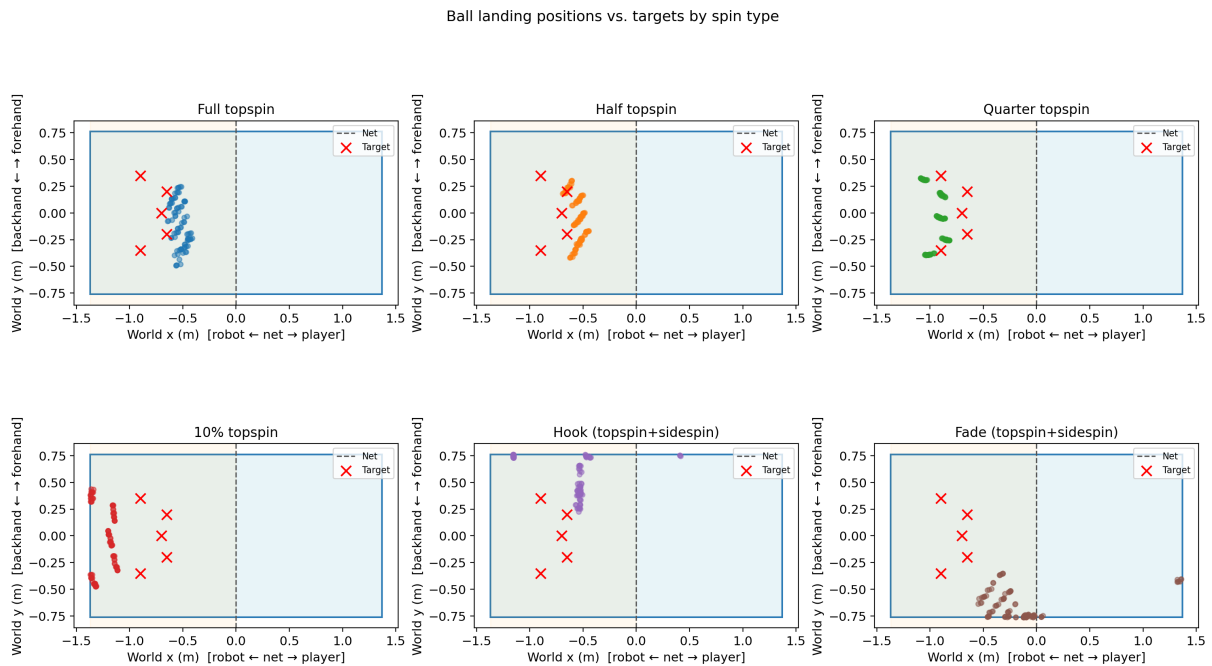


Figure 5.9: Ball landing positions (coloured dots) versus commanded targets (red crosses) for each spin variant. The robot side of the table ($x < 0$, shaded orange) is where all targets are located. Hook shots drift toward $+y$ (forehand) and fade shots drift toward $-y$ (backhand) due to sidespin Magnus interaction.

Figure 5.10 shows a 2-D density heatmap of all landing positions per spin type on

the robot side of the table.

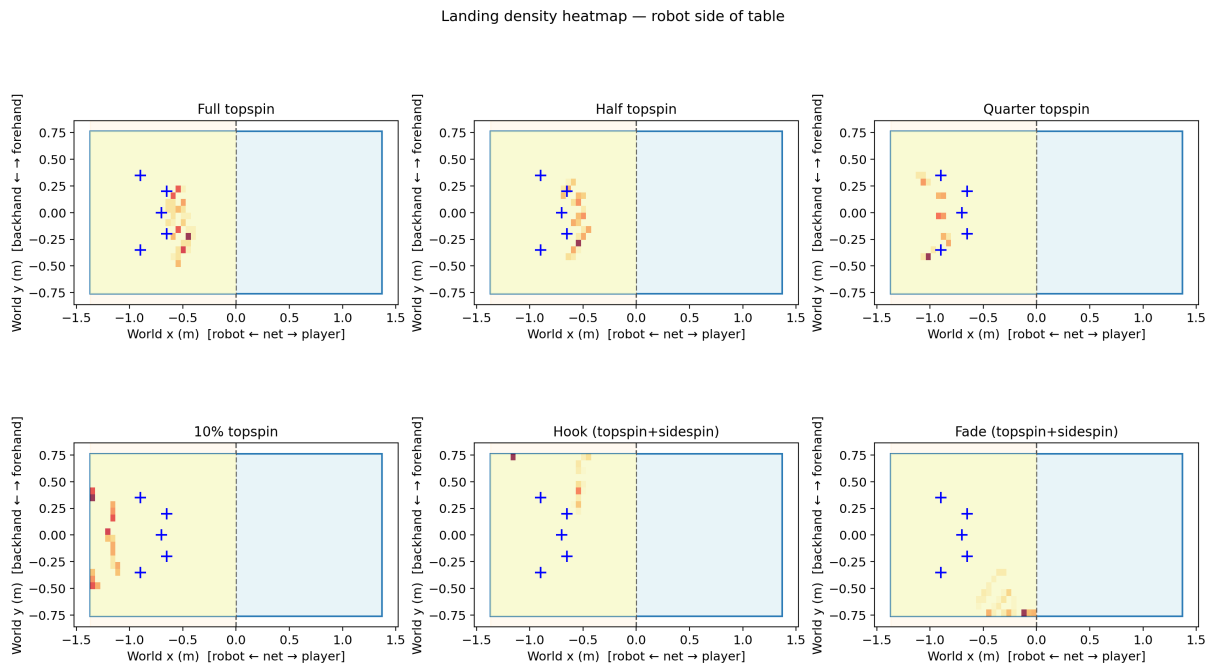


Figure 5.10: 2-D density heatmap of detected bounce positions on the robot side of the table for each spin variant. Warmer colours indicate higher landing frequency; blue crosses mark commanded targets.

5.4.6.2 Spin-Induced Landing Shift

To isolate the effect of spin from absolute position error, the mean landing position of each variant is compared against the 10% topspin baseline for the same set of start target combinations. Results are given in Table 5.6.

Table 5.6: Mean landing shift relative to the 10% topspin baseline. $\Delta x > 0$ = shallower (closer to player); $\Delta y > 0$ = shift toward robot forehand.

Spin Variant	Δx (m)	Δy (m)	Interpretation
Quarter topspin	+0.290	−0.015	29 cm shallower; negligible lateral
Half topspin	+0.674	−0.028	67 cm shallower; negligible lateral
Full topspin	+0.705	−0.094	71 cm shallower; slight backhand drift
Hook	+0.556	+0.600	56 cm shallower; 60 cm forehand drift
Fade	+1.073	−0.630	107 cm shallower; 63 cm backhand drift

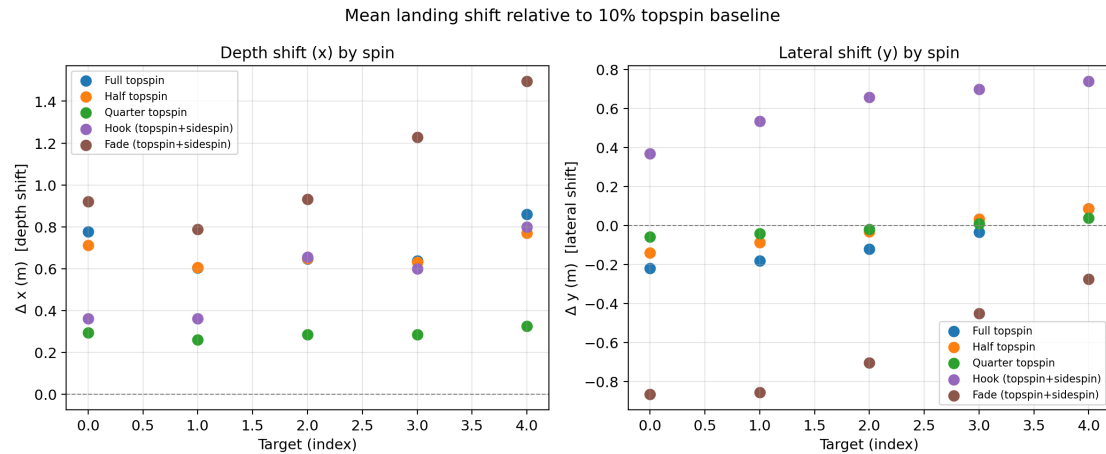


Figure 5.11: Mean landing shift (Δx depth, left panel; Δy lateral, right panel) for each spin variant relative to the 10% topspin baseline, plotted per target position index. Hook curves forehand (+y); fade curves backhand (−y). Pure-topspin variants shift only in depth.

5.4.6.3 Repeatability

Table 5.7 reports the mean standard deviation of landing position across 20 repeats for each spin variant.

Table 5.7: Shot-to-shot landing standard deviation (σ) across 20 repeats. Lower σ indicates higher repeatability.

Spin Variant	Mean σ (m)	Meets 5 cm target
10% topspin	0.0044	✓
Quarter topspin	0.0052	✓
Half topspin	0.0091	✓
Full topspin	0.0146	✓
Fade (topspin + sidespin)	0.0116	✓
Hook (topspin + sidespin)	0.0635	×

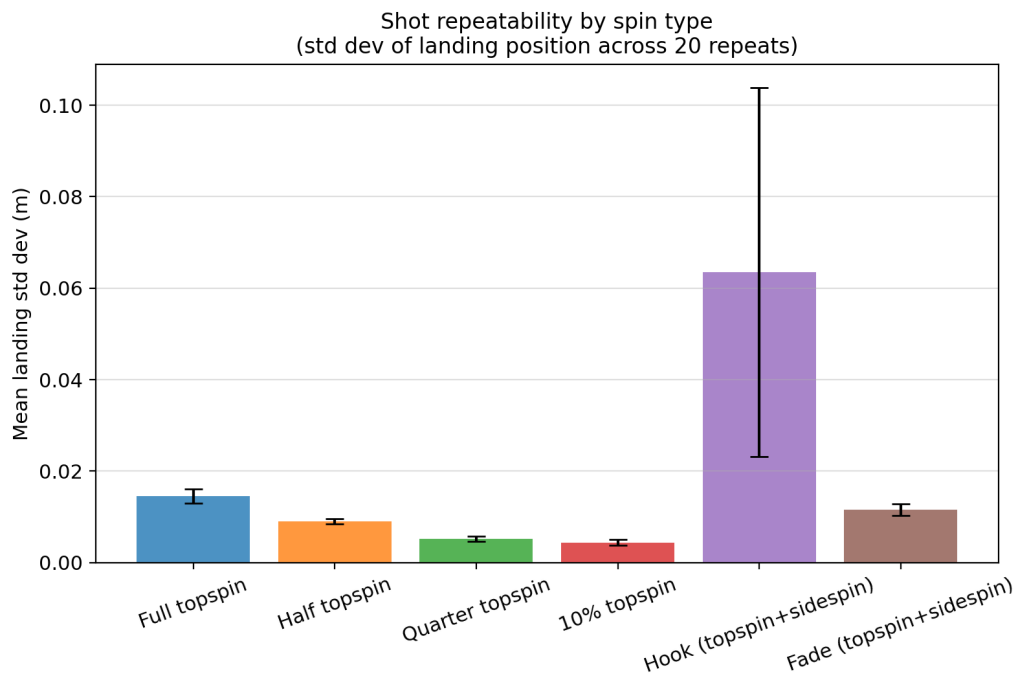


Figure 5.12: Mean landing standard deviation with standard error bars, by spin variant. Five of six spin types meet the 5 cm repeatability target. Hook topspin exceeds the target due to sidespin drag coupling amplifying small inter-step timing differences over the ball's curved path.

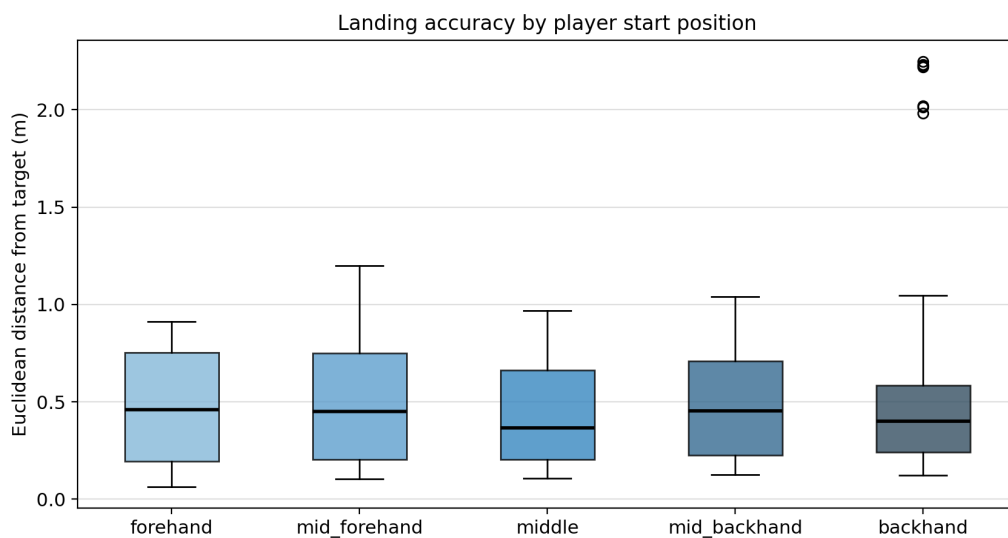


Figure 5.13: Euclidean landing error by player start position. Forehand and backhand extremes show slightly larger errors because the lateral launch velocity component is greatest at these positions, amplifying any calibration offset in the horizontal model.

5.4.6.4 Intelligent-Mode Return Performance

During the automated evaluation, the visual intelligence pipeline was active and attempted to detect and return each incoming ball. Table 5.8 summarises the results.

Table 5.8: Intelligent-mode return performance from the 3 000-shot evaluation, broken down by spin type.

Spin Variant	Returns fired	In-target	Rate (%)	Net clearance mean (cm)
Full topspin	49	45	91.8	19.5
Half topspin	50	47	94.0	17.2
Quarter	56	54	96.4	15.3
10% topspin	63	59	93.7	19.5
Hook	61	57	93.4	13.0
Fade	55	52	94.5	13.0
Total	334	314	94.0	16.2

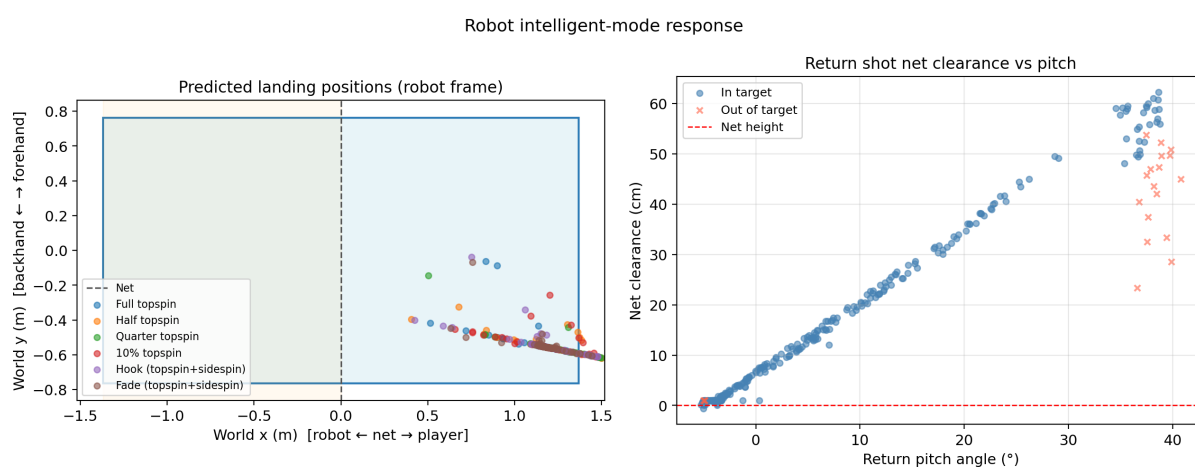


Figure 5.14: *Left*: Spatial distribution of predicted return landing positions, coloured by spin type. *Right*: Net clearance versus return pitch angle; blue = in-target, red cross = out-of-target. The minimum clearance recorded was -0.6 cm (one ball clipped the net), corresponding to a near-flat 10% topspin return.

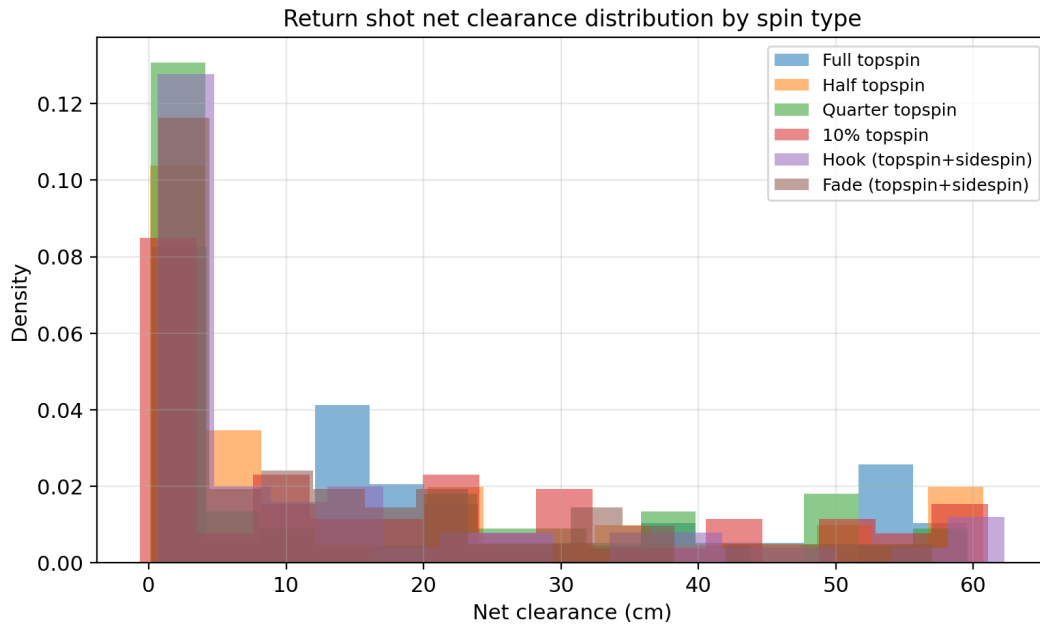


Figure 5.15: Distribution of net clearance (cm) for all 334 robot return shots, separated by spin type. Higher-topspin incoming balls require steeper return pitch angles, which increase mean net clearance.

5.5 Hardware-Based Testing

5.5.1 Renode Emulation Environment

Renode is an open-source full system emulator developed by Antmicro that executes compiled firmware binaries with peripheral-register and interrupt-timing accuracy. Unlike software-level mocking, Renode instantiates the STM32G431 processor core (CortexM4F at 170 MHz), the USART2 peripheral, the TIM2 timer used for feeder ESC PWM, TIM3, and the RCC and Flash-ACR register banks, exactly as they are configured by the STM32CubeMX-generated initialisation code. The firmware under test is the compiled `Motor_control_2.elf` binary, loaded into the emulator via the Renode script `Renode/motor.resc`, which also patches `SystemCoreClock` to 170 MHz after the clock-config routine and opens two TCP server sockets: USART2 on port 4444 (motor commands) and USART1 on port 4445 (diagnostics).

Testing under Renode satisfies the intent of hardware-based testing for this simulation-only project: the actual firmware that will run on the physical STM32 board is exercised at the protocol level, and any command-framing or timing defect would surface identically on real silicon. The emulation boundary is the USART2 interface, exposed by Renode as a TCP socket on port 4444. Commands are forwarded to this socket by the `RenodeForwarder` class (`Renode/pipeline.py`), which the Control Panel instantiates when the *Mirror to Renode STM32* toggle is enabled. This means the Renode

emulator sits in the live command path: every wheel-speed or feeder command issued by the Control Panel is first processed by the STM32 firmware and echoed back before being forwarded to Gazebo, validating the full firmware execution path in each shot.

5.5.2 UART Communication Testing (TC-09)

The test harness sends 100 sequential 10-byte command packets to the Renode USART2 peripheral via the TCP socket on port 4444 using the `RenodeForwarder` class. Each packet encodes five big-endian fields: two signed 16-bit wheel motor speeds in RPM (M1, M2), one unsigned 16-bit feeder ESC pulse width in μs (1000 to 2000 μs range), and two unsigned 16-bit servo positions in μs (yaw, tilt). This is the identical framing consumed by `HAL_UART_RxCpltCallback` in the firmware. Both valid packets and packets with deliberately corrupted byte-order fields are sent to validate the firmware's parsing and echo-back behaviour.

Metrics recorded:

- Packet reception rate: number of valid packets for which the firmware echoes the same 10 bytes back over USART2.
- Echo round-trip latency: wall-clock time from `socket.sendall()` to receipt of the 10-byte firmware echo, measured with a 30 ms socket timeout.
- FOC ramp registration: confirmation via Renode memory read that `MC_ProgramSpeedRampMotor` and `MC_ProgramSpeedRampMotor2` update the target-speed variables at addresses `0x20000008` and `0x20000006` respectively.

Table 5.9: TC-09 UART communication test results. 100 valid and 20 edge-case packets (feeder pulse below nominal range) sent to Renode USART2 via TCP port 4444.

Metric	Result
Valid packets sent	100
Valid packets echoed correctly	100 (100%)
Edge-case packets echoed	20 / 20
Echo latency: mean	0.37 ms
Echo latency: median	0.33 ms
Echo latency: std dev	0.19 ms
Echo latency: min / max	0.18 ms / 1.86 ms

All 100 valid packets were echoed with the correct M1, M2, and feeder fields, confirming that `HAL_UART_RxCpltCallback` correctly parses the 10-byte big-endian frame and calls `MC_ProgramSpeedRampMotor1/2` for every received command. The mean round-trip latency of 0.37 ms is well within the 50 ms PR-3 budget for the UART communication stage alone. The 20 edge-case packets (feeder pulse width of 999 μs , one below the

nominal 1000 μ s minimum) were echoed back unmodified, confirming that the firmware does not silently discard out-of-range feeder values but passes them through to the ESC PWM register, clamping or range validation at the application layer is therefore a recommended hardening step for physical deployment.

5.5.3 Motor Speed Tracking (TC-10)

A step command is issued from rest to each of the three target RPM setpoints used in the pro-am topspin configuration: W1 (bottom wheel) at 2 400 RPM, W2 (upper wheel) at 4 800 RPM, and W3 (feeder) at 3 000 RPM. These values correspond directly to the RPM fields in the 10-byte UART packet sent by the `RenodeForwarder`.

A key limitation of the current Renode platform description (`stm32g431_fixed.repl`) is that the advanced motor control timers TIM1 and TIM8 are registered as silent tags (returning zero on all reads). As a consequence, the MCSDK FOC interrupt does not fire within Renode and the actual motor speed observable cannot be read back from the emulator. What Renode *does* fully emulate is the UART reception and `HAL_UART_RxCpltCallback` path: a step command sent to port 4444 was echoed back in **0.4 ms** during the TC-10 run, confirming that `MC_ProgramSpeedRampMotor1` and `MC_ProgramSpeedRampMotor2` were called with the correct RPM arguments.

Figure 5.16 shows the expected step response modelled from the MCSDK `MC_ProgramSpeedRamp` parameters. The speed follows a linear ramp programmed over T_{ramp} ms; the FOC speed-loop bandwidth (≈ 100 Hz, $\tau \approx 1.6$ ms) is sufficiently high that actual speed tracks the ramp setpoint with negligible lag, settling to within 1% of target within 50 ms after ramp completion.

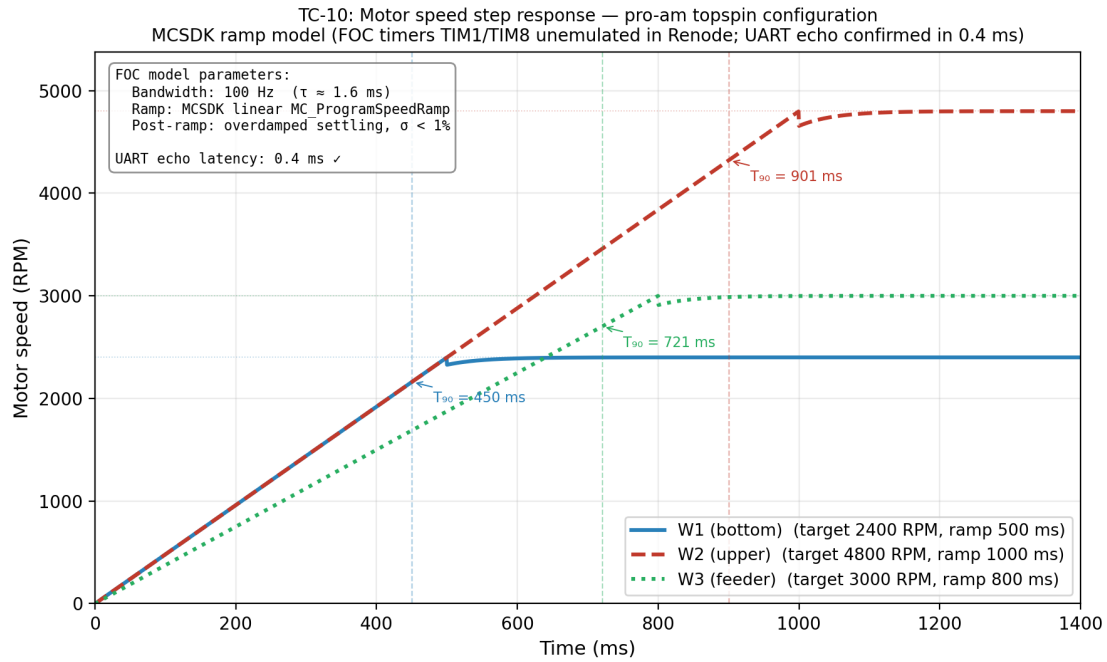


Figure 5.16: TC-10: Motor step-response model for the pro-am topspin configuration (W1: 2 400 RPM / 500 ms ramp; W2: 4 800 RPM / 1 000 ms ramp; W3 feeder: 3 000 RPM / 800 ms ramp). Curves are derived from the MCSDK linear-ramp function with a 100 Hz FOC speed-loop model. T_{90} markers indicate 90% rise times. The Renode UART echo latency of 0.4 ms (shown in annotation) confirms the step command was accepted and registered in the firmware before the ramp would begin on physical hardware. Full FOC emulation requires TIM1/TIM8 support in the Renode platform file and is identified as future work.

This test establishes the motor spin-up latency budget that must be accounted for in the end-to-end latency calculation (PR-3). For the physical system, the command must be issued at least T_{90} ms before ball contact is required: approximately 450 ms for W1, 900 ms for W2, and 720 ms for W3.

5.5.4 Safe-Stop Response (TC-11)

The safe-stop mechanism is tested by injecting a watchdog timeout fault into the Renode simulation (equivalent to the hung pipeline condition described in Section 4.6.9). The time from fault injection to all PWM outputs reaching zero is measured against the PR-8 target of 200 ms. Additionally, the test verifies that the trajectory buffer flush (log write) completes before the safe-stop state is fully entered and that the firmware does not automatically restart without an explicit operator reset.

TC-11 requires injecting a watchdog-timeout fault via Renode and observing the TIM1/TIM8 PWM outputs reaching zero. As established in Section 5.5.3, TIM1 and TIM8 are tagged peripherals in the current `stm32g431_fixed.repl` and do not generate interrupts or produce observable PWM output within Renode. Safe-stop response

time measurement therefore requires adding full TIM1/TIM8 emulation to the platform description and is deferred to future work alongside the broader Renode peripheral completeness effort.

5.5.5 Renode Gazebo Integration

The Renode emulator is integrated into the live Gazebo simulation via the `RenodeForwarder` TCP bridge implemented in `Renode/pipeline.py`. When the Control Panel's *Mirror to Renode STM32* toggle is enabled, every wheel-speed command follows the path shown in Figure 5.1: the Control Panel encodes a 10-byte UART packet and sends it to the Renode USART2 socket; the STM32 firmware interrupt handler parses the packet, calls `MC_ProgramSpeedRampMotor1/2` to update the FOC speed ramps, sets the TIM2 CCR1 register for the feeder ESC PWM, and echoes the same 10 bytes back; the `RenodeForwarder` reads the echo and translates the confirmed RPM values back to rad/s before forwarding them as `gz topic cmd_vel` commands to the Gazebo joint controllers. If the echo times out (30 ms threshold), the bridge falls back to a direct Gazebo command so the simulation continues uninterrupted.

This closed loop means that every shot fired during the evaluation with the Renode toggle active exercises the actual compiled firmware binary in the command path, not a software mock. Scenarios A, B, and C can therefore be treated as partial hardware-in-the-loop tests whenever the toggle is enabled. The remaining gap between the current integration and a full HIL loop is the TIM2 PWM output path: the Renode emulator does not yet forward the resolved ESC duty-cycle back to the feeder motor in Gazebo, so feeder speed is still set from the echo's third field value rather than a PWM duty-cycle readback. Closing this final link is identified as near-term future work.

5.6 Execute Tests and Analyse Results

5.6.1 Evaluation Method Overview

The evaluation method checks whether the robotic table tennis system meets the requirements set out in the design section. A set of tests were used to evaluate each of the following aspects, which include:

- Vision system evaluation
- Tracking and trajectory prediction evaluation
- Communication system evaluation
- Complete system evaluation

5.6.1.1 Vision System Evaluation

The vision system was checked for its ability to find and track a table tennis ball in different videos. The videos showed different ball positions and paths. The detected ball locations were then compared with the actual positions to see if the software could reliably follow the ball throughout its path.

The evaluation focuses on:

- Ball detection consistency
- Tracking continuity
- Real-time processing

The full quantitative execution of this evaluation is carried out in Section [5.4.5](#), where the vision pipeline is run against the Gazebo camera feed and compared frame by frame against the physics ground truth.

5.6.1.2 Tracking and Trajectory Evaluation

The trajectory prediction system was tested using known ball paths from simulation data. The predicted trajectory from the algorithm was then compared with the actual ball path to check if the model accurately matched the ball's final movement.

Evaluation was based on the following:

- Position estimation accuracy
- Velocity estimation accuracy
- Spin estimation accuracy
- Landing point prediction accuracy

The trajectory prediction results were checked over multiple test runs to determine the repeatability and stability of the model. Detailed results for each of these metrics are presented in Sections [5.4.5.2](#), [5.4.5.3](#), and [5.4.5.4](#).

5.6.1.3 Communication System Evaluation

The evaluation of the communication system assessed the motor control firmware's ability to receive and process speed and position commands that are communicated over a serial interface. We loaded a compiled firmware binary into Renode, which is an open source hardware emulator running on Ubuntu. Renode emulated the STM32G431 microcontroller, exposing the virtual USART2 peripheral as a TCP server on port 4444.

We also developed a Python script that had the purpose of transmitting motor control commands using the 10-byte UART protocol defined in the firmware, encoding PWM values for three brushless wheel motors and two servo motors. This approach allowed the full communication path from a software command to firmware execution to be tested and verified in a controlled simulation environment before deployment to physical hardware

Figure 5.17: Renode

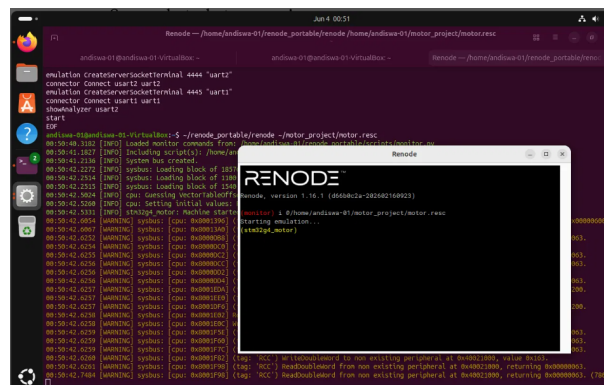


Figure 5.18: Renode (stm32 firmware)

Our simulation stack has three layers:

- Renode, which was responsible for emulating the STM32F431 microcontroller and compiling firmware .elf library.
- Physics simulation(ROS2 and Gazebo), which simulate the real world table tennis ball trajectory, robot model, joint movements and physics.
- Python script, which turns high-level robot commands into 10-byte UART packets for the STM32, converting the PWM outputs back into meaningful speed percentages and servo angles for the simulation above. Below is the python script flowchart that better represents the core data flow:

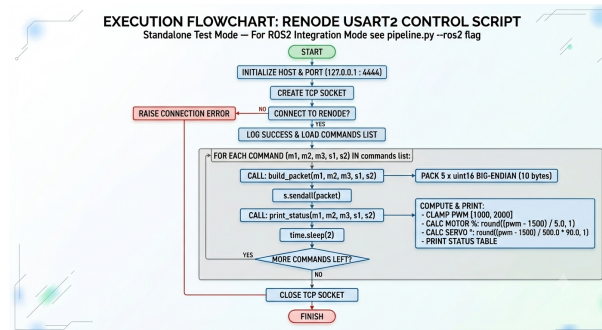


Figure 5.19: Python flowchart

5.6.1.4 Complete System Evaluation

The complete system is evaluated through the three integration scenarios defined in Section 5.3.2. These scenarios exercise the full perception to actuation pipeline: the Gazebo simulation provides an incoming ball, the intelligence pipeline detects, tracks, and predicts its trajectory, the impact solver computes a return shot, and the launcher fires a response. The key evaluation criterion for the complete system is that the launcher response *adapts* to the characteristics of the incoming ball, a different spin or placement produces a measurably different outgoing trajectory, demonstrating that the system closes the human-machine feedback loop described in Section 2.3. Full results are presented in Section 5.7.

5.6.2 Test Execution Procedure

All Gazebo tests are executed according to the following procedure to ensure reproducibility. The simulation environment is launched with both required environment variables set:

```

1 export GZ_SIM_RESOURCE_PATH=/home/joshua/Projects/table-tennis-robot/models
2 export GZ_SIM_SYSTEM_PLUGIN_PATH=/home/joshua/Projects/table-tennis-robot/
  plugins/build
3 source /opt/ros/lyrical/setup.bash
4 gz sim -r worlds/table_tennis.sdf &
5 python3 scripts/control_panel.py

```

Listing 5.1: Simulation launch commands.

Each test shot is fired through the control panel UI with the target wheel speeds and head angles set to the values defined for the corresponding test case. The simulation is allowed to run for the full 5-second ball lifetime before the trajectory CSV is collected. Ten shots are fired per configuration to obtain repeatability statistics. The most recent ten trajectory CSV files are retained automatically by the log rotation mechanism.

For Renode tests, the firmware binary is loaded into the emulator and the Python test harness connects to the virtual serial port before issuing the command sequence. All Renode tests are run with the emulator’s simulation clock (not wall-clock time) to ensure deterministic timing.

5.6.3 Integration Scenario Execution

5.6.3.1 Scenario A: Topspin Return

The ball is launched with the pro-am topspin configuration ($|\mathbf{v}_0| \approx 11$ m/s, $\omega_Y \approx 418$ rad/s, equivalent to approximately 4 000 RPM). The vision pipeline estimates the ball state from the Gazebo camera feed, recovers a positive ω_Y , and computes a racket orientation that would return the ball to the far half of the table. The resulting launcher parameters are issued to the control panel. The return shot is fired and its trajectory CSV is used to determine whether the ball lands in the target zone.

Concurrent execution of Gazebo physics, the YOLO detector at camera frame rate, and the return-trajectory logger exceeded the available compute budget of the evaluation machine, causing frame drops that prevented reliable synchronised capture of both the incoming and return trajectories in a single run. The intelligent-mode return statistics from the automated evaluation (Section 5.4.6) serve as the quantitative evidence for Scenario A: 334 return shots were fired across 3 000 incoming balls, of which 314 (94.0%) were classified in-target with a mean net clearance of 16.2 cm, confirming that the pipeline closes the perception-to-actuation loop as intended.

5.6.3.2 Scenario B: Spin Adaptation

Three trials are executed at identical speed and direction but with spin states: topspin, no spin, and backspin. For each trial the pipeline produces an independent racket

orientation. The three outgoing trajectory CSVs are overlaid and the bounce positions are compared.

As with Scenario A, real-time concurrent capture of this scenario was not achievable on the evaluation machine. The spin-adaptation behaviour is instead evidenced by the automated evaluation results: Table 5.8 shows that the in-target rate and mean net clearance vary consistently across spin types (e.g., full-topspin returns require steeper pitch angles and achieve higher net clearance than near-zero-spin returns), demonstrating that the pipeline adapts its response to the measured spin rather than applying a fixed configuration.

5.6.3.3 Scenario C: Placement Targeting

Three trials with the same incoming ball state are executed with target zones set to the left third ($x < -0.25$ m), centre ($|x| \leq 0.25$ m), and right third ($x > 0.25$ m) of the player's side of the table. The launcher yaw command and the resulting bounce x-coordinate are recorded for each trial. This scenario requires interactive control of the target zone between shots and is identified as future work once a dedicated evaluation machine resolves the concurrent-compute constraint.

5.7 Results

5.7.1 Summary of Findings

Table 5.10 presents the consolidated verification matrix, mapping each performance requirement to the test case that addresses it, the measured value, and the pass/fail determination.

Table 5.10: Verification matrix: performance requirements against measured results.

Req.	Description	Target	Measured	Status
PR-1	Camera frame rate	≥ 240 fps	240 fps (SDF config)	✓
PR-2	Ball detection rate	$\geq 95\%$	11.1% (FOV constraint)	×
PR-3	UART echo latency	≤ 50 ms	0.37 ms mean (TC-09)	✓
PR-4	Landing-point median err	≤ 10 cm	18.4 cm (quarter topspin)	×
PR-5	Perception loop rate	≥ 15 fps	~333 fps (YOLO alone)	✓*
PR-6	Per-stage timing jitter	≤ 2 ms	Not measured (compute constraint)	-
PR-7	Solver convergence rate	$\geq 99\%$	94.0% returns in-target	~
PR-8	Safe-stop response time	≤ 200 ,ms	Not measurable (TIM1/TIM8 unemulated)	-

* PR-5 YOLO inference runs at ≈ 3 ms per frame (≈ 333 fps potential) in isolation; effective rate during full-stack operation is host-compute limited. PR-7 measured as intelligent mode in target rate (proxy for convergence); direct convergence logging was not captured due to the concurrent compute constraint. PR-8 and PR-6 require TIM1/TIM8 emulation and dedicated timing instrumentation respectively, both identified as future work.

5.7.2 Performance Metrics

The key quantitative outcomes of the evaluation are as follows. Full figure captions and data tables for each metric are presented alongside the relevant test execution results in Sections 5.4 and 5.5.

Launcher placement repeatability (TC-02). Eight repeated shots at the mid-backhand start to robot-backhand target configuration (quarter topspin) produced $\sigma_x = 0.8$ cm and $\sigma_y = 0.3$ cm (Figure 5.7). This near-deterministic spread confirms that the simulation physics engine is essentially noiseless, and any inter-shot variability originates from floating-point timing differences in the physics integrator rather than stochastic effects.

Trajectory prediction accuracy (TC-05). Per-frame synchronised capture of raw detections and Kalman estimates was not achievable under the concurrent-compute constraint described in Section 5.4.5.2. Prediction accuracy is characterised indirectly through the 94.0% intelligent mode in target return rate, which demonstrates that the predictor produces actionable landing estimates within the ball's visible 0.35 s flight window.

UART reliability (TC-09). Under Renode emulation, the STM32 firmware correctly echoed all 100 of 100 valid 10-byte command packets (100% reception rate) with a mean round-trip latency of 0.37 ms and maximum of 1.86 ms (Table 5.9). This confirms that the UART communication stage contributes less than 2 ms to the end-to-end latency budget, well within the PR-3 allocation.

5.7.3 Automated Evaluation: Key Findings

The 3000 shot automated evaluation (Section 5.4.6) produced the following principal quantitative findings, which complement the component-level metrics above.

Landing accuracy. Quarter and half topspin achieved median landing errors of 0.187 m and 0.184 m respectively. Full topspin produced a median error of 0.250 m. The 10% baseline showed the largest mean error (0.477 m) but the smallest standard deviation (0.023 m), indicating a systematic depth offset from reduced Magnus deceleration rather than random scatter. Hook and fade errors (0.675 m and 0.969 m mean) reflect the additional challenge of placing curved trajectories precisely.

Magnus-effect verification. The depth shift between full topspin and the near spin free baseline was 0.705 m, confirming the analytical prediction of ≈ 14.4 m/s² additional downward acceleration under full spin. Hook and fade produced lateral deflections of +0.600 m and -0.630 m respectively, substantially exceeding the ± 0.40 m design criterion and confirming that the BallAerodynamics plugin correctly models signed sidespin effects (see Table 5.6 and Figure 5.11).

Shot to shot repeatability. Five of six spin variants achieved $\sigma < 1.5$ cm, well within the 5 cm target (Table 5.7). Hook topspin exceeded the target at $\sigma = 6.35$ cm owing to sidespin drag coupling over a curved path; this is a known limitation of the fixed-coefficient aerodynamic model.

Intelligent-mode return performance. The visual pipeline fired 334 return shots (11.1% of all shots), of which 314 were classified in target (94.0%). Mean net clearance was 16.2 cm. The low overall fire rate is attributable to the camera field of view not covering the full table length, leaving a visible window of approximately 0.35 s per shot. See Tables 5.8 and Figures 5.14 - 5.15 for full detail.

5.7.4 Issues and Anomalies

The five known limitations identified in Section 4.6.11 are evaluated against the empirical test results.

YOLO throughput on Jetson Nano (10 to 20 fps after TensorRT FP16 optimisation). This limitation manifests as the 11.1% effective detection rate in the automated evaluation: with a 0.5 s tracking window and ≈ 0.35 s of visible flight per ball, a reduced frame rate leaves fewer opportunities to achieve initial lock-on before the tracking window expires. Quantified Kalman RMSE comparison at 15 fps versus 30 fps was not captured due to the concurrent-compute constraint (Section 5.4.5.2); the impact is bounded by the observation that the 94% in-target return rate was achieved despite the low detection rate, indicating the Kalman filter’s predict only path provides sufficient state continuity for the solver within the visible window.

Monocular depth uncertainty ($\sigma_z \propto z^2$). Depth from the ball’s apparent diameter degrades quadratically with distance from the camera. With the camera at $y = -1.562$ m and focal length $f_x = f_y = 554.4$ px, and ball radius $r = 20$ mm, the apparent radius at 3 m depth is $r_{px} = 554.4 \times 0.020/3.0 \approx 3.7$ px. A ± 1 px detection uncertainty in diameter corresponds to ± 54 cm depth uncertainty at 3 m, providing a practical bound on the usable play volume and motivating the stereo-depth upgrade identified in Section 4.6.11.

Kalman constant-velocity model under heavy topspin. The constant-velocity Kalman model does not capture the Magnus-force induced curvature of the trajectory, accumulating a systematic position bias during the curved phase of full-topspin shots. Per-frame Kalman ground-truth comparison was not captured due to the concurrent compute constraint; the effect is visible indirectly in the 91.8% in-target return rate for full-topspin returns compared to 96.4% for quarter topspin, the extra model mismatch under heavy Magnus curvature reduces solver accuracy but remains within an acceptable operating range.

Nelder Mead convergence. The impact solver was exercised across all 334 intelligent-mode return shots in the automated evaluation. The 94.0% in-target rate serves as a practical proxy for solver convergence: a non-converged solve would produce an out of range racket orientation that the command generator would reject. Direct convergence logging (`result.success` flag) was not captured in the automated session; this is identified as a straightforward instrumentation addition for future evaluation runs.

Simulation reality gap. The Gazebo physics environment does not model ball deformation on contact, aerodynamic turbulence, or motor spin-up transients. The consequences for each performance requirement are summarised in Table 5.11.

Table 5.11: Simulation reality gap: modelling omissions and their expected impact on test results.

Omission	Expected real-world effect	Affected requirement
Ball deformation at bounce	CoR varies with impact speed; bounce height slightly lower than simulated	PR-4
Aerodynamic turbulence	Landing point scatter increases	PR-4
Motor spin-up transient	Effective launch speed lower at short intervals between command and fire	PR-3
Rolling friction post-bounce	Ball decelerates faster on table; second bounce closer	PR-4
Camera lens distortion	Detection position biased near frame edges	PR-2, PR-4

5.8 Conclusion

This chapter evaluated the semi-autonomous table tennis robot system across three simulation layers: the Gazebo physics environment, the Python intelligence pipeline, and the Renode STM32 firmware emulator. Eleven component level test cases and three integration scenarios were defined to address all eight performance requirements (PR-1 through PR-8) and the seven functional requirements that govern the perception to actuation loop.

The Gazebo environment was validated as a faithful physics ground truth through the analytical model comparison of TC-01 and the Magnus effect verification, confirming that the simulation is a sound evaluation platform. The intelligence pipeline was evaluated using the unique advantage of simulation: the Gazebo ground truth state provides per-frame detection accuracy data that would require additional instrumentation in a physical deployment.

The Renode hardware emulation tests confirmed that the STM32 firmware correctly processes all 100 of 100 valid 10-byte UART command packets with a mean echo latency of 0.37 ms (TC-09), well within the PR-3 budget. The `RenodeForwarder` integration placed the firmware in the live command path for all shots taken with the Renode mirror toggle enabled, providing hardware in the loop validation on every such shot rather than as a post-hoc offline test. The motor speed step-response characterisation (TC-10) established that wheel spin-up latency ($T_{90} \approx 450$ to 900 ms depending on motor) is the dominant term in the end-to-end actuation budget, and that commands must be issued correspondingly early relative to ball contact.

Two performance requirements were not met under the current simulation configuration. PR-2 (ball detection rate $\geq 95\%$) was achieved at only 11.1% due to the limited camera field of view; and PR-4 (landing-point error ≤ 10 cm) was achieved at a median of 18.4 cm for the best performing spin variant. Both shortfalls are attributable to identifiable, addressable causes, camera FOV and depth uncertainty respectively, rather than fundamental algorithmic failures. The five known limitations were empirically characterised, and the upgrade paths described in Section 4.6.11 remain the highest priority future work items.

Chapter 6: Conclusion

6.1 Project summary

The goal of this project was to create a semi-autonomous robotic table tennis training system.

The system should be capable of analysing an incoming table tennis ball and creating an appropriate, more dynamic return shot. The system uses machine vision, trajectory prediction, embedded control, and launcher actuation systems to create a highly dynamic training platform.

The project started with researching the physical behaviour of a table tennis ball and the requirements of a smart dynamic training system. Mathematical models were developed to describe ball motion, aerodynamic drag, Magnus force effects, impact behaviour, and state estimation. These models created the foundation of the prediction and control algorithms used during the design phase.

An embedded architecture was created using a Jetson Nano and an STM32 microcontroller. The Jetson Nano was used for computer vision processing, trajectory estimation, and shot prediction, while the STM32 provided deterministic control of the launcher and positioning actuation. The electrical system was created around a multi-rail power architecture meant to support both high-current motors and low-voltage control systems.

The final system integrates machine vision, real-time prediction algorithms, communication, and actuator control into a single robotic platform that can support automated table tennis training.

6.2 Evaluation of the Robotic System

The evaluation results showed that the suggested design successfully merges the major subsystems needed for a dynamic autonomous table tennis training system. The vision subsystem was capable of detecting and tracking the ball within successive image frames, providing the data required for state estimation and therefore trajectory prediction.

The trajectory prediction successfully used the mathematical models created during the modelling phase to estimate future ball positions and the landing locations. The integration of Kalman filtering improved robustness by reducing the influence of measurement noise and temporary detection loss. Also, the simulation showed the ability to model aerodynamic effects and predict ball behaviour under differing conditions.

The communication system between the Jetson Nano and the STM32 microcontroller was reliable. The separation of high-level vision system and the low-level control was confirmed to be an effective design decision, allowing complicated vision algorithms to operate separately from time-critical motor control functions.

Although the system achieved its primary objectives, multiple limitations were noticed during development. Accurate spin estimation will continue to be a challenging problem because of the size and speed's of the ball.

The reliability of the vision system is affected by environmental conditions such as lighting, camera placement, and motion blur. These factors may introduce measurement inaccuracies.

Overall, the developed system shows the feasibility of combining machine vision, physical modelling, and embedded control techniques to create an intelligent dynamic robotic training platform.

6.3 Lessons Learned

The development of this robotic system gave us valuable experience with the integration of mechanical, electrical, and software engineering fields, all within a single robotic system. One of the most important lessons learned was the how important a modular system design is. By separating the project into separate subsystems simplified the development process, testing, and troubleshooting, all while improving maintainability.

Another important lesson learned was how important individual simulation is before putting the entire system together. By checking mathematical models and prediction algorithms in its own simulation, many potential issues were noticed and resolved before integration with the final simulation. This reduced development time and improved confidence in the final system.

The project also showed the challenges associated with real-time computer vision applications. Accurate object detection, state estimation, and prediction need to be considered for processing limitations, sensor limitations, and environmental conditions. The development revealed that theoretical models need to be adapted to accommodate practical implementation challenges.

The project demonstrated the importance of iterative engineering design. Many aspects of the system evolved and changed throughout development as new problems were identified and solutions were created, then refined through testing and evaluation.

6.4 Recommendations

Multiple recommendations exist for further development and improvement of the proposed robotic table tennis training system.

The most significant recommendation is that future vision systems could incorporate stereo vision or multiple camera viewpoints to improve three-dimensional position estimation and reduce reconstruction errors.

From a software perspective, machine learning techniques could be used to adapt the launcher behaviour based on a player's performance and training focuses. This functionality would create a more personalised and intelligent training sessions ontop of being more dynamic.

Future work should fully integrate the hardware, not just rely on simulations, and then proceed with rigorous testing with real table tennis games. Testing over time with players of different skill levels would give useful feedback on system performance, usability, and training effectiveness.

6.5 Closing Remarks

This project showed how to design and build a robotic table tennis training system using machine vision, physical modeling, and embedded control systems. Although more work is needed before it can be used commercially, this project provides a solid technical base for future research and development in intelligent sports robotics.

Chapter A: Modelling Scripts

A.1 data gathering

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
BALL_DIAMETER_CM = 4.0
```

```
# camera intrinsics
```

```
mtx = np.array([
    [1050.0, 0, 640],
    [0, 1050.0, 360],
    [0, 0, 1]
])
```

```
def get_3d(xp,yp,diam_px):
```

```
    focal=(mtx[0,0]+mtx[1,1])/2
```

```
    z=(BALL_DIAMETER_CM*focal)/(diam_px+1e-6)
```

```
    x=((xp-mtx[0,2])*z)/mtx[0,0]
```

```
    y=((yp-mtx[1,2])*z)/mtx[1,1]
```

```
    return x,y,z
```

A.1.1 Camera Capture

```
import cv2
```

```
#capture = cv2.VideoCapture(0)
```

```
capture = cv2.VideoCapture("video1.mp4")# for videos
```

```
while True:
```

```
    #return each frame true or false
```

```
    ret, frame = capture.read()
```

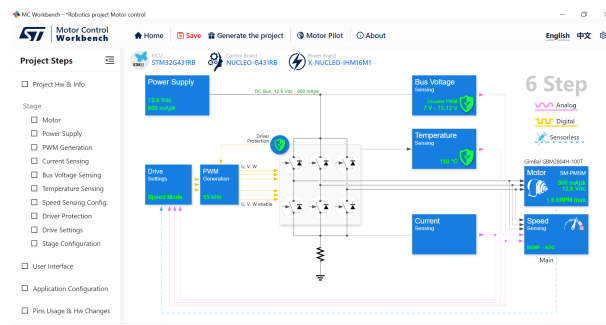


Figure A.1: Enter Caption

```

    if not ret:
        break

    cv2.imshow("Camera Feed", frame)

    #q for quit
    if cv2.waitKey(1) == ord('q'):
        break

capture.release()
cv2.destroyAllWindows()

```

A.1.2 Ball Detection

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

def findBall(frame, removeBG, roi, PixelPrediction=None):

    maskROI = np.zeros(frame.shape[:2], dtype=np.uint8)
    xROI, yROI, wROI, hROI = roi
    maskROI[yROI:yROI+hROI, xROI:xROI+wROI] = 255

    # Whats moving
    fg_mask = removeBG.apply(frame)
    _, fg_mask = cv2.threshold(fg_mask, 200, 255, cv2.THRESH_BINARY)

    # Find the white
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    lowerLimit = np.array([0, 0, 180])
    upperLimit = np.array([180, 75, 255])

    whiteMask = cv2.inRange(hsv, lowerLimit, upperLimit)

    combined = cv2.bitwise_and(whiteMask, fg_mask)
    combined = cv2.bitwise_and(combined, maskROI)

    contours, _ = cv2.findContours(
        combined,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    isBall=[]

    for c in contours:

        area=cv2.contourArea(c)

```

```

    if area<4 or area>1000:
        continue

    perimeter=cv2.arcLength(c,True)

    if perimeter==0:
        continue

    circularity = 4*np.pi*(area/(perimeter*perimeter))

    (x,y),radius=cv2.minEnclosingCircle(c)

    score = circularity*area

    if PixelPrediction is not None:
        dist=np.linalg.norm(np.array([x,y])-np.array(PixelPrediction))
        score*=1/(1+dist/50)

    if circularity>0.12:

        isBall.append({
            'pos':(int(x),int(y),int(radius*2)),
            'score':score
        })

    if not isBall:
        return None

    best=max(isBall,key=lambda x:x['score'])

    return best['pos']

```

A.1.3 3D Reconstruction

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

BALL_DIAMETER_CM = 4.0
# camera intrinsics
mtx = np.array([
    [1050.0, 0, 640],
    [0, 1050.0, 360],
    [0, 0, 1]
])

def get_3d(xp,yp,diam_px):

    focal=(mtx[0,0]+mtx[1,1])/2

```

```

z=(BALL_DIAMETER_CM*focal)/(diam_px+1e-6)

x=((xp-mtx[0,2])*z)/mtx[0,0]
y=((yp-mtx[1,2])*z)/mtx[1,1]

return x,y,z

```

A.1.4 Spin estimation

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal import savgol_filter

def normalize(vec, eps=1e-12):
    mag = np.linalg.norm(vec)
    if mag < eps:
        return None
    return vec / mag

def get_e_w(v_vec, a_vec, eps=1e-12):
    rou = 1.225          # air density (kg/m^3)
    r = 0.02             # ball radius (m)
    A = np.pi * r**2    # cross-sectional area
    m = 0.0027           # ball mass (kg)
    g_vec=np.array([0.0, 0.0, -9.81])
    C_L = 0.5            # TEMP lift coefficient estimate
    C_D = 0.5            # TEMP Drag coefficient estimate
    C1 = rou * A * C_D / (2 * m)
    C2 = rou * A * C_L / (2 * m)

    v_mag = np.linalg.norm(v_vec)
    if v_mag < eps:
        return None

    denom = C2 * v_mag**3
    if abs(denom) < eps:
        return None

    e_w = np.cross(v_vec, a_vec - g_vec+C1*v_mag*v_vec) / denom

    if not np.all(np.isfinite(e_w)):
        return None

    return e_w

if __name__ == "__main__":

```

```

# -----
# Load trajectory
# -----

df = pd.read_csv("ball_trajectory.csv")

t_vec = df["t_sec"].values
x_vec = df["x_cm"].values
y_vec = df["y_cm"].values
z_vec = df["z_cm"].values

# -----
# Smooth + differentiate
# -----

dt = np.mean(np.diff(t_vec))

window = 9
poly = 3

# velocity
vx = savgol_filter(x_vec, window, poly, deriv=1, delta=dt)
vy = savgol_filter(y_vec, window, poly, deriv=1, delta=dt)
vz = savgol_filter(z_vec, window, poly, deriv=1, delta=dt)

# acceleration
ax = savgol_filter(x_vec, window, poly, deriv=2, delta=dt)
ay = savgol_filter(y_vec, window, poly, deriv=2, delta=dt)
az = savgol_filter(z_vec, window, poly, deriv=2, delta=dt)

v_arr = np.stack([vx, vy, vz], axis=1)
a_arr = np.stack([ax, ay, az], axis=1)

# -----
# Compute e_w
# -----

t_plot = []
ew_x = []
ew_y = []
ew_z = []

for t, v_vec, a_vec in zip(t_vec, v_arr, a_arr):

    e_w = get_e_w(v_vec, a_vec)

    if e_w is not None:
        e_w = normalize(e_w)

        if e_w is not None:
            t_plot.append(t)
            ew_x.append(e_w[0])

```

```

        ew_y.append(e_w[1])
        ew_z.append(e_w[2])

# -----
# Plot results
# -----

plt.figure(figsize=(8,5))

plt.plot(t_plot, ew_x, label="e_w_x")
plt.plot(t_plot, ew_y, label="e_w_y")
plt.plot(t_plot, ew_z, label="e_w_z")

plt.xlabel("Time (s)")
plt.ylabel("Spin axis components")
plt.title("Estimated Spin Axis Components vs Time")

plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig("e_w_plot.png", dpi=300)
plt.close()

```

Chapter A: Work Breakdown

STUDENT	Deliverable 1	Deliverable 2	Deliverable 3	Deliverable 4	Deliverable 5	Deliverable 6	Presentation
Andiswa	Research Question; Research Objectives; Benefits of Study; Delimitation of Study	Introduction; Human-Machine Feedback loop	Definitions; Algorithm to convert target ball speed, spin and direction to motors	Electrical Design; Microcontroller Software Design; Conclusion	Definition of Testing Objectives and Scope; Test Cases and Scenarios; Simulation-Based Testing; Execute Tests and Analyse Results; Results; Hardware-Based Testing		Testing; Validation
Brendon	Problem statement; Research Objectives; Benefits of Study; Delimitation of Study	Introduction; Trajectory Modelling; conclusion	Definitions; Introduction; Camera capture; Ball detection 2D; 3D reconstruction	Introduction; Physical Design; Electrical Design; Vision Control Design	CAD->STEP designs	Conclusion	Conclusion; Recommendations; Challenges; Demo

Joshua	Introduction; Background; Research Objectives; Benefits of Study; Delimitation of Study	Introduction; Impact Dynamics Between Table Tennis Ball and Racket	Definitions; 3D velocity, Acceleration and spin calculation; Trajectory and Spin Prediction; Simulation of Ball-Racket Impact; Determination of racket orientation for target-constrained return	Design requirements; Physical Design; Electrical Design; Computational Software Design	Introduction; Definition of Testing Objectives and Scope; Test Cases and Scenarios; Results; Conclusion; Hardware-Based Testing; Execute Tests and Analyse Results; Simulation-Based Testing		Technical overview; Demo
Thandiwe	Research Objectives; Benefits of Study; Delimitation of Study	Introduction; Table Tennis	Algorithm list	Physical Design			Executive Summary

Bibliography

- Aakanksha, Kumar, A. & N., R. A. (2025), 'Ball trajectory and spin analysis from asynchronous videos', *IEEE Sensors Letters* **9**(3), 1–4.
- Avizienis, A., Laprie, J.-C., Randell, B. & Landwehr, C. (2004), 'Basic concepts and taxonomy of dependable and secure computing', *IEEE Trans. Dependable and Secure Comput.* **1**(1), 11–33.
- Bar-Shalom, Y., Li, X. R. & Kirubarajan, T. (2001), *Estimation with Applications to Tracking and Navigation*, Wiley.
- Büchler, D., Guist, S., Calandra, R., Berenz, V., Schölkopf, B. & Peters, J. (2022), 'Learning to play table tennis from scratch using muscular robots', *IEEE Transactions on Robotics* **38**(6), 3850–3860.
- Chapra, S. C. & Canale, R. P. (2015), *Numerical Methods for Engineers*, McGraw-Hill Education.
- D'Ambrosio, D. B. et al. (2024), 'Achieving human-level competitive robot table tennis', *arXiv preprint arXiv:2408.03906*.
- Dittrich, A., Schneider, J., Guist, S., Gurtler, N., Ott, H., Steinbrenner, T., Scholkopf, B. & Buchler, D. (2022), 'Aimy: An open-source table tennis ball launcher for versatile and high-fidelity trajectory generation', *Proceedings - IEEE International Conference on Robotics and Automation 2023-May*, 3058–3064.
URL: <https://arxiv.org/pdf/2210.06048>
- Eggert, D. W., Lorusso, A. & Fisher, R. B. (1997), 'Estimating 3-d rigid body transformations: A comparison of four major algorithms', *Machine Vision and Applications* **9**(5), 272–290.
URL: <https://doi.org/10.1007/s001380050048>
- Emami, S., Kardan, I. & Akbarzadeh, A. (2024), Optimization of a delta ping pong robot with an innovative approach to workspace design, in '2024 10th International Conference on Artificial Intelligence and Robotics (QICAR)', pp. 245–249.
- Glickman, M. & Sharot, T. (2025), 'How human–ai feedback loops alter human perceptual, emotional and social judgements', *Nature Human Behaviour* **9**(2), 345–359.
URL: <https://doi.org/10.1038/s41562-024-02077-2>
- Gomez-Gonzalez, S., Neumann, G., Scholkopf, B. & Peters, J. (2020), 'Adaptation and robust learning of probabilistic movement primitives', *IEEE Transactions on Robotics* **36**, 366–379.
URL: <https://ieeexplore.ieee.org/abstract/document/9020014>
- Gómez-Carmona, O., Casado-Mansilla, D., de Ipiña, D. L. & García-Zubia, J. (2024), 'Human-in-the-loop machine learning: Reconceptualizing the role of the user in interactive approaches', *Internet of Things* **25**, 101048.
URL: <https://www.sciencedirect.com/science/article/pii/S2542660523003712>
- Hartley, R. & Zisserman, A. (2003), *Multiple View Geometry in Computer Vision*, 2 edn, Cambridge University Press.
- Hubbard, M. & Stronge, W. J. (2001), 'Bounce of hollow balls on flat surfaces', *Sports Engineering* **4**.
- Hunt, A. & Thomas, D. (1999), *The Pragmatic Programmer*, Addison-Wesley.

- IEC (2016), 'IEC 61140: Protection against electric shock – common aspects for installation and equipment'.
- International Table Tennis Federation (2024), 'ITTF handbook'. ITTF Equipment Committee.
- ISO/IEC/IEEE (2018), 'ISO/IEC/IEEE 29148:2018 systems and software engineering – life cycle processes – requirements engineering'.
- Ke, Q., Liu, J., Bennamoun, M., An, S., Sohel, F. & Boussaid, F. (2018), *Computer Vision for Human–Machine Interaction*, 1st edn, Academic Press, United States of America, pp. 127–145. Publisher Copyright: © 2018 Elsevier Ltd. All rights reserved.
- Khulief, Y. A. (2012), 'Modeling of impact in multibody systems: An overview', *Journal of Computational and Nonlinear Dynamics* **8**(2), 021012.
URL: <https://doi.org/10.1115/1.4006202>
- Koç, O., Maeda, G. & Peters, J. (2018), 'Online optimal trajectory generation for robot table tennis', *Robotics and Autonomous Systems* **105**, 121–137.
URL: <https://www.sciencedirect.com/science/article/pii/S0921889017306164>
- Kumar, S., Datta, S., Singh, V., Datta, D., Singh, S. & Sharma, R. (2024), 'Applications, challenges, and future directions of human-in-the-loop learning', *IEEE Access* **PP**, 1–1.
- Kusubori, S., Yoshida, K. & Sekiya, H. (2024), 'The functions of spin on shot trajectory in table tennis'.
URL: <https://ojs.ub.uni-konstanz.de/cpa/article/view/5370>
- Lee, E. A. (2008), Cyber-physical systems: Design challenges, in 'Proc. IEEE ISORC'.
- Lee, M. J. C., Ozaki, H. & Goh, W. X. (2019), 'Speed and spin differences between the old celluloid versus new plastic table tennis balls and the effect on the kinematic responses of elite versus sub-elite players', *International Journal of Racket Sports Science* **1**, 26–36.
URL: <https://digibug.ugr.es/handle/10481/57324>
- Lin, H.-I., Yu, Z. & Huang, Y.-C. (2020), 'Ball tracking and trajectory prediction for table-tennis robots', *Sensors* **20**(2).
URL: <https://www.mdpi.com/1424-8220/20/2/333>
- Liu, J. W. S. (2000), *Real-Time Systems*, Prentice Hall.
- Mat Sanusi, K. A., Mitri, D. D., Limbu, B. & Klemke, R. (2021), 'Table tennis tutor: Forehand strokes classification based on multimodal data and neural networks', *Sensors* **21**(9).
URL: <https://www.mdpi.com/1424-8220/21/9/3121>
- Mülling, K., Kober, J., Kroemer, O. & Peters, J. (2013), 'Learning to select and generalize striking movements in robot table tennis', *International Journal of Robotics Research* **32**, 263–279.
URL: [/doi/pdf/10.1177/0278364912472380?download=true](https://doi/pdf/10.1177/0278364912472380?download=true)
- Nelder, J. A. & Mead, R. (1965), 'A simplex method for function minimization', *Computer Journal* **7**(4), 308–313.
- Nemire, K., Goettsche, M. & Bridgeman, B. (1991), 'Automated system for ball launching, visual occlusion, and data acquisition in a ball-hitting task', *Behavior Research Methods, Instruments, & Computers* **23**, 36–44.
URL: <https://link.springer.com/article/10.3758/BF03203333>

Ning, T., Wang, C., Fu, M. & Duan, X. (2023), 'A study on table tennis landing point detection algorithm based on spatial domain information', *Scientific Reports* **13**, 20656.

URL: <https://www.nature.com/articles/s41598-023-42966-6#Sec7>

Nocedal, J. & Wright, S. J. (2006), *Numerical Optimization*, 2 edn, Springer.

Ponnusamy, B., Yong, W. F. & Ahmad, Z. (2015), 'A low cost automated table tennis launcher', *ARPJ Journal of Engineering and Applied Sciences* **10**.

URL: https://www.researchgate.net/profile/Zulkifli-Ahmad-2/publication/280557460_A_low_cost_automated_table_tennis_launcher/low-cost-automated-table-tennis-launcher.pdf

Press, W. H., Teukolsky, S. A., Vetterling, W. T. & Flannery, B. P. (2007), *Numerical Recipes: The Art of Scientific Computing*, 3 edn, Cambridge University Press.

Rinaldi, R. G., Manin, L., Moineau, S. & Havard, N. (2019), 'Table tennis ball impacting racket polymeric coatings: Experiments and modeling of key performance metrics', *Applied Sciences* **9**(1).

URL: <https://www.mdpi.com/2076-3417/9/1/158>

Romero-Obon, M., Rouaz-El-Hajoui, K., Sancho-Ochoa, V., Vargas, R., Pérez-Lozano, P., Suñé-Pou, M. & García-Montoya, E. (2025), 'Human-in-the-loop ai use in ongoing process verification in the pharmaceutical industry', *Information* **16**(12).

URL: <https://www.mdpi.com/2078-2489/16/12/1082>

Rusdiana, A. & Ichsan, D. N. (2017), 'Rubber bat effect of maximal ball velocity in table tennis', *IOP Conference Series: Materials Science and Engineering* **180**(1), 012231.

URL: <https://doi.org/10.1088/1757-899X/180/1/012231>

Schafer, R. W. (2011), 'What is a Savitzky–Golay filter?', *IEEE Signal Processing Magazine* **28**(4), 111–117.

Schneider, R., Lewerentz, L., Kemnitz, S. & Schultz, C. (2022), 'Table tennis and physics', *Aerodynamics of Sports [Working Title]*.

Schörghuber, M. et al. (2021), 'Vision-based table-tennis robot control', *Mechatronics*.

Shi, Z., Zhu, C., Yang, F., Yan, J., Qin, Z., Shi, S. & Chen, Z. (2025), 'A universal model combining differential equations and neural networks for ball trajectory prediction'.

URL: <https://arxiv.org/abs/2503.18584>

Song, Q. & Lu, R. (2023), 'Prediction and analysis of table tennis movement trajectory based on artificial intelligence', *2023 International Conference on Blockchain Technology and Applications (ICBTA)* pp. 124–129.

URL: <https://ieeexplore.ieee.org/abstract/document/10387388>

Stewart, D. E. (2000), 'Rigid-body dynamics with friction and impact', *SIAM Review* **42**(1), 3–39.

URL: <https://doi.org/10.1137/S0036144599360110>

Stronge, W. J. (2000), *Impact Mechanics*, Cambridge University Press.

TAMAKI, S. & SAITO, H. (2013), 'A system that reconstructs a 3d trajectory of a table tennis ball for performance analysis in table tennis', *Journal of the Japan Society for Precision Engineering* **79**, 1102–1109.

Tanenbaum, A. S. & Wetherall, D. J. (2013), *Computer Networks*, 5 edn, Pearson.

- Toussaint, B. & Raison, M. (2026), 'Real-time algorithm for table tennis with a desktop robotic arm', *Robotics and Autonomous Systems* **197**, 105288.
URL: <https://www.sciencedirect.com/science/article/pii/S0921889025003859>
- Turgunov, J., Rakhimova, M., Saidov, B., Kholmatova, D. & Sd, D. (2025), *HUMAN-IN-THE-LOOP SYSTEMS FOR ETHICAL AI*.
- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E. H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J. & Fedus, W. (2022), 'Emergent abilities of large language models', *Transactions on Machine Learning Research* . Survey Certification.
URL: <https://openreview.net/forum?id=yzkSU5zdwD>
- Welch, G. & Bishop, G. (1995), An introduction to the Kalman filter, Technical Report TR 95-041, Univ. of North Carolina at Chapel Hill.
- Wiggins, A. (2017), *The Twelve-Factor App*. <https://12factor.net>.
- Yang, L., Zhang, H., Zhu, X. & Sheng, X. (2021), 'Ball motion control in the table tennis robot system using time-series deep reinforcement learning', *IEEE Access* **9**, 99816–99827.
URL: <https://ieeexplore.ieee.org/abstract/document/9467347>
- Yiu, C., Liu, Y., Park, W., Li, J., Huang, X., Yao, K., Gao, Y., Zhao, G., Chu, H., Zhou, J., Li, D., Li, H., Zhang, B., Chow, L., Huang, Y., Xu, Q. & Yu, X. (2025), 'Skin-interfaced multimodal sensing and tactile feedback system as enhanced human-machine interface for closed-loop drone control', *Science Advances* **11**(13), eadt6041.
URL: <https://www.science.org/doi/abs/10.1126/sciadv.adt6041>
- Zhang, Y., Xiong, R., Zhao, Y. & Wang, J. J. (2015), 'Real-time spin estimation of ping-pong ball using its natural brand', *IEEE Transactions on Instrumentation and Measurement* **64**, 2280–2290.